

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

=====

Дата генерации: 11.09.2026, 18:43:47
Файлов исходного кода: 87
Суммарный объём кода: 1.16 МВ
Глав/билетов в пособии: 30

=====

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

=====

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Патта (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-func]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

- 24. src/data/glossary.ts
- 25. src/data/graphContent.ts
- 26. src/data/quizzes.ts

Билеты (учебный контент)

- 27. src/data/tickets/ticket1_5.ts
- 28. src/data/tickets/ticket14_20.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterView.tsx
- 37. src/components/CompilerPanelContent.tsx
- 38. src/components/DfsBridgesSimulator.tsx
- 39. src/components/DijkstraViz.tsx
- 40. src/components/DPVisualizer.tsx
- 41. src/components/EulerSimulator.tsx
- 42. src/components/ExpressQuiz.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphTraversalViz.tsx
- 45. src/components/HeapViz.tsx
- 46. src/components/hints/demoKit.tsx
- 47. src/components/hints/demosExtra.tsx
- 48. src/components/hints/demosGraph.tsx
- 49. src/components/hints/demosStruct.tsx
- 50. src/components/hints/MiniDemo.tsx
- 51. src/components/hints/TermPopover.tsx
- 52. src/components/JohnsonViz.tsx
- 53. src/components/KosarajuViz.tsx
- 54. src/components/KruskalSimulator.tsx
- 55. src/components/MnemonicCards.tsx
- 56. src/components/MobileToc.tsx
- 57. src/components/Navbar.tsx

```
86. .github/workflows/deploy-pages.yml
87. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/87: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```
```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

name: Deploy app to GitHub Pages

on:

push:

branches:

```
- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
```



```
sudo apt-get update -qq
sudo apt-get install -y --no-install-recommen
# На Ubuntu политика ImageMagick иногда запре
# разрешаем то, что нужно пайплайну (text ->
for policy in /etc/ImageMagick-6/policy.xml /
    if [ -f "$policy" ]; then
        sudo sed -i 's/rights="none" pattern="\{P
    fi
done
convert -version
```

- name: Install dependencies
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt
run: npm run export:txt
- name: Step 2 – txt → png
env:
 EXPORT_DENSITY: \${github.event.inputs.densi
run: npm run export:png
- name: Step 3 – png → pdf
run: npm run export:pdf
- name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
- name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \`\${public}/export/full_code_all_pages
 echo "| \`\${public}/export/full_code_all_pages

index bdc3c33..ff49c0c 100644

--- a/.github/workflows/rebuild-pdf.yml

+++ b/.github/workflows/rebuild-pdf.yml

@@ -42,15 +42,15 @@ jobs:

steps:

- name: Checkout

- uses: actions/checkout@v4

+ uses: actions/checkout@v6

with:

fetch-depth: 0

persist-credentials: true

- name: Setup Node.js

- uses: actions/setup-node@v4

+ uses: actions/setup-node@v7

with:

- node-version: "22"

+ node-version: "24"

cache: npm

- name: Install ImageMagick + DeJaVu fonts

@@ -97,7 +97,7 @@ jobs:

} >> "\$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

name: full-code-export

path: |

@@ -107,7 +107,7 @@ jobs:

retention-days: 30

- name: Upload artifacts (PNG-страницы)

- uses: actions/upload-artifact@v4

+ uses: actions/upload-artifact@v7

with:

Универсальное пособие • полный исходный код

```
-----  
    <body class="bg-slate-950 text-slate-100 min-h-screen  
....
```

```
## `public/favicon.svg`
```

```
### Полное содержимое после изменений
```

```
````xml
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
....
```

```
Patch относительно `main`
```

```
````diff
```

```
diff --git a/public/favicon.svg b/public/favicon.svg  
new file mode 100644  
index 00000000..3bbbba0  
--- /dev/null  
+++ b/public/favicon.svg  
@@ -0,0 +1,5 @@  
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64  
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>  
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#  
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>  
+</svg>  
....
```

```
## `src/App.tsx`
```

```
### Полное содержимое после изменений
```

```
````tsx
```

```
import { useCallback, useEffect, useMemo, useState } from
import { chapters } from "../data/content";
```



```

 }, [goTo, next, prev]));

const progress = useMemo(() => ((index + 1) / chapters.length) * 100);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>

 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
 </>
) : (
 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
 <div className="hidden lg:block lg:w-80 shrink-0">
 <Sidebar selectedChapterId={activeChapterId}>
 </div>
 <main id="main-content" className="flex-1 min-w-0">
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between">
 <div className="min-w-0">
 <div className="text-[10px] uppercase">
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium">
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index}>
 </div>
 </main>
 </div>
)}
 </Navbar>
 </div>
);

```

```

 </footer>
 </div>
);
}
....

Patch относительно `main`

```diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
-  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+  const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
    </main>
    </div>
  </>
-  ) : (
+  ) : activeTab === "simulator" ? (
    <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
      <SimulatorHub
        onOpen={setModalVizId}
@@ -105,6 +106,8 @@ export default function App() {
      </>
    </main>
  ) : (
    <main>

```

```

<div className="flex items-center gap-3 w-full" >
  <div className="bg-gradient-to-tr from-indigo-400 to-indigo-600" >
    <Sparkles className="w-6 h-6" />
  </div>
  <div className="min-w-0 flex-1" >
    <h1 className="text-xl font-extrabold text-slate-900" >
      Универсальное пособие
    </h1>
    <p className="text-xs text-indigo-400 font-medium" >
      Подготовка к экзаменам: Алгоритмы, Структуры данных
    </p>
  </div>
</div>

```

```

<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
        activeTab === 'guide'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center gap-2
      text-sm font-medium transition-colors duration-200 whitespace-nowrap ${
        activeTab === 'simulator'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
          : 'text-slate-400 hover:text-white'
      }`}
  >
    <CodeIcon className="w-4 h-4" /> Симулятор
  </button>
</div>

```

```

        </a>
        <button
          id="download-html-btn"
          onClick={saveBook}
          disabled={busy}
          className="flex items-center justify-center
            er:bg-amber-600/20 border border-amber-500/
          title="Скачать всё пособие одним автономным
        >
          {busy ? <Loader2 className="w-4 h-4 animate
        </button>
      </div>
    </div>
  </header>
);
};
....

```

Patch относительно `main`

```

````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

```

-----  
### Полное содержимое после изменений

```
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
  if (!runtimePromise) {
    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
      .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
  }
  return runtimePromise;
}
```

```

        const captured = [...stdout, ...stderr].join("");
        setOutput(`${captured}${captured && !captured.end`
        setRuntimeMessage("Не удалось выполнить код – про
    } finally {
        setRunning(false);
    }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
    setCode(STARTER_CODE);
    setStdin("");
    setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
    <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
        <div className="max-w-6xl mx-auto">
            <div className="flex flex-col sm:flex-row sm:items-center">
                <div>
                    <div className="inline-flex items-center gap-2">
                        <Terminal className="w-4 h-4" /> Боковая панель
                    </div>
                    <h2 className="text-2xl sm:text-3xl font-extrabold">
                        <p className="text-slate-400 mt-2 max-w-2xl">
                            Пишите небольшой код и запускайте его прямо в браузере,
                            который переводит CPython в WebAssembly.
                        </p>
                    </div>
                    <a
                        href="https://pyodide.org/"
                        target="_blank"
                        rel="noreferrer"
                        className="inline-flex items-center gap-1.5"
                    >
                        Как это работает <ExternalLink className="w-4 h-4" />
                    </a>
                </div>
            </div>
        </div>
    </main>

```

```

    }}
    spellCheck={false}
    aria-label="Код Python"
    className="block w-full min-h-[360px] res
    -none focus:ring-2 focus:ring-inset focus:r
    placeholder="Напишите Python-код..."
  />

  <div className="border-t border-slate-800 b
    <label className="block px-4 pt-3 text-[1
      Ввод для input() • по одной строке на к
    </label>
    <textarea
      id="python-stdin"
      value={stdin}
      onChange={(event) => setStdin(event.tar
      spellCheck={false}
      rows={2}
      placeholder={'Например:\nАлиса'}
      className="block w-full resize-y bg-tra
    eholder:text-slate-700"
    />
  </div>
</section>

<section className="rounded-2xl border border
  <div className="flex items-center justify-b
    <div className="flex items-center gap-2 t
      <Terminal className="w-4 h-4 text-emera
    </div>
    <span className={`inline-flex items-cente
      {runtimeReady && <CheckCircle2 classNam
      {runtimeReady ? "готов" : "не загружен"
    </span>
  </div>
  <pre className="min-h-[360px] max-h-[560px]
  x] leading-6 text-slate-300">
    {output}
  
```

Универсальное пособие • полный исходный код

```
-----  
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/pyodide.js`  
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.23.0/full/index.html`  
+  
+const STARTER_CODE = `# Первый запуск загрузит Python и выполнит код  
+name = "алгоритмы"  
+  
+for number in range(1, 4):  
+    print(f"{number}. Учим {name}!")  
+`;  
+  
+type PyodideRuntime = {  
+  runPythonAsync: (source: string) => Promise<unknown>  
+  setStdout: (options: { batched: (message: string) => void }) => void  
+  setStderr: (options: { batched: (message: string) => void }) => void  
+  setStdin: (options: { stdin: () => string | number | null }) => void  
+};  
+  
+type PyodideModule = {  
+  loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>  
+};  
+  
+let runtimePromise: Promise<PyodideRuntime> | null = null;  
+  
+function getPythonRuntime() {  
+  if (!runtimePromise) {  
+    runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL, {  
+      (module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL })  
+    });  
+  }  
+  return runtimePromise;  
+}  
+  
+function errorText(error: unknown) {  
+  if (error instanceof Error) return error.message;  
+  return String(error);  
+}  
+  
+export function PythonCompiler() {
```



```

+
+  const reset = () => {
+    setCode(STARTER_CODE);
+    setStdin("");
+    setOutput("Нажмите «Запустить», чтобы увидеть резу.
+  };
+
+  return (
+    <main className="max-w-screen-2xl mx-auto px-4 sm:p
+      <div className="max-w-6xl mx-auto">
+        <div className="flex flex-col sm:flex-row sm:i
+          <div>
+            <div className="inline-flex items-center g
+              <Terminal className="w-4 h-4" /> Боковая
+            </div>
+            <h2 className="text-2xl sm:text-3xl font-e
+            <p className="text-slate-400 mt-2 max-w-2x
+              Пишите небольшой код и запускайте его пр
+              который переводит CPython в WebAssembly.
+            </p>
+          </div>
+          <a
+            href="https://pyodide.org/"
+            target="_blank"
+            rel="noreferrer"
+            className="inline-flex items-center gap-1.5
+          >
+            Как это работает <ExternalLink className="
+          </a>
+        </div>
+
+        <div className="grid xl:grid-cols-[minmax(0,1.
+          <section className="rounded-2xl border borde
+            <div className="flex flex-wrap items-cente
+              <div className="flex items-center gap-2"
+                <span className="w-2.5 h-2.5 rounded-f
+                <span className="text-sm font-bold tex
+                <span className="text-[11px] text-slate

```

```

+
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+             <label className="block px-4 pt-3 text-slate-700" style={{display: block; margin-bottom: 10px;}}>
+                 Ввод для input() • по одной строке на строку
+             </label>
+             <textarea
+                 id="python-stdin"
+                 value={stdin}
+                 onChange={(event) => setStdin(event.target.value)}
+                 spellCheck={false}
+                 rows={2}
+                 placeholder={'Например:\nАлиса'}
+                 className="block w-full resize-y bg-transparent border border-slate-800"
+                 style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}
+             />
+         </div>
+     </section>
+
+     <section className="rounded-2xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 20px; margin: 10px 0;}}>
+         <div className="flex items-center justify-between" style={{border: 1px solid #333; border-radius: 10px; padding: 10px; margin: 10px 0;}}>
+             <div className="flex items-center gap-2" style={{flex: 1;}}>
+                 <Terminal className="w-4 h-4 text-emerald-500" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}} />
+             <span className={`inline-flex items-center gap-2`} style={{margin: 10px 0;}}>
+                 {runtimeReady && <CheckCircle2 className="text-emerald-500" style={{height: 1em; width: 1em;}} />}
+                 {runtimeReady ? "готов" : "не загружен"}
+             </span>
+         </div>
+         <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+             {output}
+         </pre>
+         <div className="border-t border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>
+             </div>
+     </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3" style={{margin-top: 10px;}}>
+     <div className="flex gap-2 rounded-xl border border-slate-800 p-4" style={{border: 1px solid #333; border-radius: 10px; margin: 10px 0;}}>

```

```
plugins: [react(), tailwindcss(), viteSingleFile()],
resolve: {
  alias: {
    "@": path.resolve(__dirname, "src"),
  },
},
server: {
  host: "0.0.0.0",
  port: 5173,
  strictPort: true,
  // предпросмотр может открываться через внешний про
  allowedHosts: true,
},
preview: {
  host: "0.0.0.0",
  port: 4173,
  strictPort: true,
  allowedHosts: true,
},
});
```
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
```

2. ****Структура JSON/TS:**** Каждая новая малая страница должна иметь структуру `{ "content": ... }` для вставки в `src/data/content.ts`.
3. ****Строгая структура контента:****
 - Заголовок с цветным бейджем (например, `bg-indigo-100`)
 - Определение/правило в центре (`border-blue-500` или `border-blue-500/2px`)
 - Обязательная сетка аналогий (2 колонки: неформальный и формальный язык)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы. Используй только классы из `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/87: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТР0К: 14 | РАЗМЕР: 6

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, >
    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/87: metadata.ison

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 31

```
{
  "name": "Remix Remix: ExamPrep Reader",
```

```
"devDependencies": {
  "@tailwindcss/vite": "4.1.17",
  "@types/node": "22.19.17",
  "@types/pdfkit": "^0.17.6",
  "@types/react": "19.2.7",
  "@types/react-dom": "19.2.3",
  "@vitejs/plugin-react": "5.1.1",
  "esbuild": "^0.28.2",
  "tailwindcss": "4.1.17",
  "typescript": "5.9.3",
  "vite": "7.3.2",
  "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и структурам данных",
}
```

ФАЙЛ 7/87: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1.2 KB

algv0 – Универсальное пособие по алгоритмам и структурам данных

Интерактивная web-читалка (React + Vite + Tailwind) с подсветкой кода и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash
npm install
npm run dev # предпросмотр на http://0.0.0.0:5174
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает **весь** исходный код репозитория в PDF-файл.

- * ****Содержание**** – на десктопе боковая панель с поиском на телефоне – липкая строка «Содержание · N из M» и в
- * ****Переходы между темами**** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиату
- * ****Всплывающие подсказки по терминам**** – текст главы а известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается кар мини-анимацией и кнопкой «Показать симулятор» (симуля Словарь – `src/data/glossary.ts`, мини-анимации – `src
- * ****Реестр интерактивов**** – `src/components/vizRegistry и для панели под главой, и для модальки из подсказки.

Структура

...

- src/
 - App.tsx – оболочка, привязка глав к визуалу
 - components/ – интерактивные симуляторы (Дейкстра, сортировка)
 - data/ – контент пособия (главы/билеты)
- scripts/export/ – пайплайн код → txt → png → pdf
- public/export/ – сгенерированные TXT и PDF (обновление)
- .github/workflows/ – автоматизация

— — —

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструмен
азуемые теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `<h2>` и
- * Подзаголовки: Используют тег `<h3>`.

Текст и параграфы

- * Обычный текст содержится в тегах ``<r>``.

Универсальное пособие • полный исходный код

```
-----
> * **Аналогия** – в HTML главы есть блок «Аналогия ...»
> * **2D** – к главе привязан интерактивный визуализатор
>
> Всего глав в пособии: **25**.
```

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|--------------------|-----------|----------------------|
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| Билет | Тема | Статус |
|---------------|--------------------------------|---|
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. Контент в холостую. |
| **7** | Планарность и раскраска графов | Номерной главы нет; есть блок аналогии**. |
| **11** | Мосты в графах | Номерной главы нет; есть блок аналогии**. |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; есть блок аналогии**. |

Дополнительно – «осиротевшие» визуализаторы без глав (контент в холостую)

- * `stack-dfs` → `StackViz.tsx` – ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – ****Очередь и BFS****
- * `heap-beam-search` → `HeapViz.tsx` – ****Куча и лучевой поиск****
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – ****Дерево отрезков****
- * `dynamic-programming` → `DPVisualizer.tsx` – ****Динамическое программирование****

| | | | |
|----|---|---|---|
| 9 | 9. Графы. Топологическая сортировка | 1 | - |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | 1 | - |
| 12 | 12. Графы. Точки сочленения | 1 | - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | - |
| 16 | 16. Графы. Кратчайший путь. Флойд | - | - |
| 17 | 17. Графы. Алгоритм Джонсона | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - |
| 22 | 22. Z-функция | 4 | - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - |
| - | Обзор: Кратчайшие пути и Графы | - | - |
| - | Алгоритмы на карте | - | - |
| - | Эйлер: Путь ≠ Цикл | - | - |
| - | Мосты в графах: код + визуализация | 1 | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш
3. ****Добавить 2D**** к 4 главам «Уровня 4» — минимум `inli`
4. ****Решить судьбу `alg-map`**** — раскрыть или удалить.

ФАЙЛ 9/87: `tsconfig.json`

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
```



```
-----
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algov0/, локально
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

```

const onKey = (e: KeyboardEvent) => {
  const target = e.target as HTMLElement | null;
  if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
  if (e.metaKey || e.ctrlKey || e.altKey) return;
  if (sideOpen) return;
  if (e.key === "ArrowRight" && next) goTo(next.id);
  if (e.key === "ArrowLeft" && prev) goTo(prev.id);
};
window.addEventListener("keydown", onKey);
return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev, sideOpen]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <>
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />
          <div className="flex flex-col lg:flex-row max-w-7xl mx-auto">
            <div className="hidden lg:block lg:w-80 shrink-0">
              <Sidebar selectedChapterId={activeChapterId} />
            </div>
            <div className={`flex flex-1 min-w-0 w-full`} >
              <main id="main-content" className={`flex flex-col lg:max-w-none lg:border-r lg:border-slate-800`} >
                <div className="flex items-center justify-between">
                  <div className="min-w-0">

```

```

    })
  </div>

  {!sideOpen && (
    <div className="hidden xl:flex fixed right-0 top-0
      <button
        onClick={() => setSideOpen(true)}
        className="writing-vertical bg-emerald-500 text-white p-2
ow-xl shadow-emerald-900/30 flex flex-col items-center justify-center
        style={{ writingMode: "vertical-rl", textOrientation: "mixed" }}
        title="Открыть компилятор 50/50 – слева"
      >
        <Terminal className="w-4 h-4 rotate-90" />
        <span className="tracking-widest">КОМПИЛЯТОР</span>
        <span className="text-[10px] opacity-0.5">50/50</span>
      </button>
    </div>
  )}
</div>

  {!sideOpen && (
    <button
      onClick={() => setSideOpen(true)}
      className="lg:hidden fixed bottom-6 right-6 bg-emerald-500 text-white p-2
shadow-emerald-900/30 transition-colors"
      title="Открыть компилятор"
    >
      <Terminal className="w-6 h-6" />
    </button>
  )}

<div className="lg:hidden">
  <SideCompilerDrawer
    chapterId={activeChapter.id}
    open={sideOpen}
    onClose={() => setSideOpen(false)}
    onOpenFull={() => {
      setSideOpen(false);
    }}
  />
</div>

```

```

-----
const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/g, '\\$&');

// Проверяем, поддерживает ли движок lookbehind с \p{L}
let supportsLookbehind = true;
try {
  // eslint-disable-next-line @typescript-eslint/no-unnecessary-type-assertion
  new RegExp("(?<![\\p{L}])test", "giu");
} catch {
  supportsLookbehind = false;
}

/** Одна большая регулярка из всех меток: длинные раньше коротких */
function buildRegex(): RegExp {
  // Сортируем по длине убыв., чтобы длинные метки имели приоритет
  const sorted = [...GLOSSARY_LABELS].sort((a, b) => b.length - a.length);
  const alternation = sorted.map((l) => escapeRe(l.label));
  if (supportsLookbehind) {
    // границы «не буква/цифра» – \b не работает с кириллицей
    return new RegExp(`(?<![\\p{L}\\p{N}_-])(${alternation})`);
  }
  // Фолбэк без lookbehind: захватываем границы в группу
  return new RegExp(`^(^|[\\p{L}\\p{N}_-])(${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l.label, l.id]));

/**
 * Проходит по тексту главы и оборачивает известные термины в 
 * <span class="term-hint" data-term-id="...">, чтобы на странице
 * повесить всплывающую карточку (как превью ссылок в Википедии)
 */
export function annotateTerms(root: HTMLElement): void {
  if (!cachedRegex) cachedRegex = buildRegex();
  const re = cachedRegex;

  const perTerm = new Map<string, number>();
  const perSurface = new Map<string, number>();

```

```

const usedSurface = perSurface.get(surface) ?? 0;
if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE)
  perTerm.set(id, usedTerm + 1);
perSurface.set(surface, usedSurface + 1);

// m.index указывает на начало всего совпадения (включая префикс)
const matchStart = hasPrefixGroup ? m.index + prefix.length : m.index;
const matchEnd = matchStart + surfaceRaw.length;

if (matchStart > last) frag.appendChild(document.createTextNode(text.slice(last, matchStart)));
// Если был префикс (пробел/знак), он уже в предыдущем фрагменте
if (hasPrefixGroup && prefix && last === m.index)
  // prefix уже будет вставлен как часть slice выше
  // поэтому нужно вставить prefix отдельно если он не пуст
  // Вставим его как текст перед термином.
  if (prefix) frag.appendChild(document.createTextNode(prefix));
} else if (hasPrefixGroup && prefix && matchStart === m.index)
  // already handled
}
const span = document.createElement("span");
span.className = "term-hint";
span.dataset.termId = id;
span.tabIndex = 0;
span.setAttribute("role", "button");
span.setAttribute("aria-label", `Подсказка: ${surfaceRaw}`);
span.textContent = surfaceRaw;
frag.appendChild(span);
last = matchEnd;
}

if (last === 0) continue;
if (last < text.length) frag.appendChild(document.createTextNode(text.slice(last, text.length)));
textNode.parentNode?.replaceChild(frag, textNode);
}
}

/**
 * Навешивает разметку терминов на контейнер при смене

```

ФАЙЛ 13/87: src/hooks/useVizControl.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 35 | РАЗМЕР: 1.3 KB

```
import { useEffect } from "react";
```

```
type Action = "step" | "play" | "pause" | "reset" | "prev";
```

```
/**
```

```
 * Позволяет боковой панели управлять демо: Шаг, Пуск, Пауза, Сброс, Предыдущий
```

```
 * Визуализация подписывается: useVizControl(chapterId, handlers)
```

```
 */
```

```
export function useVizControl({
```

```
  chapterId: string,
```

```
  handlers: {
```

```
    onStep?: (line?: number) => void;
```

```
    onPlay?: () => void;
```

```
    onPause?: () => void;
```

```
    onReset?: () => void;
```

```
    onPrev?: () => void;
```

```
  };
```

```
) {
```

```
  useEffect(() => {
```

```
    const h = (e: Event) => {
```

```
      const d = (e as CustomEvent).detail;
```

```
      if (!d) return;
```

```
      if (d.chapterId && d.chapterId !== chapterId) return;
```

```
      const action: Action = d.action;
```

```
      if (action === "step") handlers.onStep?.(d.line);
```

```
      else if (action === "play") handlers.onPlay?.();
```

```
      else if (action === "pause") handlers.onPause?.();
```

```
      else if (action === "reset") handlers.onReset?.();
```

```
      else if (action === "prev") handlers.onPrev?.();
```

```
    };
```

```
    window.addEventListener("viz:control", h as EventListener);
```

```
    return () => window.removeEventListener("viz:control", h);
```

```
  }, [chapterId, handlers.onStep, handlers.onPlay, handlers.onPause, handlers.onReset, handlers.onPrev]);
```

```
}
```

```
        window.dispatchEvent(  
            new CustomEvent("viz:sync", {  
                detail: { chapterId, vars: clean, line },  
            })  
        );  
    }, [chapterId, JSON.stringify(vars), line]);  
}
```

ФАЙЛ 15/87: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 K

```
@import "tailwindcss";  
  
@layer utilities {  
    .neon-glow-blue {  
        box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);  
    }  
    .neon-glow-emerald {  
        box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);  
    }  
    .neon-glow-rose {  
        box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);  
    }  
    .neon-glow-yellow {  
        box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);  
    }  
    .no-scrollbar::-webkit-scrollbar {  
        display: none;  
    }  
    .no-scrollbar {  
        scrollbar-width: none;  
    }  
}  
  
@keyframes pulse-gold {
```

```
    animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
    cursor: help;
    border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
    color: #c7d2fe;
    border-radius: 3px;
    padding: 0 1px;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
    background: rgba(99, 102, 241, 0.22);
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
```



```
.chapter-body :where(.text-3xl) {
  font-size: 1.4rem !important;
}
.chapter-body :where(.text-2xl) {
  font-size: 1.2rem !important;
}
.chapter-body :where(.text-xl) {
  font-size: 1.05rem !important;
}
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
  from {
    width: 0%;
  }
  to {
    width: 100%;
  }
}
```

```
declare module '*.jpg' {
  const src: string;
  export default src;
}
declare module '*.jpg?url' {
  const src: string;
  export default src;
}
declare module '*.jpeg' {
  const src: string;
  export default src;
}
declare module '*.png' {
  const src: string;
  export default src;
}
```

ФАЙЛ 19/87: src/viz/VizObject.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 94 | РАЗМЕР: 4.1 KB

```
// Базовый объект визуализации с двумя наследниками
export abstract class Visualization {
  abstract getType(): string;
  abstract describe(): string;
}
```

```
// Наследник 1: код реализации – строки которые видит п
export class CodeImpl extends Visualization {
  lines: string[];
  constructor(lines: string[]) {
    super();
    this.lines = lines;
  }
  getType() { return "code"; }
```

```
// Парсит строку вида "i, j = 0, 0" или "a, b, c, d = 1"
export function parseAssignments(line: string): Record<
  const out: Record<string, number | string> = {};
  // убрать коммент
  const code = line.split("#")[0].trim();
  if (!code) return out;
  if (!code.includes("=")) return out;
  const [left, right] = code.split("=").map(s => s.trim)
  if (!left || !right) return out;
  // левая часть: "i, j" или "a, b, c, d" или "ans"
  const leftVars = left.split(",").map(s => s.trim()).f
  // правая часть: "0, 0" или "4, 1, 3, 2" или "min(a,b
  // пробуем распарсить как числа
  if (right.startsWith("min") || right.startsWith("max"
    // для конструкции ans = min(a,b,c,d) – не парсим к
    // оставляем как строку, но подсветка будет по a,b,
    return out;
  }
  const rightVals = right.split(",").map(s => s.trim())
  // если одно значение и несколько vars: "a,b,c,d = 1"
  // если количество совпадает
  if (leftVars.length === rightVals.length) {
    for (let i = 0; i < leftVars.length; i++) {
      const k = leftVars[i];
      const vStr = rightVals[i];
      const num = Number(vStr);
      out[k] = Number.isNaN(num) ? vStr : num;
    }
  } else if (leftVars.length === 1 && rightVals.length
    const num = Number(rightVals[0]);
    out[leftVars[0]] = Number.isNaN(num) ? rightVals[0]
  } else if (rightVals.length === 1 && leftVars.length
    // "i, j = 0" – распарсить как оба = 0?
    const num = Number(rightVals[0]);
    const val = Number.isNaN(num) ? rightVals[0] : num;
    for (const k of leftVars) out[k] = val;
  }
}
```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    Аналогия 1: Школьные линейки
  </div>
  <p class="text-slate-300 text-sm mb-2" style="color: #a0c0ff; font-size: 0.9em; margin-bottom: 10px;">
    отрезок длины 7. Ты берешь две заготовленные заготовки, берешь отрезок длины 7 (перекрывая друг друга по 1 см), ничего складывать, просто пересекаем куски.
  </p>
</div>
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    Аналогия 2: Ступени
  </div>
  <p class="text-slate-300 text-sm mb-2" style="color: #a0c0ff; font-size: 0.9em; margin-bottom: 10px;">
    нужно покрыть отрезок, мы берем две самые большие ступени
  </p>
</div>
<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <summary class="p-4 font-bold text-slate-300" style="color: #a0c0ff; font-weight: bold; padding: 10px;">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300" style="padding: 10px;">
    <pre class="bg-slate-950 p-4 rounded" style="background-color: #000; color: #a0c0ff; padding: 10px;">
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);</pre>
    </div>
  </details>
</div>
</div>
</section>`,
  },
  {

```

```

<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate-300"
    -b border-slate-700/50">
      Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300"
      <pre class="bg-slate-950 p-4 rounded"
pair<Node*, Node*> split(Node* t, int x) {
  if (!t) return {nullptr, nullptr};
  if (t->val <= x) {
    auto [L, R] = split(t->right, x);
    t->right = L;
    return {t, R};
  } else {
    auto [L, R] = split(t->left, x);
    t->left = R;
    return {L, t};
  }
}</pre>
      </div>
    </details>
  </div>
</section>,
{
  id: "splay-tree",
  title: "4. Splay-дерево",
  type: "html",
  description: "Дерево поиска, которое чинит само себя",
  category: "Продвинутые структуры",
  content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

```

```
<pre class="bg-slate-950 p-4 rounded-lg over-
ading-relaxed">вставили 10, 20, 30, 40, 50
```

10

//

20

//

30

//

40

//

50</pre>

высота = 5, поиск 50 = 5 сравнений
это уже не дерево, а односвязный

```
<p class="text-slate-300 text-sm mt-4">Есть
<ul class="list-disc list-inside text-slate
  <li><span class="text-white font-bold">
каждой вставки чинят дерево, чтобы оно <em>
/li>
```

```
  <li><span class="text-white font-bold">
менно быть кривым. Но каждый раз, когда мы
ровнее становится.</li>
</ul>
```

```
</div>
```

```
<!-- 2. Аналогии -->
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap
  <div class="bg-slate-800 p-6 rounded-lg bor
    <div class="absolute -top-3 left-4 bg-s
border-emerald-500 shadow-md">
```

Аналогия 1: стопка бумаг на столе

```
</div>
```

```
<p class="text-slate-300 text-sm">У теб
скиваешь его и, поработав, кладёшь <span cl
неделю самые нужные бумаги сами собой оказ
ет ровно это: «взял → положил наверх».</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bor
  <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

```
<h3 class="text-lg font-bold text-white mb-3">Обозначения
<p class="text-slate-300 text-sm mb-4">Обозначения: узел, который поднимаем, <span class="font-weight-bold">g</span> – дед (родитель родителя). Смотри
```

```
<div class="overflow-x-auto -mx-2 px-2">
  <table class="w-full text-xs sm:text-sm" >
    <thead>
      <tr class="text-left text-slate-300">
        <th class="py-2 pr-3 font-weight-normal">Имя узла
        <th class="py-2 pr-3 font-weight-normal">Родитель
        <th class="py-2 pr-3 font-weight-normal">Действие
        <th class="py-2 font-weight-normal">Действие
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-200">
        <td class="py-3 pr-3 font-weight-normal">Имя узла
        <td class="py-3 pr-3">деда
        <td class="py-3 pr-3">сам х
        <td class="py-3">х стал корнем
      </tr>
      <tr class="border-b border-slate-200">
        <td class="py-3 pr-3 font-weight-normal">Имя узла
        <td class="py-3 pr-3">х и р
        <td class="py-3 pr-3"><span class="font-weight-normal">аво-право)</span>
        <td class="py-3">х поднялся
      </tr>
      <tr>
        <td class="py-3 pr-3 font-weight-normal">Имя узла
        <td class="py-3 pr-3">х и р
        <td class="py-3 pr-3"><span class="font-weight-normal">аво-право)</span>
        <td class="py-3">р и г стали
      </tr>
    </tbody>
  </table>
</div>
```

стало: кустик высоты 2

глубина ветки упала вдвое!</pre>

```
<p class="text-slate-400 text-xs mt-4">
  ем.</p>
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><span>
  з, змейка (zig-zag) – крути СНИЗУ вверх.</p>
```

```
</div>
```

<!-- 6. Полный пример -->

```
<div class="bg-slate-800/60 p-6 rounded-xl border">
```

```
<h3 class="text-lg font-bold text-white mb-4">Дерево
```

```
<p class="text-slate-300 text-sm mb-4">Дерево
```

```
  3 шага, а потом делаем Splay(10).</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg overflow-x">
```

```
0 leading-relaxed">старт                               после Zi
```

```
      глубина 3 → 1
```

```
      10 в корне
```

```
      40
     /
    30
   /
  20
 /
10
```

```
      30
     /  \
    10   40
     \
    20
```

```
      10
     \
    30
   /  \
  20   40
```

итог: было 4 узла в линию (высота 4) → стало высота 3,
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути</span>. Все узлы, мимо
```

```
</div>
```

<!-- 7. Амортизация -->

```
<div class="bg-slate-800/60 p-6 rounded-xl border">
```

```
<h3 class="text-lg font-bold text-white mb-4">Дерево
```

```
<p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag
```



```

        </div>
    </div>

    <!-- 9. Код -->
    <details class="bg-slate-900/40 rounded-lg border
        <summary class="p-4 font-bold text-slate-400
        open:border-b border-slate-700/50">
        Код: rotate + splay + find (нажми, что
    </summary>
    <div class="p-5 text-sm text-slate-300 space
        <pre class="bg-slate-950 p-4 rounded ov
        xed">// узел: { key, left, right, parent }

// один поворот: x встаёт на место своего родителя
function rotate(x) {
    const p = x.parent, g = p.parent;

    if (p.left === x) {                // правый поворот
        p.left = x.right;
        if (x.right) x.right.parent = p;
        x.right = p;
    } else {                            // левый поворот (зеркал
        p.right = x.left;
        if (x.left) x.left.parent = p;
        x.left = p;
    }

    p.parent = x;                      // родитель уехал вниз
    x.parent = g;                      // x занял его место
    if (g) { if (g.left === p) g.left = x; else g.right =
}

// поднимаем x в корень
function splay(x) {
    while (x.parent) {
        const p = x.parent, g = p.parent;

        if (!g) {                      // ZIG: деда н

```

```

    </div>
    <div class="bg-rose-950/30 rounded-xl p-5 b
      <p class="text-rose-300 font-bold mb-2">
      <ul class="list-disc list-inside text-s
        <li>Нет гарантии на <em>отдельную</em>
        <li>Не годится для реального времени
        <li>Дерево меняется даже при чтении
        <li>Постоянные записи в память при
      </ul>
    </div>
  </div>

<!-- 11. Вопросы -->
<details class="bg-slate-900/40 rounded-lg bord
  <summary class="p-4 font-bold text-slate-400
    open:border-b border-slate-700/50">
    Вопросы, которые задают на экзамене (
  </summary>
  <div class="p-5 text-sm text-slate-300 spac
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Последний
енник или преемник). Splay делают всегда, да
бы, и амортизация сломалась бы.</p>
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Потому что
евоорачивают бамбук в другую сторону (см. бл
    </div>
    <div>
      <strong class="text-white block mb-
      <p class="text-slate-400">Чередовани
вает один из них в корень, превращая дерево
ступает только для <em>одной</em> конкретной
    </div>
    <div>
      <strong class="text-white block mb-

```

```

        ый верхний угол отрезается дважды! Поэтому
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "graph-planar-colors",
        title: "7. Графы. Планарные. Покраска",
        type: "html",
        description: "Формула Эйлера и теорема о 4 красках.",
        category: "Графы. Теория",
        content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 r
        <h2 class="text-2xl sm:text-3xl font-bold text-
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-blue-400
            <div class="bg-slate-900 p-4 rounded-lg mb-
                <p class="text-lg text-blue-300 italic
                <p class="text-xl font-mono text-white"
            ый граф можно раскрасить 4 цветами.</p>
        </div>
        <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg
                <div class="absolute -top-3 left-4
            rder border-emerald-500 shadow-md">
                Аналогия 1: Карта мира
            </div>
            <p class="text-slate-300 text-sm mb
        . Границы не пересекаются в одной точке по-
        ы не сливались цветами, Всегда можно всего
        </div>
    </div>
`
    }
}

```

```

        </div>
        <p class="text-slate-300 text-sm mb-6">
получить опыт, нужна работа". Алгоритм выявления
        </div>
    </div>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
        category: "Графы. Продвинутое",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
            <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">Макро-вершины — это вершины орграфа, которые принадлежат одной и той же компоненте сильной связности.
                <p class="text-xl font-mono text-white">Макро-вершины орграфа можно представить как ориентированный ациклический граф (DAG), где каждая макро-вершина инвертирована в порядке top-sort.</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        Аналогия 1: Улицы с односторонним движением
                    </div>
                    <p class="text-slate-300 text-sm mb-6">Представьте себе город, где все улицы имеют одностороннее движение. Если вы хотите добраться из одной точки в другую, вы должны следовать направлению движения улиц. Это аналогично тому, как в орграфе вы можете двигаться только по рёбрам в определённом направлении. В DAG (DAG — это граф без циклов) вы можете двигаться только в одном направлении, что позволяет вам находить путь от одной точки к другой. В орграфе с сильной связностью (SCC) вы можете двигаться в обоих направлениях, что делает его более сложным для анализа. В DAG вы можете двигаться только в одном направлении, что позволяет вам находить путь от одной точки к другой. В орграфе с сильной связностью (SCC) вы можете двигаться в обоих направлениях, что делает его более сложным для анализа.
                </div>
            </div>
        </div>
    </div>
</section>`
    }
}
```

```

</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает свя.",
    category: "Графы. Продвинутые",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400 m
      <div class="bg-slate-900 p-4 rounded-lg mb-
        <p class="text-lg text-rose-300 italic m
        <p class="text-xl font-mono text-white"
тей > 1).</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg
          <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              Аналогия 1: Центральный роутер
            </div>
            <p class="text-slate-300 text-sm mb
              правильный роутер - и два сегмента офиса б
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",

```

```

        content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-blue-300 italic">
          <p class="text-xl font-mono text-white">
            d-длина.</p>
        </div>
        <div class="grid grid-cols-1 gap-6">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
              Аналогия: Копаем туннель под .
            </div>
            <p class="text-slate-300 text-sm mb-4">
              ней земли. Но если начать копать одновременно
              в 2 РАЗА меньше работы (геометрически: площ
            </div>
          </div>
        </div>
      </div>
    </div>
  </section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
          <p class="text-xl font-mono text-white">
            которое торчит из посещенных в непосещенных
          </div>
        <div class="grid grid-cols-1 gap-6">
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
              ✦ Аналогия: Заражение вирусом
            </div>
            <p class="text-slate-300 text-sm mb-4">
              ли вирус). Мы стартуем из одного города, и
              Сеть растет только из одного связного куска
            </div>
          </div>
        </div>
      </div>
    </div>
  </section>`,
  },
  {
    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">

```

```

        i], который одновременно является её суффикс
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2"
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg"
            <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
                Аналогия 1: Бумеранг (Префикс
            </div>
            <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
мя перепроверок!</p>
        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg l
        <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300"
            <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",

```



```

    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}

```

```

    </div>

```

```

    </details>

```

```

    </div>

```

```

    </div>

```

```

</section>`,

```

```

    },

```

```

    {

```

```

        id: "complexity-classes",

```

```

        title: "24. Классы сложности, сведение задач",

```

```

        type: "html",

```

```

        description: "",

```

```

        category: "Теория сложности",

```

```

        content: `

```

```

<section id="complexity-classes" class="mb-12 scroll-mt-12">

```

```

    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

        <span class="bg-purple-600 text-white px-4 py-1 rounded-md">

```

```

        <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">

```

```

    </div>

```

```

    <div class="space-y-8">

```

```

        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">

```

```

            <h3 class="text-xl font-bold text-purple-400">

```

```

            <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

```

                <p class="text-lg text-purple-300 italic">

```

```

                <p class="text-xl font-mono text-white">

```

```

                Сведение (Reduction) A->B: Если я умею решать A, то

```

```

                </div>

```

```

            <div class="grid grid-cols-1 xl:grid-cols-2">

```

```

                <!-- Аналогия 1 -->

```

```

                <div class="bg-slate-800 p-6 rounded-lg">

```

```

                    <div class="absolute -top-3 left-4 right-4">

```

```

                    <div class="border border-emerald-500 shadow-md">

```

```

                        Аналогия 1: Судoku (P vs NP)

```

```

                    </div>

```

```

                    <p class="text-slate-300 text-sm mb-4">

```

```

                    пример сортировка чисел). Класс <b>NP</b> —

```

```

-----
vars: Record<string, CompilerVarInfo>;
/** Доп. пояснение что синхронизируется */
note: string;
/** Какой визуализатор связан */
synchint: string;
}

/**
 * Хардкод синхронизации i,k,j,n,m для каждой страницы.
 * Компилятор НЕ содержит исполняемого кода – только код
 * визуализации. Значения n,m,i,k,j – это то, что показ
 *
 * Если добавится новая страница – добавь сюда запись,
 */
export const CHAPTER_COMPILER_META: Record<string, Chap
// билет 1
"segment-trees": {
  title: "Дерево отрезков",
  vizId: "segment-trees",
  vars: {
    n: { label: "n", example: "6", desc: "длина массива"
    m: { label: "m", example: "8", desc: "размер дерева"
    i: { label: "i", example: "1", desc: "индекс вершины"
    k: { label: "k", example: "0", desc: "левая граница"
    j: { label: "j", example: "5", desc: "правая граница"
  },
  note: "mid = (k+j)//2 – разбиение; запрос охватываемый узлом"
  synchint: "Визуализатор подсвечивает узлы v с отрезком [l,r]"
},
// билет 2
"sparse-table": {
  title: "Разрезанная таблица",
  vizId: "sparse-table",
  vars: {
    n: { label: "n", example: "8", desc: "длина массива"
    m: { label: "m", example: "4", desc: "LOG = floor(log2(n))"
    i: { label: "i", example: "2", desc: "начало отрезка"
    j: { label: "j", example: "2", desc: "степень j: 2^j <= m-i+1"

```

```

    i: { label: "i", example: "x", desc: "целевой узел" },
    k: { label: "k", example: "p", desc: "родитель p" },
    j: { label: "j", example: "g", desc: "дед g" },
    m: { label: "m", example: "-", desc: "-" },
  },
  note: "rotate(p) затем rotate(x) vs rotate(x) дважды",
  syncHint: "Покадровый разбор Zig/ZigZig/ZigZag",
},
// билет 5
"prefix-sums-2d": {
  title: "Префиксные суммы 1D/2D",
  vizId: "prefix-sums-2d",
  vars: {
    n: { label: "n", example: "4", desc: "высота матрицы" },
    m: { label: "m", example: "4", desc: "ширина матрицы" },
    i: { label: "i", example: "2", desc: "индекс строки" },
    j: { label: "j", example: "2", desc: "индекс столбца" },
    k: { label: "k", example: "r2,c2", desc: "угол записи" },
  },
  note: "P[i][j]=A[i-1][j-1]+P[i-1][j]+P[i][j-1]-P[i-1][j-1]",
  syncHint: "Демо 1D: P[i]=a[0..i-1]; 2D: подсвечиваем",
},
// билет 6
"graph-dfs-bfs": {
  title: "DFS / BFS на графе",
  vizId: "graph-dfs-bfs",
  vars: {
    n: { label: "n", example: "6", desc: "число вершин" },
    m: { label: "m", example: "7", desc: "число рёбер" },
    i: { label: "i", example: "u", desc: "текущая вершина" },
    j: { label: "j", example: "v", desc: "сосед v в а" },
    k: { label: "k", example: "lvl", desc: "уровень/глубина" },
  },
  note: "DFS – стек (вглубь), BFS – очередь (вширь, уровень)",
  syncHint: "Визуализация графа подсвечивает фронт поиска",
},
// билет 7 – planarity (новый id)
"planarity-euler-formula": {

```

```

    synchint: "DFS раскрашивает каждую компоненту своим
},
// билет 9
"top-sort": {
  title: "Топологическая сортировка",
  vizId: "top-sort",
  vars: {
    n: { label: "n", example: "5", desc: "V – вершины"
    m: { label: "m", example: "6", desc: "E – дуги" }
    i: { label: "i", example: "u", desc: "текущая вер"
    j: { label: "j", example: "v", desc: "сосед v (ис"
    k: { label: "k", example: "order", desc: "позиция"
  },
  note: "Kahn (indeg) или DFS-postorder + reverse: k -
  synchint: "Демо одежды: «трусы перед штанами»; подс
},
// билет 10
"scc-kosaraju": {
  title: "SCC – Косарайю",
  vizId: "scc-kosaraju",
  vars: {
    n: { label: "n", example: "6", desc: "V – вершины"
    m: { label: "m", example: "7", desc: "E – дуги" }
    i: { label: "i", example: "order", desc: "порядок"
    j: { label: "j", example: "v", desc: "вершина v, c"
    k: { label: "k", example: "compId", desc: "номер c"
  },
  note: "Фаза1: DFS и стек order; Фаза2: DFS на GT в
  synchint: "Визуализатор показывает два прохода и ра
},
// билет 11 – мосты
"bridges-code": {
  title: "Мосты (DFS + tin/low)",
  vizId: "bridges-code",
  vars: {
    n: { label: "n", example: "6", desc: "V – вершины"
    m: { label: "m", example: "7", desc: "E – рёбра"
    i: { label: "i", example: "v", desc: "текущая вер

```

```

    vars: {
      n: { label: "n", example: "5", desc: "V" },
      m: { label: "m", example: "7", desc: "E" },
      i: { label: "i", example: "v", desc: "текущая вершина" },
      j: { label: "j", example: "deg(v)", desc: "степень вершины" },
      k: { label: "k", example: "cntOdd", desc: "k = количество чётных" },
    },
    note: "Эйлеров цикл ⇔ все j чётные; путь ⇔ k=2 (начало и конец)",
    synchHint: "Симулятор подсвечивает нечётные вершины",
  },
  "graph-euler": {
    title: "Эйлеров граф",
    vizId: "euler-path-vs-cycle",
    vars: {
      n: { label: "n", example: "5", desc: "V" },
      m: { label: "m", example: "7", desc: "E" },
      i: { label: "i", example: "v", desc: "v" },
      j: { label: "j", example: "deg", desc: "deg" },
      k: { label: "k", example: "odd", desc: "odd count" },
    },
    note: "same",
    synchHint: "Эйлеров обход",
  },
  // билет 14
  dijkstra: {
    title: "Дейкстра",
    vizId: "dijkstra",
    vars: {
      n: { label: "n", example: "6", desc: "V — число вершин" },
      m: { label: "m", example: "9", desc: "E — число рёбер" },
      i: { label: "i", example: "u", desc: "извлечённая вершина" },
      j: { label: "j", example: "v", desc: "сосед v: рёбра (u,v)" },
      k: { label: "k", example: "w", desc: "w(u,v) ≥ 0; w(u,v) — вес рёбра" },
    },
    note: "heap по dist; O((n+m) log n); не работает при отрицательных весах",
    synchHint: "Визуализатор показывает heap, dist[] и tree",
  },
  // билет 15

```

```

    note: "Шаг1 Bellman из s → h; Шаг2 перевзвешивание;
    synchHint: "Визуализатор показывает h, перевзвешенную
},
// билет 18
"mst-kruskal": {
  title: "Краскал",
  vizId: "mst-kruskal",
  vars: {
    n: { label: "n", example: "5", desc: "V – вершины"},
    m: { label: "m", example: "7", desc: "E – рёбра (E)" },
    i: { label: "i", example: "idx", desc: "индекс те"},
    j: { label: "j", example: "v", desc: "конец ребра"},
    k: { label: "k", example: "u", desc: "начало ребра"},
  },
  note: "сортировка  $O(m \log m)$  + DSU; лес постепенно растёт
  synchHint: "Подсвечивается очередное ребро i: k–j с миним",
},
// билет 19
"mst-prima": {
  title: "Прим",
  vizId: "mst-prima",
  vars: {
    n: { label: "n", example: "5", desc: "V" },
    m: { label: "m", example: "7", desc: "E" },
    i: { label: "i", example: "u", desc: "последняя вершина"},
    j: { label: "j", example: "v", desc: "лучший кандидат"},
    k: { label: "k", example: "w", desc: " $w = \min \text{edge}$ " },
  },
  note: "дерево растёт как пятно: берём минимальное рёбро, не образующее цикла
  synchHint: "Граф: дерево подсвечено, heap показывает",
},
// билет 20
"mst-boruvka": {
  title: "Борувка",
  vizId: "mst-boruvka",
  vars: {
    n: { label: "n", example: "9", desc: "V – деревни"},
    m: { label: "m", example: "14", desc: "E" },
  },

```

```

vizId: "aho-corasick",
vars: {
  n: { label: "n", example: "4", desc: "число слов",
  m: { label: "m", example: "11", desc: "длина текста",
  i: { label: "i", example: "v", desc: "вершина бор",
  j: { label: "j", example: "c", desc: "символ с → k",
  k: { label: "k", example: "link", desc: "суффикс",
},
note: "строим бор, затем BFS: очереди по k; go по f",
synchint: "Бор + автомат + анимация водопада символов",
},
// билет 24
"complexity-classes": {
  title: "Классы сложности",
  vizId: undefined,
  vars: {
    n: { label: "n", example: "n", desc: "размер входных данных",
    m: { label: "m", example: "p(n)", desc: "полином",
    i: { label: "i", example: "A", desc: "задача A, к",
    j: { label: "j", example: "B", desc: "задача B, к",
    k: { label: "k", example: "cert", desc: "сертификат",
  },
  note: "P – детерминированно за poly; NP – проверка",
  synchint: "Нет граф-симуляции – показываем схему св",
},
// вводные
intro: {
  title: "Введение",
  vizId: "intro",
  vars: {
    n: { label: "n", example: "6", desc: "V для превы",
    m: { label: "m", example: "9", desc: "E" },
    i: { label: "i", example: "s", desc: "исток s" },
    j: { label: "j", example: "t", desc: "сток t" },
    k: { label: "k", example: "dist", desc: "dist – p",
  },
  note: "Карточки-мнемоники на старте",
  synchint: "Мнемокарточки и обзор",

```

```

    note: "рекурсия DFS = стек вызовов",
    synchint: "Тарелки-стек и рекурсия DFS",
  },
  "queue-bfs": {
    title: "Очередь → BFS",
    vizId: "queue-bfs",
    vars: {
      n: { label: "n", example: "6", desc: "размер очереди", },
      i: { label: "i", example: "head", desc: "голова очереди", },
      j: { label: "j", example: "tail", desc: "хвост очереди", },
      k: { label: "k", example: "v", desc: "вершина v = queue[i]", },
      m: { label: "m", example: "-", desc: "-" },
    },
    note: "BFS: enqueue(s) → пока очередь не пуста: k=dequeue()",
    synchint: "Очередь в столовой и BFS волна",
  },
  "dynamic-programming": {
    title: "Динамика",
    vizId: "dynamic-programming",
    vars: {
      n: { label: "n", example: "10", desc: "размер DP-таблицы", },
      m: { label: "m", example: "10", desc: "M – второй массив", },
      i: { label: "i", example: "i", desc: "итерация по i", },
      j: { label: "j", example: "j", desc: "итерация по j", },
      k: { label: "k", example: "opt", desc: "k = argmin" },
    },
    note: "заполняем таблицу по i,j; k – выбор предшествующего элемента",
    synchint: "Таблица DP построчно, подсветка переходов",
  },
};

export function getCompilerMeta(id: string): ChapterCompilerMeta {
  return CHAPTER_COMPILER_META[id];
}

export function getFallbackMeta(id: string): ChapterCompilerMeta {
  return {
    CHAPTER_COMPILER_META[id] ?? {

```



```

-----
    if (meta.vars.i && meta.vars.j) {
        return `i, j = 0, 0 # ${meta.title}\n# ${vars}`;
    }
    return `n, m = ${meta.vars.n?.example ?? 0}, ${meta.v
}

/** Полный код алгоритма – показывается по кнопке "пока
export function getFullCode(id: string): string | null
    if (id === "sparse-table") {
        return `# Разреженная таблица – построение
# Визуализация – отдельный объект: SparseTableVars + Sp
for j in range(1, LOG):
    for i in range(n - (1<<j) + 1):
        ST[i][j] = min(ST[i][j-1], ST[i + (1<<(j-1))][j

# 2D: 4 блока → 1 (отдельный наследник)
# a,b,c,d = блок 2x2
for kx in range(4):
    for ky in range(4):
        ST[r][c][kx][ky] = min(
            ST[r][c][kx-1][ky], ST[r+(1<<(kx-1))][c][kx-1][
        # иначе по горизонтали: min(ST[r][c][kx][ky-1], ST[
`;
    }
    if (id === "prefix-sums-2d") {
        return `# Префиксные суммы
P[0]=0
for i in range(1,n+1):
    P[i]=P[i-1]+a[i-1] # i
# 2D: a,b,c,d → s
for i in range(1,n+1):
    for j in range(1,m+1):
        P[i][j]=a[i-1][j-1]+P[i-1][j]+P[i][j-1]-P[i-1][j-1]
`;
    }
    if (id === "floyd") {
        return `for k in range(n):
for i in range(n):

```

```
    <p><strong class="text-white">Эйлер  
ин раз. Вершины повторять можно.</p>
```

```
    <p><strong class="text-white">Эйлер
```

```
    <div class="p-3 bg-slate-950 rounded
```

```
        <p class="text-sm text-blue-300
```

```
        <p><strong class="text-white">П
```

```
        <p><strong class="text-white">Ц
```

```
    </div>
```

```
  </div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
  <strong>Билет 13 (Различие пути и цикла
```

```
  <br>* <span class="text-green-400 font-  
шел-вошёл-вышел).
```

```
  <br>* <span class="text-yellow-400 font-  
финиш).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap
```

```
  <div class="bg-slate-800 p-6 rounded-lg bor
```

```
    <div class="absolute -top-3 left-4 bg-s  
order-green-500 shadow-md">
```

```
      Путь: Нормальная прогулка
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb-4">
```

```
      Ты вышел из дома <strong>(нечётная)  
талый пришёл в бар <strong>(вторая нечётная
```

```
      Домой вернуться не смог, потому что
```

```
    </p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bor
```

```
  <div class="absolute -top-3 left-4 bg-r  
der-rose-500 shadow-md bg-slate-950">
```

```
    Цикл: Гамильтонов синдром (болезнь
```

```
  </div>
```

```

content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-
  <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

    <div class="bg-slate-900 p-4 rounded-lg mb-
      <p class="text-lg text-emerald-300 ital
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">tin[v
        <p><strong class="text-white">low[v
        <div class="p-3 bg-slate-950 rounde
          <span class="text-rose-400 font
          <p class="text-xl font-bold tex
          <p class="text-xs text-slate-40
        </div>
      </div>
    </div>
  </div>
  <div>
    <p class="text-slate-300 text-sm">
      Если из сына и и всех его потомков <str
      Единственная ниточка. Порвётся – граф р
    </p>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bord
      <div class="absolute -top-3 left-4 bg-s
      border-emerald-500 shadow-md">
        Мост: Единственная веревка
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Представь, что ты альпинист. Ты спу
        Из пещеры и нет других выходов – то

```

```
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;         // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                 // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}
```

</div>

<p class="text-xs text-slate-400">Запуск

```
<div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `class="text-white">V=5, E=10 > 9</code> – БАХ, непланарен!`

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забудь

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
```

```
<div class="absolute -top-3 left-4 bg-slate-900 border-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `class="text-white">V=6, E=9 ≤ 12</code> – проходит, но он не`

Почему? Потому что у двудольного графа

Отсюда более строгое ограничение: `<`

`class="text-white">9 > 8</code>`

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
```

```
<summary class="p-4 font-bold text-slate-400 border-rose-500 shadow-md">
```

Доказательство непланарности К5 (Скрыть)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space-y-2">
```

```
<p class="text-blue-400">Доказательство
```

```
<p>
```

```
<p class="text-slate-300 text-sm">
```

```
  <strong>Важно:</strong> Это работает то-
  еплятся к точкам сочленения. То есть, <strong>
  ег - это тупик со степенью 1).
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
  <div class="bg-slate-800 p-6 rounded-lg border-
```

```
    <div class="absolute -top-3 left-4 bg-slate-800
    rder-rose-500 shadow-md">
```

```
      Аналогия: Главный перекресток
```

```
    </div>
```

```
    <p class="text-slate-300 text-sm mb-4">
```

```
      Представь город, где два района соединены
      её перекроют (удалят вершину) — районы будут
      g>хрупкость</strong> системы.
```

```
    </p>
```

```
  </div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border-
```

```
  <div class="absolute -top-3 left-4 bg-emerald-900
  er border-emerald-500 shadow-md">
```

```
    Телеком: Центральный свитч
```

```
  </div>
```

```
  <p class="text-slate-300 text-sm mb-4">
```

```
    У тебя несколько компьютеров в одной комнате
    абелем (мостом). Сами свитчи — это точки со-
```

```
  </p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border-
```

```
  <summary class="p-4 font-bold text-slate-400 border-
  order-slate-700/50">
```

```
    Душный мод: Код и корень DFS (Скрыто)
```

```
  </summary>
```

```

        <ul class="list-disc list-inside text-sm text-gray-600">
          <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
          <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
          <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong>
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
        </ul>
      </div>
    </div>
  </section>`,
  },
];

```

ФАЙЛ 23/87: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp";

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
];

```

```
import type { DemoKind } from "../components/hints/MiniHints";

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
  id: string;
  /** Варианты написания в тексте (регистр не важен). */
  labels: string[];
  title: string;
  /** Объяснение «на пальцах» – 1–2 фразы. */
  short: string;
  /** Строгая формулировка / что важно не забыть. */
  formal?: string;
  complexity?: string;
  /** Какая мини-анимация показывается в карточке. */
  demo?: DemoKind;
  /** id полного симулятора (см. vizRegistry) – кнопка «симулятор» */
  simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину"],
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом соседи соседей и так далее.",
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину –",
  },
];
```



```
{
  id: "queue",
  labels: ["очередь", "очереди", "очередью", "FIFO"],
  title: "Очередь (FIFO)",
  short: "Очередь в столовой: кто первым встал, тот и ест",
  formal: "push_back / pop_front за  $O(1)$ . Основа BFS и Dijkstra",
  demo: "queue",
  simulator: "queue-bfs",
},
{
  id: "dsu",
  labels: ["DSU", "СНМ", "система непересекающихся множеств"],
  title: "DSU — система непересекающихся множеств",
  short:
    "«Кто твой староста?» Каждый элемент знает своего старосту",
  formal:
    "find со сжатием путей + union по рангу/размеру. Поддержка операций",
  complexity: "почти  $O(1)$  — обратная функция Аккермана",
  demo: "dsu",
  simulator: "mst-kruskal",
},
{
  id: "trie",
  labels: ["бор", "бора", "боре", "бору", "бором", "trie"],
  title: "Бор (trie, префиксное дерево)",
  short:
    "Дерево, где путь от корня по буквам и есть слово",
  formal: "Из корня по символам; в вершине хранится ф. слово",
  complexity: "поиск слова —  $O(|\text{слова}|)$ ",
  demo: "trie",
  simulator: "aho-corasick",
},
{
  id: "bfs-on-trie",
  labels: [
    "обход дерева",
    "обход бора",
    "обход в ширину по бору",
  ],
}
```

```

    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,
    formal: "if  $d[v] > d[u] + w(u,v) \rightarrow d[v] = d[u] + w(u,v)$ 
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "префикс"],
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от начала»
    formal: " $P[0]=0, P[i]=P[i-1]+a[i-1]$ . Сумма  $a[l..r] = P[r]-P[l-1]$ .",
    complexity: "препроцессинг  $O(n)$ , запрос  $O(1)$ ",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "sparse-table",
    labels: [
        "разреженная таблица", "разреженная таблица", "sparse-table",
        "разреженная", "двоичные подъёмы", "двоичный подъём",
    ],
    title: "Разреженная таблица (метод двоичных подъёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1, 2, 4, 8, ...
        м.",
    formal: " $ST[i][j] = \text{op}(ST[i][j-1], ST[i+2^{(j-1)}][j-1])$ ",
    complexity: "препроцессинг  $O(n \log n)$ , запрос  $O(1)$ "
}

```

```

    labels: [
        "остовное дерево", "остовного дерева", "остовное",
        "минимальное остовное дерево", "MST",
    ],
    title: "Минимальное остовное дерево (MST)",
    short:
        "Связать все точки проводами так, чтобы суммарная
    formal: "V-1 ребро, никаких циклов. Краскал (жадно и
        но).",
    demo: "mst",
    simulator: "mst-kruskal",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизи
        "амортизированное", "амортизированного", "амортизи
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер
    formal: "Метод потенциалов: дорогая операция оплачи
        ассив.",
    demo: "amortized",
},
{
    id: "np",
    labels: [
        "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
        "класс P", "полином", "полиномиальное",
    ],
    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (су
        ые в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полн
    demo: "np",
},

```



```

    },
    {
      id: "planarity",
      labels: ["планарный", "планарность", "планарного",
        title: "Планарность",
      short:
        "Граф планарен, если его можно нарисовать на листе бумаги",
      formal:
        " $V - E + F = 2$  для связного планарного графа. Курсив",
      simulator: "planarity-euler-formula",
    },
    {
      id: "splay",
      labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "всестороннее",
      ],
      title: "Splay и повороты",
      short:
        "Каждый раз, когда к узлу обратились, его «вытягивают»",
      formal:
        "Три случая: zig (родитель – корень), zig-zig (узлы в одну сторону), zig-zag (узлы в разные стороны).",
      demo: "amortized",
      simulator: "splay-tree",
    },
    {
      id: "adjacency",
      labels: ["матрица смежности", "список смежности", "матрица смежности",
        title: "Способы хранить граф",
      short:
        "Матрица смежности – таблица «есть ли ребро», быстрый доступ, экономно для разреженных графов.",
      formal: "Матрица: проверка  $O(1)$ , память  $O(V^2)$ . Список: проверка  $O(V)$ , память  $O(V)$ .",
    },
    {
      id: "dijkstra",

```

```

    complexity: "O(V³) времени, O(V²) памяти",
    demo: "floyd",
    simulator: "floyd",
},
{
    id: "prim",
    labels: ["Прима", "Прим", "Prim"],
    title: "Алгоритм Прима",
    short:
        "Остов растёт как плесень из одной точки: на кажды
    formal:
        "Множество посещённых + куча рёбер на границе. До
        усок.",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-prim",
},
{
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
        "Все компоненты одновременно шлют «гонца» к ближайш
    formal:
        "Фаза: для каждой компоненты ищем минимальное исх
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-boruvka",
},
{
    id: "kruskal",
    labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч
    formal:
        "Сортировка O(E log E) + E операций union/find. C
    complexity: "O(E log E)",

```

```

        "Координаты бывают гигантские и дробные, а нам ну
        порядковым номером.",
    formal:
        "sort + unique + binary search: значение → индекс
        ,
    complexity: "O(n log n)",
    demo: "coord-compress",
},
{
    id: "treap",
    labels: ["Treap", "декартово дерево", "декартова де
    title: "Treap = Tree + Heap",
    short:
        "Одно дерево живёт по двум правилам сразу: по ключу
        ча. Случайность и держит его сбалансированным."
    formal:
        "Базис — split(t, x) (разрезать по ключу) и merge
        я через них. Ожидаемая высота O(log n).",
    complexity: "ожидаемо O(log n)",
    demo: "heap",
},
{
    id: "bst",
    labels: ["бинарное дерево поиска", "дерево поиска",
    title: "Бинарное дерево поиска",
    short:
        "Слева всё меньше корня, справа — всё больше. Пои
    formal:
        "Без балансировки дерево может вырождаться в списо
    complexity: "O(высоты): от O(log n) до O(n)",
    demo: "segment-tree",
},
{
    id: "inclusion-exclusion",
    labels: [
        "включений-исключений", "включения-исключения", "
        "вырезание прямоугольников",
    ],

```

```

"Строковые алгоритмы держатся на трёх вопросах: где (Z-функция) и где встречаются слова словаря (бордер).",
formal:
"Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция.",
demo: "prefix-function",
simulator: "string-kmp",
},
{
id: "rotation",
labels: ["поворот", "повороты", "поворота", "поворо"],
title: "Поворот (rotation) в дереве",
short:
"Локальная перевеска двух узлов: ребёнок встаёт на место своего родителя, а родитель перемещается на место дившемся месту. Порядок ключей при этом не ломается.",
formal:
"Правый поворот вокруг p:  $x = p.left; p.left = x$ . Аналогично для левого поворота. Также есть Zig-Zag.",
complexity: "O(1)",
demo: "zig",
simulator: "splay-rotations",
},
{
id: "zig",
labels: ["Zig", "Zag", "зиг"],
title: "Zig — одиночный поворот",
short:
"Самый простой случай: родитель x уже сидит в корневом узле. Если дерево не сбалансировано, то оно заканчивается, если глубина была нечётной.",
formal: "Применяется ровно один раз за всю операцию.",
demo: "zig",
simulator: "splay-rotations",
},
{
id: "zig-zig",
labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
title: "Zig-Zig — x и родитель с одной стороны",
short:
"Дерево вытянулось в «бамбук»: g → p → x идут в одну сторону, и x — это левый или правый ребёнок p.",
formal:
"Если x — левый ребёнок p, то делаем правый поворот вокруг p, а затем левый поворот вокруг g. Если x — правый ребёнок p, то делаем левый поворот вокруг p, а затем левый поворот вокруг g. Если x — левый ребёнок g, то делаем левый поворот вокруг g. Если x — правый ребёнок g, то делаем правый поворот вокруг g.",
complexity: "O(1)",
demo: "zig-zig",
simulator: "splay-rotations",
},
}

```



```

    title: "Обзор: Кратчайшие пути и Графы",
    type: "html",
    category: "Графы",
    content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
        <p class="text-slate-300 text-sm mb-4">Пред
        оезда). Алгоритмы поиска кратчайшего пути п
        <div class="mt-8 mb-4">
            <div id="slot-mnemonic-cards"></div>
        </div>
    </div>
</div>
</section>`
  },
  {
    id: "dijkstra",
    title: "Билет 14. Дейкстра",
    type: "html",
    category: "Графы",
    content: `<section id="dijkstra-content" class="mb-12">
<div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">

        <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">
            <p class="text-xl font-mono text-white">

```

```

    </div>
</section>`
  },
  {
    id: "bellman-ford",
    title: "Билет 15. Форд-Беллман",
    type: "html",
    category: "Графы",
    content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 r
  <h2 class="text-3xl font-bold text-white">Форд-Б
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">Параноик
      <p class="text-xl font-mono text-white">Параноик
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
        <div class="border-emerald-500 shadow-md">
          ♂ Аналогия 1: Параноик
        </div>
        <p class="text-slate-300 text-sm mb-4">Параноик
        берет ВСЕ возможные дороги V-1 раз подряд.
        <div id="slot-chapter-image-bellman">
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l

```

Аналогия 1: Автомагистрали

```
</div>
<p class="text-slate-300 text-sm mb-6">
  евле долететь из Москвы в Париж, если мы сд
  <div id="slot-chapter-image-floyd" class="h-40 w-40">
</div>

<!-- Аналогия 2 -->
<div class="bg-slate-900 p-6 rounded-lg">
  <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
    Время работы
  </div>
  <p class="text-slate-300 text-sm mb-6">
    три вложенных цикла. Сложность  $O(V^3)$ . Если
  </div>
</div>

<div id="slot-floyd-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
  id: "aho-corasick",
  title: "Билет 23. Алгоритм Ахо-Корасик",
  type: "html",
  category: "Строки",
  content: `<section id="aho-corasick" class="mb-12">
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1 rounded">
  <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-l">
    <h3 class="text-xl font-bold text-blue-400 mb-4">`
```

```

<div class="p-5 text-sm text-slate-300 space-y-4">
  <p>Каждая вершина имеет таблицу переходов <code>
    ссылку <code>term_link</code> (ближайшая мен
  <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
    map<char, int> go;          // Предвычисленные пер
    int link = 0;              // Суффиксная ссылка: куда
    int term_link = 0;         // Терминальная ссылка: мат
    bool is_terminal = false;
    vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
    queue<int> q;
    // Корни и их дети
    for (auto const& [ch, child] : nodes[0].trie) {
        nodes[child].link = 0;
        q.push(child);
    }

    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto const& [ch, child] : nodes[v].trie) {
            // Суффиксная ссылка ребёнка – это переход
            nodes[child].link = nodes[nodes[v].link].go

            // Если суффиксная ссылка сама является сло
            // Иначе наследуем terminal_link
            if (nodes[nodes[child].link].is_terminal) {
                nodes[child].term_link = nodes[child].l
            } else {
                nodes[child].term_link = nodes[nodes[ch
            }

            q.push(child);

```

```
export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
  explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
  "euler-path-vs-cycle": [
    {
      question: "В связном графе ровно 2 вершины с нечётными степенями",
      options: [
        "Есть эйлеров цикл (вернусь домой!)",
        "Есть эйлеров путь (из одной нечётной в другую)",
        "Ни пути, ни цикла нет — это полный провал"
      ],
      correctIndex: 1,
      explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше 2 — эйлерова ни пути, ни цикла нет."
    },
    {
      question: "Почему гамильтонов цикл — это NP-полная задача?",
      options: [
        "Потому что Гамильтон — болезнь, и лечить её тяжело",
        "Потому что для эйлерова цикла есть простой критерий",
        "Потому что Эйлер был умнее Гамильтона"
      ],
      correctIndex: 1,
      explanation: "Для эйлерова цикла работает критерий степени вершин. Для гамильтонова цикла нет такого критерия. Это NP-полная задача, которую не известно, как решить за полиномиальное время."
    },
    {
      question: "Куда ведёт эйлеров путь, если нечётных вершин ровно 2?",
      options: [
        "Всё ещё можно пройти по всем рёбрам ровно раз",
        "Такого графа не существует",
        "Эйлерова пути нет — нужно удалять лишние рёбра"
      ],
      correctIndex: 0,
      explanation: "Если в графе ровно 2 вершины с нечётными степенями, то существует эйлеров путь, который начинается в одной из этих вершин и заканчивается в другой."
    }
  ]
```

```

"planarity-euler-formula": [
  {
    question: "Планарный граф имеет  $V=6$ ,  $E=12$ . Возможны ли такие графы?",
    options: [
      "Да, если это двудольный граф",
      "Нет, нарушается неравенство  $E \leq 3V - 6$ ",
      "Да, если нет треугольных граней"
    ],
    correctIndex: 1,
    explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$  – на грани возможности. Но если бы было так, то такой граф был бы планарен. Так что такой граф непланарен."
  },
  {
    question: "Сколько граней у планарного графа с  $V=7$  и  $E=10$ ?",
    options: [
      "3 грани",
      "5 граней",
      "10 граней"
    ],
    correctIndex: 1,
    explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$ . Не 3, не 10, а 5."
  },
  {
    question: "Почему  $K_5$  непланарен?",
    options: [
      "Потому что  $10 > 3*5 - 6 = 9$  – нарушение",
      "Потому что Эйлер был против",
      "Потому что 5 вершин – несчастливое число"
    ],
    correctIndex: 0,
    explanation: "У  $K_5$ :  $V=5$ ,  $E=10$ ,  $3V - 6 = 9$ .  $10 > 9$  – нарушение неравенства."
  }
]
};

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
        ы рассылать 10 000 писем, он пишет одно письмо в
        счетах сотрудников только тогда, когда сопро-
        женное обновление).</p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 bottom-4
        border-rose-500 shadow-md">
            Аналогия 2: Матрешки-коробки
        </div>
        <p class="text-slate-300 text-sm mb-4">
        ше, и так далее. Если нам нужно покрасить по-
        Внутри всё синее!" на большую коробку. Опус-
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg">
        <summary class="p-4 font-bold text-slate-300">
        -b border-slate-700/50">
            Строгое доказательство / Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">
function push(node) {
  if (lazy[node] !== 0) {
    lazy[2 * node] += lazy[node];
    tree[2 * node] += lazy[node];
    lazy[2 * node + 1] += lazy[node];
    tree[2 * node + 1] += lazy[node];
    lazy[node] = 0;
  }
}</pre>
        </div>
    </details>
  </div>
</div>

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            нужно покрыть отрезок, мы берем две самые бо
        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg border-1
        <summary class="p-4 font-bold text-slate-300
            -b border-slate-700/50">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300"
                <pre class="bg-slate-950 p-4 rounded
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
            </div>
        </details>
    </div>
</div>
</section>`
    },
    {
        id: "treap",
        title: "3. Декартово дерево (Treap)",
        type: "html",
        description: "Дерево поиска по ключу и Куча по приоритету",
        category: "Продвинутые структуры",
        content: `
<section id="treap" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
        <div class="space-y-8">
            <div class="bg-slate-700/50 p-6 rounded-xl border-1">
                <h3 class="text-xl font-bold text-blue-400">
                <div class="bg-slate-900 p-4 rounded-lg mb-4">
                    <p class="text-lg text-blue-300 italic">
                    <p class="text-xl font-mono text-white">

```



```

    }
  }</pre>
</div>
</details>
</div>
</div>
</section>`
},
{
  id: "splay-tree",
  title: "4. Splay-дерево",
  type: "html",
  description: "Дерево поиска, которое самобалансирует",
  category: "Продвинутые структуры",
  content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        (Zig, Zig-Zig, Zig-Zag), гарантируя амортизацию
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            border-emerald-500 shadow-md">
            Аналогия 1: VIP-лифт
          </div>
          <p class="text-slate-300 text-sm mb-4">
            вится корнем дерева (VIP-статус). Если ты член
            ка почти не требуется времени!</p>
          </div>

```

Оно поднимет узел, на котором
ска. За счет сдвига этого листа в корень, д
/p>

<div>

```
<strong class="text-white block
```

Если агент постоянно переключается между находящимися на разных концах дерева, его можно считать свинг-агентом. Постоянное свинг-переключение превратит

<div>

```
<strong class="text-white block
```

Хотя один поиск может стоить
емах обладают прекрасным свойством: они не
по пути. "Палочное" дерево прессуется в сб
осов.

</section>

4.

3

```
id: "prefix-sums-2d",
```

```
title: "5. Двумерная матрица префиксных сумм".
```

```
type: "html",
```

description: "Сжатие суммы подматрицы за $O(1)$ ".

category: "Алгоритмы матрицы".

```
content: `
```

```
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10":
```

```
<div class="flex items-center mb-6 flex-wrap gap-3":
```

`<span class="bg-indigo-600 text-white px-4 py-1`

class="text-2xl sm:text-3xl font-bold text-violet-600">

</div>

```

category: "Графы. Пути",
content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Алгоритм Дейкстры
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Представьте себе человека, который хочет добраться из пункта А в пункт Б, но не знает, как это сделать. Он спрашивает у других людей, как добраться, и получает разные советы. Он выбирает один из них и идет по этому пути. Если он обнаружит, что этот путь не самый лучший, он может вернуться и выбрать другой.
        <p class="text-xl font-mono text-white">Аналогия 1: Позитивное планирование
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Позитивное планирование
          </div>
          <p class="text-slate-300 text-sm mb-4">
            (нельзя поехать и заработать). Он всегда выбирает лучший вариант.
          </p>
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
            Ограничения
          </div>
          <p class="text-slate-300 text-sm mb-4">
            дный и не умеет пересматривать уже "зафиксированные" варианты.
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`

```

```

        </div>
      </div>
    </section>`
  },
  {
    id: "floyd",
    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

      <div class="bg-slate-800 p-6 rounded-lg border">
        <div class="absolute -top-3 left-4 bg-slate-700/50
border-emerald-500 shadow-md">
          Суть фокуса (По шагам)
        </div>
        <p class="text-slate-300 text-sm mb-4">
          Главный герой – <b>внешний цикл</b>
          шаг <code class="bg-slate-900 text-pink-400">
          шу себе проходить через вершину k?</i>
        </p>
        <ul class="text-sm text-slate-300 space-y-2">
          <li><b>Шаг k=0:</b> Ты знаешь только
          <li><b>Шаг k=1:</b> Разрешаем пересечение
          e> через путь <code class="text-indigo-300">
          <li><b>Шаг k=2:</b> Разрешаем пересечение

```

```

<ol class="text-sm text-slate-300" style="list-style-type: none;">
  <li>Добавляем <b>фиктивную вершину</b>
  <li>Запускаем от неё <b>медленные обходы</b> «потенциалы»</li>
  <li>Меняем старые веса: новое равенство (старые веса + разность потенциалов) - берем минимальное</li>
  <li><b>Магия!</b> Все веса теперь являются разностями потенциалов (разность потенциалов между вершинами)</li>
  <li>Теперь смело запускаем <b>быстрые обходы</b> (поиск минимального)</li>
</ol>
</div>
</div>
<div id="slot-johnson-viz" class="mt-8"></div>
</div>
</section>`
},
{
  id: "mst-kruskal",
  title: "18. Графы. Остовное дерево. Краскал",
  type: "html",
  category: "Графы. Остовные",
  content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">18. Графы. Остовное дерево. Краскал</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Остовное дерево</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-emerald-400">Остовное дерево</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Остовное дерево - это подграф, который является деревом и содержит все вершины графа.</p>
        <p class="text-xl font-mono text-white">Остовное дерево можно найти с помощью алгоритма Краскал (проверяем через DSU) - берем в ответ.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4">

```

```

        ли вирус). Мы стартуем из одного города, и
        Сеть растет только из одного связного куска
    </div>
</div>
</div>
</div>
</section>`
},
{
    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">
                <p class="text-xl font-mono text-white">
                ается с соседом.</p>
            </div>
        <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border
                rder border-emerald-500 shadow-md">
                    Аналогия: Племена
                </div>
                <p class="text-slate-300 text-sm mb-4">
                изкому племени для объединения. К вечеру пле
                ства шлют гонцов к ближайшему чужому царству.
            </div>
        </div>
    </div>
</div>

```

```
<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

```
  ♂ Аналогия 1: Поиск без отка
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

```
  Мы идём по строке слева направо. К  
строки СЕЙЧАС закончился под моим пальцем?»
```

```
  Представь, ты ищешь в тексте слова  
x</code>. Тупой алгоритм начнет искать заново  
</code>, а это начало нашего слова! Не надо
```

```
</p>
```

```
</div>
```

```
<!-- Аналогия 2 -->
```

```
<div class="bg-slate-900 p-6 rounded-lg
```

```
  <div class="absolute -top-3 left-4 border  
border-rose-500 shadow-md">
```

```
    Аналогия 2: LLM стриминг
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb
```

```
  Для LLM, которая стримит токены  
каждом шаге. Если мы частично совпали с зап  
акого места этого слова мы можем продолжить
```

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg
```

```
  <summary class="p-4 font-bold text-slate-300  
-b border-slate-700/50">
```

```
    Пример вычисления (Скрыто)
```

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
  <p>Тот же пример: <b>abcabcd</b></p>
```

```
  <ul class="list-disc pl-5 space-y-2
```

```
    <li><b>a</b>: Префиксов нет. <code>
```

```
    <li><b>ab</b>: Из начала ("a")
```

```
>
```

```
<section id="string-z-func" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          ca 0) и её суффиксом, начинающимся с i.</p>
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            border-emerald-500 shadow-md">
            Аналогия 1: Линейка-шаблон
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Тут мы берём всю строку и «прикидываю
            я начну читать строку отсюда, насколько длин
            Это самый быстрый способ найти i
          >, считаешь Z-функцию, и там, где <code>Z[i]
          </p>
        </div>

        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            border-rose-500 shadow-md">
            Аналогия 2: LLM Самокопираст
          </div>
          <p class="text-slate-300 text-sm mb-4">
            В LLM Z-функция может применяться
            [i]</code> (где <i>i</i> указывает назад на
```



```

        </details>
      </div>
    </div>
  </section>`
  },
  {
    id: "aho-corasick",
    title: "23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",
    content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border-l">
        <h3 class="text-xl font-bold text-blue-400 mb-4">

          <div class="bg-slate-900 p-4 rounded-lg mb-6 text">
            <p class="text-sm text-blue-300 italic mb-2">Осн
            <p class="text-lg font-mono text-white">Бор (Тр
          </div>
          <p class="text-slate-300 text-sm">Позволяет найти
            го один проход.</p>
        </div>

        <!-- Аналогии -->
        <div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
          <div class="bg-slate-800 p-6 rounded-lg border bo">
            <div class="absolute -top-3 left-4 bg-slate-700
              emerald-500 shadow-md">
              Аналогия 1: Паутина (Бор)
            </div>
            <p class="text-slate-300 text-sm mb-4">Представ
              ет — мы падаем по <b>суффиксной ссылке</b> ("

```

```

    },
    {
      id: "complexity-classes",
      title: "24. Классы сложности, сведение задач",
      type: "html",
      category: "Теория сложности",
      content: `
<section id="complexity-classes" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border
      <h3 class="text-xl font-bold text-purple-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-1">
        <p class="text-lg text-purple-300 italic">
        <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 l
            rder border-emerald-500 shadow-md">
              Аналогия 1: Судoku (P vs NP)
            </div>
            <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс <b>NP</b> –
енную сетку (ответ), он БЫСТРО (за класс P)
          </div>
          <!-- Аналогия 2 -->
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
              rder border-purple-500 shadow-md">
                Аналогия 2: Машина-переводчик
              </div>
              <p class="text-slate-300 text-sm mb

```

```
(V + E).</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия: Матрица = Таблица
    </div>
    <p class="text-slate-300 text-sm mb
гупо, но проверка "знает ли Вася Петю" мгр
    </div>
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
        Аналогия: Списки = Контакты
      </div>
      <p class="text-slate-300 text-sm mb
т. Оптимально для почти всех задач.</p>
    </div>
  </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-indigo-400"

  <div class="grid grid-cols-1 xl:grid-cols-2"
    <!-- Аналогия DFS -->
    <div class="bg-slate-800 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
        ♂ DFS (Поиск в глубину) = Жадный
      </div>
      <p class="text-slate-300 text-sm mb
        <b>Что делает:</b> Жадный алгоритм находит ближайшую
ую вершину (например, минимальную по номеру) и пробует другой путь.
      </p>
```

```
vector<vector<int>>> adj;
```

```
void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}
```

```
<pre class="bg-slate-950 p-3 rounded
```

```
// BFS (Очередь)
queue<int> q;
q.push(start_node);
vis[start_node] = true;
```

```
while(!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : adj[v]) {
        if (!vis[u]) {
            vis[u] = true;
            q.push(u);
        }
    }
}
```

```
</div>
```

```
</details>
```

```
</div>
```

```
</div>
```

```
</section>`,
```

```
},
```

```
{
```

```
    id: "graph-planar-colors",
```

```
    title: "7. Графы. Планарные. Покраска",
```

```
    type: "html",
```

```
    description: "Формула Эйлера и теорема о 4 красках.",
```

```
    category: "Графы. Теория",
```

```
    content: `
```

```
<section id="graph-planar-colors" class="mb-12 scroll-m
```

```
    <div class="flex items-center mb-6 flex-wrap gap-3"
```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-emerald-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-emerald-300 italic"
      <p class="text-xl font-mono text-white">
        ество компонент.</p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2"
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 b
          rder border-emerald-500 shadow-md">
            Аналогия 1: Острова
          </div>
          <p class="text-slate-300 text-sm mb"
            карту – остались ли серые (неисследованные)
            ь через океан – столько и компонент.</p>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "top-sort",
    title: "9. Графы. Топологическая сортировка",
    type: "html",
    description: "Разложение задач по порядку выполнения",
    category: "Графы",
    content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border"
      <h3 class="text-xl font-bold text-blue-400

```

```

void dfs(int v) {
    visited[v] = true;
    for (int to : adj[v]) {
        if (!visited[to]) {
            dfs(to);
        }
    }
    ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
    reverse(ans.begin(), ans.end()); // Разворачиваем с
}

```

```

        </div>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая-Жару.",
        category: "Графы. Продвинутые",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">10. Графы. Компоненты сильной связности (SCC)</span>
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-700">Графы. Компоненты сильной связности (SCC)</h2>
    </div>

    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">

```

```

        <p class="text-slate-300 text-sm mb-4">
            <b>SCC (Билет 10)</b> склеивает
        </p><br><br>
        <p class="text-slate-300 text-sm mb-4">
            <b>Точки сочленения (Билет 12)</b>
            <b>Точка сочленения</b> — это вершина графа, удаление которой
            разрушает связность монолита. Вырвешь точку сочленения —
            граф распадется.
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg p-4 border border-slate-700/50">
    <summary class="p-4 font-bold text-slate-300">
        Душный мод: Код Алгоритм Косарайю
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <p class="text-emerald-400 font-bold">Алгоритм Косарайю</p>
        <ol class="list-decimal list-inside">
            <li>Запускаем DFS на исходном графе, чтобы найти
            <code>strongly connected components</code> (сильно связные компоненты).
            <li>Разворачиваем этот список (то есть берем компоненты в обратном порядке).
            <li>Переворачиваем все ребра в графе.
            <li>Идем по <code>order</code>: для каждой вершины v в порядке, обратном тому,
            в котором они были обработаны в первом DFS, делаем следующее:
            <li>Если v не имеет соседей в обратном графе (то есть все ребра,
            выходящие из v, были удалены), то v — это вершина, которая не принадлежит
            ни одной из сильно связных компонент, кроме самой себя.
            <li>Иначе, пусть w — это первый сосед v в обратном графе, который
            уже был обработан. Тогда v принадлежит той же сильно связной компоненте,
            что и w.
            <li>Повторяем процесс для всех вершин.
        </ol>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "graph-bridges",
        title: "11. Графы. Мосты",
        type: "html",
        description: "Рёбра, удаление которых расхреначивает граф",
        category: "Графы. Продвинутое",
        content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">

```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">
        <div class="text-left font-mono text-sm">
          <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
          <div class="p-3 bg-slate-950 rounded">
            <span class="text-rose-400 font">
              <p class="text-xl font-bold tex">
                <p class="text-xs text-slate-400">
              </p>
            </div>
          </div>
        </div>
      </div>
    </div>

    <p class="text-slate-300 text-sm">
      <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
    </p>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2 gap">
    <div class="bg-slate-800 p-6 rounded-lg bor">
      <div class="absolute -top-3 left-4 bg-s
rder-rose-500 shadow-md">
        Аналогия: Главный перекресток
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Представь город, где два района сое
её перекроют (удалят вершину) – районы буд
g>хрупкость</strong> системы.
      </p>
    </div>
```



```

        if (low[to] >= tin[v] && p != -1)
            IS_CUTPOINT(v);
        ++children;
    }
}
if (p == -1 && children > 1)
    IS_CUTPOINT(v);
}

```

```

        </div>
    </div>
</details>

```

```

<div class="bg-indigo-950/60 p-6 rounded-xl border-2 border-indigo-400">
    <h4 class="text-lg font-bold text-indigo-400">Глубокий дуальный мост
    <p class="text-slate-300 text-sm mb-4">
        Это глубокий дуальный мост между ориентированным и неориентированным графом.
    </p>
    <ul class="list-disc list-inside text-sm text-slate-300">
        <li><strong class="text-rose-400">Точки сочленения</strong> и показывают физическую уязвимость связности графа.
        <li><strong class="text-emerald-400">SCC</strong> (сильно связные компоненты) и показывают структурную устойчивость графа.
        <li><strong>Как точка сочленения рушит связность графа?</strong>
            Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая точка сочленения в этом графе — это и есть критическая SCC! Если удалить эту SCC, граф распадется. Одно слабое звено изолирует целые касты SCC.
        </li>
    </ul>
</div>
</div>
</section>`,
},
{
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
}

```

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
}
```

```
interface Edge {
  u: number;
  v: number;
}
```

```
const nodes: Node[] = [
  { id: 0, x: 250, y: 50, label: "0" },
  { id: 1, x: 100, y: 150, label: "1" },
  { id: 2, x: 250, y: 250, label: "2" },
  { id: 3, x: 400, y: 150, label: "3" },
  { id: 4, x: 550, y: 150, label: "4" },
  { id: 5, x: 700, y: 50, label: "5" },
  { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 },
  { u: 3, v: 4 }, // 3 and 4 are articulation points (3
  { u: 4, v: 5 },
  { u: 5, v: 6 },
  { u: 6, v: 4 },
];
```

```

        tin: [...currentTin],
        low: [...currentLow],
    });
};

recordStep("Начало DFS (Тарьян) для поиска точек со

const dfs = (v: number, p: number = -1) => {
    visited.add(v);
    currentTin[v] = currentLow[v] = timer++;
    let children = 0;

    recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

    for (const to of adj[v]) {
        if (to === p) continue;

        if (visited.has(to)) {
            currentLow[v] = Math.min(currentLow[v], curren
            recordStep(
                `Обратное ребро ${v}-${to}. Обновляем low[${
                v,
            });
        } else {
            children++;
            dfs(to, v);
            currentLow[v] = Math.min(currentLow[v], curren

            recordStep(
                `Возврат в ${v} из ${to}. Обновляем low[${v
                v,
            });

            if (currentLow[to] >= currentTin[v] && p !==
                ap.add(v);
            recordStep(
                `Найдена точка сочленения: вершина ${v}
                v,

```

```

useEffect(() => {
  let interval: NodeJS.Timeout;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    interval = setInterval(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIndex(0);
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Info className="w-5 h-5 text-rose-400" />
        Моделирование поиска точек сочленения
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
        text-white"
      >
        {isPlaying ? (
          <Pause className="w-4 h-4 ml-1" />
        ) : (
          <Play className="w-4 h-4 ml-1" />
        )}
        {isPlaying ? "Пауза " : "Старт "}

```

```

    { /* Nodes */ }
    { nodes.map({ node } => {
      const isCurrent = currentStep.currentNode === node.id
      const isVisited = currentStep.visited.has(node.id)
      const isAp = currentStep.ap.has(node.id);

      const fill = isCurrent
        ? "#3b82f6"
        : isAp
          ? "#e11d48"
          : isVisited
            ? "#10b981"
            : "#1e293b";
      const stroke = isCurrent
        ? "#60a5fa"
        : isAp
          ? "#f43f5e"
          : isVisited
            ? "#34d399"
            : "#334155";
      const r = isAp ? 24 : 20;

      return (
        <g key={node.id}>
          <circle
            cx={node.x}
            cy={node.y}
            r={r}
            fill={fill}
            stroke={stroke}
            strokeWidth="3"
            className="transition-all duration-300"
          />
          <text
            x={node.x}
            y={node.y}
            dy=".3em"
            textAnchor="middle"

```

```

    }}}}
  </svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
  <h4 className="text-sm font-bold text-slate-400">
    Статус алгоритма
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-white font-medium min-h-16">
      {currentStep.desc}
    </p>
  </div>

  <div className="space-y-4 flex-1 overflow-y-ai
    <div className="mb-4">
      <h5 className="text-xs text-slate-500 font-medium">
        Точка сочленения
      </h5>
      <div className="flex items-center gap-2">
        <div className="w-3 h-3 rounded-full bg-slate-900">
          Точка сочленения
        </div>
      </div>
      <div className="flex items-center gap-2">
        <div className="w-3 h-3 rounded-full bg-slate-900">
          Посещена
        </div>
      </div>
      <div className="flex items-center gap-2">
        <div className="w-3 h-3 rounded-full bg-slate-900">
          Текущая
        </div>
      </div>
      <div className="flex items-center gap-2">
        <div className="w-8 h-1 bg-rose-500 border">
          Мост
        </div>
      </div>
    </div>

    <div>
      <h5 className="text-xs text-slate-500 font-medium">

```

```
import { useState, useEffect } from 'react';
import { useVizSync } from '../hooks/useVizSync';
import { useVizControl } from '../hooks/useVizControl';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
  previous: { [key: string]: string | null };
  log: string;
  hasNegativeCycle?: boolean;
  activeCodeLine: number;
}

export default function BellmanFordViz() {
  const [hasCycle, setHasCycle] = useState(false);
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodes: Node[] = [
    { id: 'S', x: 60, y: 150, name: 'База (S)' },
    { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
    { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
    { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
    { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
    { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
    { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
  ];

  const getEdges = (cycle: boolean): Edge[] => {
    const defaultEdges = [
      { u: 'S', v: 'A', w: 4 },
      { u: 'S', v: 'B', w: 5 },
    ];
  };
}
```

```

const simulationSteps: Step[] = [];
let dist: { [key: string]: number } = { S: 0, A: 999 };
let prev: { [key: string]: string | null } = { S: null };

simulationSteps.push({
  iteration: 0, checkingEdge: null,
  distances: { ...dist }, previous: { ...prev },
  log: '    Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
  activeCodeLine: 2,
});

const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
  let anyUpdate = false;
  for (const edge of edges) {
    const uDist = dist[edge.u];
    const vDist = dist[edge.v];

    let updated = false;
    if (uDist !== 999 && uDist + edge.w < vDist) {
      dist = { ...dist, [edge.v]: uDist + edge.w };
      prev = { ...prev, [edge.v]: edge.u };
      updated = true;
      anyUpdate = true;
    }

    simulationSteps.push({
      iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
      distances: { ...dist }, previous: { ...prev },
      log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}. ${
        updated ? 'Релаксация успешна! dist[${edge.v}] = ' : ''
      }${dist[edge.v]}`,
      activeCodeLine: updated ? 6 : 5,
    });
  }

  if (!anyUpdate && iter < vCount - 1) {
    simulationSteps.push({
      iteration: iter, checkingEdge: null,
      distances: { ...dist }, previous: { ...prev },
      log: `Итерация ${iter}: Нет изменений. Переходим к следующей итерации.`,
      activeCodeLine: 7,
    });
  }
}

```



```

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

useVizSync("bellman-ford", {
  n: 7,
  m: edges.length,
  i: currentStep?.iteration ?? "-",
  k: currentStep?.checkingEdge?.u ?? "-",
  j: currentStep?.checkingEdge?.v ?? "-",
}, currentStepIndex);
useVizControl("bellman-ford", {
  onStep: (line) => { setIsPlaying(false); if (typeof line === "number") {
    setCurrentStepIndex((v) => Math.min(v + 1, steps.length - 1));
  }
},
  onReset: () => { setIsPlaying(false); setCurrentStepIndex(0); },
  onPlay: () => setIsPlaying(true),
  onPause: () => setIsPlaying(false),
});

if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-3">
      <div className="p-2.5 bg-blue-600 text-white text-center">
        <span className="text-2xl"> </span>
      </div>
      <div>
        <h3 className="text-2xl font-bold text-white">

```

```

<defs>
  <marker id="arrow-bf-normal" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
  <marker id="arrow-bf-active" markerWidth=
<path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
  <marker id="arrow-bf-cycle" markerWidth=
path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
</defs>

```

```

{edges.map((edge, idx) => {
  const uNode = nodes.find((n) => n.id === u);
  const vNode = nodes.find((n) => n.id === v);
  const isChecking = currentStep.checkingEdges;
  const isNeg = edge.w < 0;
  const isCyclePart = hasCycle && ((edge.u === u && edge.v === 'D' || edge.v === v && edge.u === 'D'));

```

```

  let strokeColor = '#475569';
  let marker = 'url(#arrow-bf-normal)';
  if (isChecking) { strokeColor = '#f59e0b';
  else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#e11d48';

```

```

  return (
    <g key={idx}>
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={isChecking && isCyclePart ? 4 : 2} marker={isChecking && isCyclePart ? '4,4' : 'none'} />
      <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#64748b'} stroke={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#64748b'} />
      <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Neg' : 'Dist'} font-size="11" font-weight="bold" />
    </g>
  );
}}

```

```

{nodes.map((node) => {
  const dist = currentStep.distances[node.id];

```

```

        )
      }}}}
    </div>
  }}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-950">
        Таблица расстояний
      </h4>
      <div className="grid grid-cols-4 gap-2 text-rose-950">
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge === n.id;
          return (
            <div key={n.id} className={`p-2 rounded-lg
              ${isTgt ? 'border-slate-700/60 bg-slate-800/40' : ''}`}>
              <div className="text-xs text-slate-400">
                {n.id}
              </div>
              <div className={`font-mono font-bold text-rose-950`}
                >
                {d}
              </div>
            </div>
          );
        })}
      </div>
    </div>
  </div>
</div>

  {/* Код алгоритма */}
  <div className="w-full lg:w-1/2 bg-slate-900/60">
    <div>
      <h4 className="text-lg font-bold text-slate-400">
        Код алгоритма
      </h4>
      <div className="bg-slate-950/80 rounded-lg p-4">
        {codeLines.map(item => (
          <div key={item.line} className={`py-1.5 px-2
            ${item.line === currentStep.startLine ? 'bg-blue-900/80 text-amber-400' : ''}`}>
            {item.code}
          </div>
        ))}
      </div>
    </div>
  </div>
</div>

```

```
0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],  
};
```

```
interface Step {  
  activeLine: number;  
  queue: number[];  
  visited: number[];  
  currentNode: number | null;  
  logs: string;  
}
```

```
const generateBfsSteps = (): Step[] => {  
  const steps: Step[] = [];  
  const queue: number[] = [];  
  const visited = new Set<number>();
```

```
  // Init
```

```
  steps.push({  
    activeLine: 1,  
    queue: [],  
    visited: [],  
    currentNode: null,  
    logs: "Инициализация: начинаем с корня (0)."  
  });
```

```
  queue.push(0);
```

```
  visited.add(0);
```

```
  steps.push({  
    activeLine: 2,  
    queue: [...queue],  
    visited: Array.from(visited),  
    currentNode: null,
```

```
    logs: "Добавляем стартовую вершину 0 в очередь и по  
  });
```

```
  while (queue.length > 0) {
```

```
    steps.push({  
      activeLine: 4,
```

```

    }
  } else {
    steps.push({
      activeLine: 6,
      queue: [...queue],
      visited: Array.from(visited),
      currentNode: curr,
      logs: `Соседей нет или все уже посещены.`
    });
  }
}

steps.push({
  activeLine: 12,
  queue: [],
  visited: Array.from(visited),
  currentNode: null,
  logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
  const [stepIdx, setStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const step = STEPS[stepIdx];

  const handleNext = useCallback(() => {
    setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
  }, []);

  // @ts-expect-error зарезервировано под кнопку «назад»
  const handlePrev = useCallback(() => {
    setStepIdx(prev => Math.max(prev - 1, 0));
  }, []);
}

```

```

disabled={stepIdx >= STEPS.length - 1}
className="p-2 text-emerald-400 hover:text-
ed: hover: bg-transparent"
title="Шаг вперед"
>
  <StepForward className="w-5 h-5" />
</button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
  { /* Визуал графа */ }
  <div className="bg-slate-800 border border-slate
    50px]">
    <svg className="w-full h-full min-h-[350px]" *
      { /* Рёбра */ }
      { EDGES.map((e, idx) => {
        const n1 = NODES.find(n => n.id === e.from)
        const n2 = NODES.find(n => n.id === e.to)
        const isVisited = step.visited.includes(n
        return (
          <line
            key={idx}
            x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
            className={`transition-all duration-500
          `} `)
        )
      })
    }
  </div>
  { /* Вершины */ }
  { NODES.map(n => {
    const isCurrent = step.currentNode === n.id
    const isInQueue = step.queue.includes(n.id)
    const isVisited = step.visited.includes(n.id)

    let fill = '#1e293b'; // slate-800
    let stroke = '#475569'; // slate-600
  })
}

```

```

    });
  }}}
</svg>
</div>

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono font-size-sm tracking-wider">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">Mar {step}
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start)' },
        { line: 3, code: '' },
        { line: 4, code: 'while not queue.isEmpty()' },
        { line: 5, code: '  node = queue.pop()' },
        { line: 6, code: '  for neighbor in graph[node]:' },
        { line: 7, code: '    if neighbor not in visited:' },
        { line: 8, code: '      visited.add(neighbor)' },
        { line: 9, code: '      queue.push(neighbor)' },
      ]}.map((cl) => {
        const isActive = step.activeLine === cl.line
        return (
          <div key={cl.line} className={`px-2 py-2 border border-l-2 border-blue-400 -ml-0.5` : 'text-blue-400'}>
            <span className="opacity-40 w-6 inline-block">  </span>
            <span className="whitespace-pre">{cl.code}</span>
          </div>
        )
      })
    </div>
  </div>
</div>

```

```
-----
const imageMap: Record<string, { src: string; alt: string;
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: '    Добрый – только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',
    caption: '    Тяжёлая, но ей пофиг на ямы!',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Все ко Всем Флойд',
    caption: '    Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};

export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;

  return (
    <div>
      <div className={`rounded-2xl overflow-hidden border-2 border-gray-200`} >
        <img src={info.src} alt={info.alt} className="w-full h-full" />
      </div>
      <p className={`text-center text-xs mt-2 font-bold`} >{info.caption}</p>
    </div>
  );
}
```



```

        {index + 1} / {total}
      </span>
      <button
        disabled={!next}
        onClick={() => next && onGo(next.id)}
        title={next ? next.title : "Это последняя тема"}
        className="flex items-center gap-1 px-2.5 py-1.5
          :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
      >
        <span className="hidden sm:inline">Вперёд</span>
        <ChevronRight className="w-4 h-4" />
      </button>
    </div>
  );
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4" >
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white
          transition-all"
      >
        <div className="flex items-center gap-1.5 text-slate-700" >
          <ChevronLeft className="w-3.5 h-3.5" /> Предыдущая
        </div>
        <div className="text-sm font-bold text-slate-700">
          {prev.title}
        </div>
      </button>
    ) : (
      <div className="hidden sm:block" />
    )}
    {next && (
      <button
        onClick={() => onGo(next.id)}
        className="group text-right p-4 rounded-xl bg-white
          transition-all"
      >
        <div className="flex items-center gap-1.5 text-slate-700" >
          Следующая <ChevronRight className="w-3.5 h-3.5" />
        </div>
        <div className="text-sm font-bold text-slate-700">
          {next.title}
        </div>
      </button>
    )}
  </nav>
);

```

```

const isTouch = () => typeof window !== "undefined" && true;

export const ChapterView: React.FC<Props> = ({ chapter,
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");

  useTermHints(contentRef, [chapter.id]);

  const cancelHide = useCallback(() => {
    if (hideTimer.current) {
      window.clearTimeout(hideTimer.current);
      hideTimer.current = null;
    }
  }, []);

  const scheduleHide = useCallback(() => {
    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(anchor), 2500);
  }, [cancelHide]);

  useEffect(() => {
    setAnchor(null);
    setSaving("idle");
  }, [chapter.id]);

  const saveHtml = useCallback(async () => {
    setSaving("work");
    try {
      await downloadChapterHtml(chapter);
      setSaving("done");
      window.setTimeout(() => setSaving("idle"), 2500);
    } catch {
      setSaving("idle");
    }
  }, [chapter]);

```

```

const fullCode = getFullCode(chapter.id);
const [showCode, setShowCode] = useState(false);
useEffect(() => setShowCode(false), [chapter.id]);

return (
  <>
    <article className="chapter-body">
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-white border-emerald-500"
          style={{width: 300px, height: 30px, border: 1px solid #2e8b57, border-radius: 5px;}}
        >
          {saving === "work" ? <Loader2 className="w-3 h-3 text-emerald-400" /> : <Download className="w-3 h-3 text-emerald-400" />}
          {saving === "done" ? "Файл сохранён" : "Скачать"}
        </button>
        <button
          onClick={() => window.print()}
          title="Распечатать или сохранить в PDF"
          className="flex items-center gap-1.5 text-white border-indigo-500"
          style={{width: 300px, height: 30px, border: 1px solid #483d8b, border-radius: 5px;}}
        >
          <Printer className="w-3.5 h-3.5" /> Печать
        </button>
        <span className="text-[11px] text-slate-500 h-3.5 flex items-center gap-1.5">
          {onOpenCompiler && {
            <button
              onClick={onOpenCompiler}
              className="ml-auto hidden sm:inline-flex items-center gap-1.5 text-white shadow transition"
              style={{width: 150px, height: 30px, border: 1px solid #2e8b57, border-radius: 5px;}}
              title="Открыть компилятор – i,j и 4+1"
            >
              <Terminal className="w-3.5 h-3.5" /> Компилятор
            </button>
          }}
        </span>
      </div>
    </article>
  </>
)

```

```

{viz && {
  <section id="chapter-viz" className="mt-8 scr
  <div className="flex items-center justify-b
    <h3 className="flex items-center gap-2 te
      <Sparkles className="w-4 h-4 text-indigo
        Демонстрация: {viz.title}
      </h3>
    <div className="flex items-center gap-2">
      {fullCode && {
        <button
          onClick={() => setShowCode((v) => !
          className="flex items-center gap-1.5
t-slate-300 hover:text-white hover:border-in
        >
          <Eye className="w-3.5 h-3.5" /> {sh
        </button>
      }}
    {onOpenCompiler && {
      <button
        onClick={onOpenCompiler}
        className="flex items-center gap-1.5
700 text-emerald-300 hover:text-white hover
      >
        <Terminal className="w-3.5 h-3.5" />
      </button>
    }}
  </div>
</div>
{viz.hint && <p className="text-xs text-sla
<div className="rounded-xl border border-sla
  <viz.Component />
</div>
{fullCode && showCode && {
  <div className="mt-3 rounded-xl border bo
    <div className="flex items-center justifi
      <span className="text-xs font-mono te
span>

```

```
interface Props {
  chapterId: string;
  onOpenFull: () => void;
  onClose?: () => void;
}

export function CompilerPanelContent({ chapterId, onOpenFull, onClose }: Props) {
  const meta = getFallbackMeta(chapterId);
  const starter = getMinimalStarter(chapterId);
  const lines = useMemo(() => starter.split("\n"), [starter]);
  const vizObj = useMemo(() => new VizObject(chapterId, meta), [chapterId, meta]);

  const [live, setLive] = useState<Record<string, string>> | null>(null);
  const [playing, setPlaying] = useState(false);
  const [line, setLine] = useState(0);

  // внешний highlight от viz (когда viz шагает сама)
  useEffect(() => {
    const handler = (e: Event) => {
      const d = (e as CustomEvent).detail;
      if (d?.chapterId === chapterId || !d?.chapterId) return;
    };
    window.addEventListener("viz:sync", handler as EventListener);
    return () => window.removeEventListener("viz:sync", handler as EventListener);
  }, [chapterId]);

  useEffect(() => {
    setLine(0);
    setLive(null);
  }, [chapterId]);

  const emitHighlight = (nextLine: number) => {
    const codeLine = lines[nextLine] ?? "";
    const parsed = parseAssignments(codeLine);
    // обновляем объект
    for (const [k, v] of Object.entries(parsed)) vizObj[k] = v;
    const vars = { ...vizObj.vars.vars } as Record<string, string>;
  };
}
```

```

window.dispatchEvent(new CustomEvent("viz:control
setPlaying(action === "play");
// при play – начать автопроход по строкам
if (action === "play") {
  let cur = line;
  const iv = setInterval(() => {
    cur = Math.min(cur + 1, lines.length - 1);
    setLine(cur);
    emitHighlight(cur);
    if (cur >= lines.length - 1) {
      clearInterval(iv);
      setPlaying(false);
    }
  }, 900);
  // сохранить interval для паузы – упрощённо чер
window.addEventListener("viz:control", function
  const d = (e as CustomEvent).detail;
  if (d?.action === "pause") {
    clearInterval(iv);
    window.removeEventListener("viz:control", h
  }
} as EventListener);
}
}
};

```

```

const copy = () => navigator.clipboard.writeText(star

```

```

// при первом рендере подсветить строку 0 (i,j=0,0)
useEffect(() => {
  // небольшая задержка чтобы viz успел подписаться
  const t = setTimeout(() => emitHighlight(0), 200);
  return () => clearTimeout(t);
}, [chapterId, starter]);

```

```

return (
  <div className="flex flex-col h-full bg-[#0b1220]">
    <div className="shrink-0 flex items-center justifi

```

```

<div className="flex items-center gap-1.5 text-
  <span className={`w-2 h-2 rounded-full ${live
    WATCHES - var {line + 1}/{lines.length} · {vi
  </div>
<div className="flex flex-wrap gap-1">
  {live && Object.keys(live).length > 0 ? (
    Object.entries(live).map(([k, v]) => (
      <span key={k} className="px-1.5 py-0.5 ro
        {k} = {String(v)}
      </span>
    ))
  ) : (
    <span className="text-slate-500">{lines[lin
    )}
  </div>
<div className="mt-1 text-[10px] text-slate-500
  ruct)</div>
</div>

<div className="flex-1 overflow-auto bg-[#080d14]
  <div className="flex min-h-full">
    <div className="shrink-0 bg-[#0f172a] border-
      {lines.map((_, i) => (
        <div key={i} className={`px-1 rounded ${i
          {i + 1}
        </div>
      )}}
    </div>
    <div className="flex-1 py-3">
      {lines.map((l, i) => (
        <div key={i} className={`px-3 whitespace-p
          text-white" : "text-slate-300 border-l-2 bo
          {l || " " }
        </div>
      )}}
    </div>
  </div>
</div>

```

```

{ line: "                if (low[to] > tin[v]) {", highlight: "normal" },
{ line: "                // ЭТО МОСТ!", highlight: "normal" },
{ line: "                }", highlight: "normal" },
{ line: "            }", highlight: "normal" },
{ line: "        }", highlight: "normal" },
{ line: "    }", highlight: "normal" },
};

```

```

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
  activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

```

```

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },

```



```

    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 2, visitedIndices: [0,1,6,2],
    tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
    activeEdge: null, backEdge: [2,0],
    log: "Видим соседа А (уже посещён)! Обратное ребро
    activeCodeLine: [7,8,9]
},
{
    currentNode: 3, visitedIndices: [0,1,6,2,3],
    tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
    activeEdge: [2,3], backEdge: null,
    log: "Идём C→D. tin[D]=5, low[D]=5.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 4, visitedIndices: [0,1,6,2,3,4],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
    activeEdge: [3,4], backEdge: null,
    log: "D→E. tin[E]=6, low[E]=6.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
    activeEdge: [4,5], backEdge: null,
    log: "E→F. tin[F]=7, low[F]=7.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
    activeEdge: null, backEdge: [5,3],
    log: "Из F видим D (посещён). Обратное ребро (F,D).
    activeCodeLine: [7,8,9]
},
{

```

```

    log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм закончен.",
    activeCodeLine: []
  },
];

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  const [dfsAuto, setDfsAuto] = useState(false);
  const dfsTimer = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    if (dfsAuto) {
      dfsTimer.current = setInterval(() => {
        setDfsIdx(prev => {
          if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
          return prev + 1;
        });
      }, 2500);
    }
    return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
  }, [dfsAuto]);

  const step = DFS_STEPS[dfsIdx];

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h4 className="text-base font-bold text-white">Найдем мосты!
        <div className="flex items-center gap-1 bg-slate-800 p-1">
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
            disabled={dfsIdx === 0}
            className="p-1.5 hover:bg-slate-700 rounded text-white">
              <Undo className="h-3.5 w-3.5" />
            </button>
          <button onClick={() => setDfsAuto(!dfsAuto)}
            className={`px-2.5 py-1 rounded text-[10px] ${
              dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
            }`} />
        </div>
      </div>
    </div>
  );
}

```

```

        stroke={sc} strokeWidth={sw} strokeDash
        className="transition-all duration-500"
    }}}
    {step.currentNode !== null && step.currentNode !== null && step.currentNode !== null &&
    const n = GRAPH_NODES.find(x => x.id ===
    if (!n) return null;
    return <circle cx={n.x} cy={n.y} r={18} f
>;
    }}())
    {GRAPH_NODES.map(n => {
    const cur = step.currentNode === n.id;
    const vis = step.visitedIndices.includes(n.id);
    const st = step.stack.includes(n.id);
    return <g key={n.id}>
    <circle cx={n.x} cy={n.y} r={12}
    fill={cur ? "#10b981" : st ? "#064e3b" : "#fff"}
    stroke={cur ? "#fff" : st ? "#34d399" : "#000"}
    strokeWidth={2.5}
    className="transition-all duration-300"
    <text x={n.x} y={n.y+3} textAnchor="middle">
    label}</text>
    {vis && (
    <div
    <rect x={n.x-18} y={n.y-28} width={36} height={10}
    "transition-all duration-300" />
    <text x={n.x} y={n.y-20} textAnchor="center">
    } l:{step.low[n.id]||"?"}</text>
    </div>
    )}
    </g>;
    }}}
</svg>
<div className="flex items-center justify-between">
    <span>Шаг {dfsIdx+1}/{DFS_STEPS.length}</span>
    <div className="flex-1 mx-2 bg-slate-950 h-10 rounded" style={style}>
    <div className="bg-indigo-600 h-full transition-all duration-300">
    </div>
    </div>
    </div>

```

```

        : "bg-slate-900 text-slate-400"
      }`}>Вершина {GRAPH_NODES.find(n=>n.id === id)}
      <span className="text-slate-500" style={step.bridges} />
    </div>
  ))
}
</div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border border-slate-800" style={step.bridges} >
  <p className="text-[11px] text-slate-300 leading-tight">
    <div className="mt-2 pt-2 border-t border-slate-800">
      <span className="text-slate-400">Мосты:</span>
      <span className="font-mono font-bold text-slate-300">
        {step.bridges.length > 0 ? step.bridges : ''}
      </span>
    </div>
  </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 39/87: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect } from 'react';
import { useVizSync } from '../hooks/useVizSync';
import { useVizControl } from '../hooks/useVizControl';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {

```

```
});
```

```
const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push((0, start))' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем элемент с минимальным расстоянием' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push((dist[v], v))' },
  { line: 11, text: '    return dist // Все кратчайшие пути найдены' },
];
```

```
const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);
```

```
useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false, B: false };
  const prev: Record<string, string | null> = { A: null, B: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из Пикет-Пост-Оффиса',
    activeCodeLine: 2,
  });
});
```

```
for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
```

```

    }
  }

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра всё доставил! Все возможные кратчайшие пути найдены! ',
    activeCodeLine: 11,
  });

  setSteps(simulationSteps);
}, []);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

useVizSync("dijkstra", {
  n: 7,
  m: 11,
  i: currentStep?.currentNode ?? "-",
  j: currentStep?.checkingEdge?.v ?? "-",
  k: currentStep?.checkingEdge ? edges.find((e) => e.v === currentStep?.checkingEdge?.v) : "-",
}, currentStepIndex);

useVizControl("dijkstra", {
  onStep: (line) => { setIsPlaying(false); if (typeof line === 'number') {
    setCurrentStepIndex((v) => Math.min(v + 1, steps.length - 1));
  }
}

```

```

        onClick={() => { setIsPlaying(false); if (c
        disabled={currentStepIndex >= steps.length
        className="flex items-center gap-1 bg-indigo
        px-3 py-2 rounded-lg text-sm font-medium tr
    >
      War
    </button>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6 ml
  {/* Граф */}
  <div className="w-full lg:w-2/3 bg-indigo-950/2
    stify-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-indi
      ont-bold shadow-sm">
        War {currentStepIndex + 1} из {steps.length
    </div>

  <svg className="w-full h-auto max-h-[500px]"
    <defs>
      <marker id="arrow-dh-normal" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#47556
      </marker>
      <marker id="arrow-dh-active" markerWidth=
        <path d="M0,0 L6,3 L0,6 z" fill="#f59e0
      </marker>
      <marker id="arrow-dh-opt" markerWidth="6"
        <path d="M0,0 L6,3 L0,6 z" fill="#10b98
      </marker>
    </defs>
    {/* Пёпа */}
    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id ===
      const vNode = nodes.find((n) => n.id ===
      const isChecking =
        currentStep.checkingEdge &&
        (currentStep.checkingEdge.u === edge.u

```

```

        const dist = currentStep.distances[node.id]

        return (
          <g key={node.id} className="transition-
            <circle
              cx={node.x} cy={node.y}
              r={isCurrent ? 22 : 18}
              fill={isCurrent ? '#4f46e5' : isVis
              stroke={isCurrent ? '#a5b4fc' : isV
              strokeWidth={isCurrent ? 4 : 2}
              className="transition-all duration-
            />
            <text x={node.x} y={node.y + 5} fill=
              {node.id}
            </text>
            <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
              {dist === 999 ? '∞' : `d=${dist}`}
            </text>
            <text x={node.x} y={node.y + 35} fill=
              {node.name.split(' ')[0]}
            </text>
          </g>
        );
      }
    }
  </svg>
  <div className="w-full bg-slate-950/80 p-4 ro
    <p className="font-semibold text-amber-400
    <p className="min-h-[40px] flex items-cente
  </div>
</div>

  /* Список смежности */
  <div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2.5 flex-1 overflow-y
      {Object.keys(adjList).map((u) => {
        const isCurrentNode = currentStep.current

```



```

<h4 className="text-lg font-bold text-rose-900">
<div className="overflow-x-auto">
  <table className="w-full text-left border-collapse">
    <thead>
      <tr className="border-b border-rose-900">
        <th className="py-2 px-3">Вершина</th>
        <th className="py-2 px-3">Расстояние</th>
        <th className="py-2 px-3">Из (пред.)</th>
      </tr>
    </thead>
    <tbody className="text-sm">
      {nodes.map((n) => {
        const dist = currentStep.distances[n.id]
        const prev = currentStep.previous[n.id]
        const vis = currentStep.visited[n.id]
        const isCurr = currentStep.currentNode === n.id

        return (
          <tr key={n.id} className={`border-b border-rose-900 ${
            isCurr ? 'bg-indigo-950/60 font-medium' : ''
          }`}>
            <td className="py-2.5 px-3 flex items-center">
              <span className={`w-2 h-2 rounded ${
                vis ? 'bg-emerald-500' : 'bg-gray-400'
              }`} style={n.isLeaf ? {display: 'block', width: 10px, height: 10px} : {}}>
                {n.name}
              </span>
            </td>
            <td className="py-2.5 px-3 font-mono">
              {dist === 999 ? '∞' : dist}
            </td>
            <td className="py-2.5 px-3 text-gray-500">
              {prev || '-'}
            </td>
          </tr>
        )
      })}
    </tbody>
  </table>
</div>
</div>

```

```

    id: number;
    n: number;
    action: 'call' | 'memo_hit' | 'base_case' | 'calc' |
    desc: string;
    codeLine: number;
    memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
  /* 1 */ "function fibMemo(n, memo = {}) {",
  /* 2 */ "    if (n in memo) return memo[n];",
  /* 3 */ "    if (n <= 2) return 1;",
  /* 4 */ "    memo[n] = fibMemo(n - 1, memo) + fibMemo",
  /* 5 */ "    return memo[n];",
  /* 6 */ "}"
];

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

  useEffect(() => {
    const generatedSteps: DPStep[] = [];
    let stepId = 0;
    const memoMap: { [key: number]: number } = {};

    function simulateFib(n: number): number {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вызов fibMemo(${n}). Проверяем наличие в
        codeLine: 2,
        memoState: { ...memoMap }

```

```

        generatedSteps.push({
          id: stepId++,
          n,
          action: 'calc',
          desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
          codeLine: 5,
          memoState: { ...memoMap }
        });

    return memoMap[n];
  }

const finalRes = simulateFib(targetN);

generatedSteps.push({
  id: stepId++,
  n: targetN,
  action: 'done',
  desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
  codeLine: 6,
  memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex(prev => prev + 1);
    }, speed);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
}, [steps, speed]);

```

```

<div className="flex items-center bg-slate-950" style={{width: 1000px, height: 100px}}>
  <button
    onClick={() => { setIsPlaying(false); setSteps([0]); }}
    className="p-2 text-slate-400 hover:text-white"
    title="Сбросить"
  >
    <RotateCcw className="w-4 h-4" />
  </button>
  <button
    onClick={() => { setIsPlaying(false); setSteps([0]); }}
    disabled={currentStepIndex === 0}
    className="p-2 text-slate-400 hover:text-white"
    title="Шаг назад"
  >
    <ChevronLeft className="w-5 h-5" />
  </button>
  <button
    onClick={() => setIsPlaying(!isPlaying)}
    className={`flex items-center gap-1.5 px-2 py-2
      ${isPlaying ? 'bg-amber-500 text-slate-950 hover:bg-amber-600' : 'bg-emerald-600 text-white hover:bg-emerald-700'}
    `}
  >
    {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
    {isPlaying ? 'Пауза' : 'Пуск'}
  </button>
  <button
    onClick={() => { setIsPlaying(false); setSteps([0]); }}
    disabled={currentStepIndex === steps.length - 1}
    className="p-2 text-slate-400 hover:text-white"
    title="Шаг вперед"
  >
    <ChevronRight className="w-5 h-5" />
  </button>
</div>
</div>
</div>

```



```

        <div className="p-6 bg-slate-900/50 border-
          <span className="p-1.5 bg-emerald-500/20
            <span>{currentStep?.desc}</span>
          </div>
        </div>
      )}
    </div>

    { /* Progress Bar */
    <div className="mt-8 pt-6 border-t border-slate-8
      <div className="flex items-center justify-between
        <span>Прогресс выполнения</span>
        <span>Шаг {currentStepIndex + 1} из {steps.length}</span>
      </div>
      <div className="h-2 bg-slate-950 rounded-full overflow-hidden
        <div
          className="h-full bg-gradient-to-r from-emerald-500 to-transparent
          style={{ width: `${(currentStepIndex + 1) * 100}%` }}
        ></div>
      </div>
    </div>
  </div>
);
}

```

ФАЙЛ 41/87: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState } from "react";
import { Undo } from "lucide-react";

```

```

const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },

```

```

    }
    const nextEdges = [...c1VisitedEdges, key];
    const nextPath = [...c1Path, vId];
    setC1VisitedEdges(nextEdges);
    setC1Path(nextPath);

    if (nextEdges.length === C1_EDGES.length) {
      const startDegree = C1_EDGES.filter(e => e.u === vId).length;
      const endDegree = C1_EDGES.filter(e => e.v === vId).length;
      const isCycle = startDegree % 2 === 0 && endDegree % 2 === 0;
      if (isCycle && nextPath[0] === vId) {
        setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
      } else {
        setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С`);
      }
      setC1Done(true);
    } else {
      setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
    }
  }
};

```

```

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <button onClick={resetC1} className="flex items-center justify-center gap-2 border border-slate-700 transition-all">
          <Undo className="h-3.5 w-3.5" /> Сброс
        </button>
      </div>

      <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)] border-[1px] border-slate-800">
          <svg className="w-full h-[260px] z-10 relative">
            {C1_EDGES.map((e, i) => {
              const u = C1_NODES.find(n => n.id === e.u)!;
              const v = C1_NODES.find(n => n.id === e.v)!;
            })}
          </svg>
        </div>
      </div>
    </div>
  );

```

}

ФАЙЛ 42/87: src/components/ExpressQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

import React, { useState } from 'react';
import { CheckCircle2, XCircle, Award, RotateCcw, AlertTriangle } from 'lucide-react';

```
interface Question {  
  id: number;  
  question: string;  
  options: string[];  
  correctAnswer: number;  
  explanation: string;  
}
```

```
const quizQuestions: Question[] = [  
  {  
    id: 1,  
    question: 'Что мы делаем в алгоритме Краскала, если ребро соединяет две вершины, которые уже принадлежат одному дереву (т.е. оба конца уже красные)?',  
    options: [  
      'Мы радуемся и добавляем его в минимальное остовное дерево.',  
      'Мы перекрашиваем их в синий цвет и делим граф пополам.',  
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл.',  
      'Мы вызываем алгоритм Дейкстры для поиска нового пути.',  
    ],  
    correctAnswer: 2,  
    explanation: 'Верно! Если вершины уже в одном клане, добавление ребра создаст цикл. Это нарушит структуру дерева.',  
  },  
  {  
    id: 2,  
    question: 'Почему алгоритм Дейкстры впадает в ступор на графе с отрицательными весами?',  
    options: [  
      'Из-за отсутствия базисного случая.',  
      'Из-за того, что отрицательные веса нарушают предположение о кратчайшем пути.',  
      'Из-за того, что алгоритм не учитывает отрицательные веса.',  
      'Из-за того, что отрицательные веса создают циклы с отрицательным весом.',  
    ],  
    correctAnswer: 2,  
    explanation: 'Алгоритм Дейкстры предполагает, что все веса неотрицательны. Если есть отрицательные веса, алгоритм может найти более короткий путь, чем уже найденный, что нарушает его логику.',  
  },  
];
```



```

    options: [
      'СНМ (Система непересекающихся множеств / Disjoin',
      'Стек (LIFO).',
      'Красно-черное дерево.',
      'Хэш-таблица со списками коллизий.'
    ],
    correctAnswer: 0,
    explanation: 'Блестяще! СНМ (DSU) позволяет за прак
      .'
  }
];

```

```

export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<num
  const [selectedOption, setSelectedOption] = useState<
  const [isAnswered, setIsAnswered] = useState<boolean>
  const [score, setScore] = useState<number>(0);
  const [showResults, setShowResults] = useState<boolean>

```

```

  const currentQ = quizQuestions[currentQIndex];

```

```

  const handleSelectOption = (index: number) => {
    if (isAnswered) return;
    setSelectedOption(index);
    setIsAnswered(true);
    if (index === currentQ.correctAnswer) {
      setScore(prev => prev + 1);
    }
  };

```

```

  const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
      setCurrentQIndex(prev => prev + 1);
      setSelectedOption(null);
      setIsAnswered(false);
    } else {
      setShowResults(true);
    }
  }

```

```

        <RotateCcw className="w-5 h-5" />
        <span>Пройти тест заново</span>
      </button>
    </div>
  );
}

return (
  <div className="bg-slate-900/90 border border-slate-
    >
    <div className="flex items-center justify-between"
      <div className="flex items-center gap-3">
        <span className="bg-purple-600 text-white px-3"
          Экспресс-тест
        </span>
        <h3 className="text-xl font-bold text-white">
      </div>
      <span className="text-sm font-mono text-slate-400"
        Bonpoc {currentQIndex + 1} / {quizQuestions.length}
      </span>
    </div>

    <div className="mb-8">
      <h4 className="text-xl md:text-2xl font-bold text-white">
        {currentQ.question}
      </h4>

      <div className="space-y-4">
        {currentQ.options.map((option, idx) => {
          const isSelected = selectedOption === idx;
          const isCorrect = idx === currentQ.correctAnswer;

          let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
          if (isAnswered) {
            if (isCorrect) {
              optionStyle = "bg-emerald-950/80 border border-emerald-500";
            } else if (isSelected) {
              optionStyle = "bg-rose-950/80 border-rose-500";
            }
          }
          return (
            <div className={optionStyle} key={idx}>
              {option}
            </div>
          );
        })}
      </div>
    </div>
  );
}

```

```

        <p className="text-slate-300 text-sm leading-
          {currentQ.explanation}
        </p>
      </div>
    )}

    <div className="flex justify-end pt-4 border-t bo
      <button
        onClick={handleNext}
        disabled={!isAnswered}
        className={
          "px-8 py-3.5 font-bold rounded-xl transition
          (!isAnswered
            ? "bg-slate-800 text-slate-500 border bor
            : "bg-indigo-600 hover:bg-indigo-500 acti
          }
        >
          {currentQIndex + 1 < quizQuestions.length ? '
        </button>
      </div>
    </div>
  );
};

```

ФАЙЛ 43/87: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizSync } from '../hooks/useVizSync';
import { useVizControl } from '../hooks/useVizControl';
import { VizObject } from '../viz/VizObject';

```

```

interface Step {
  k: number; i: number; j: number;
  matrix: number[][]; updated: boolean;

```

```
const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V' },
  { line: 2, text: '    dist = copy(matrix)' },
  { line: 3, text: '    for k from 0 to V - 1: // Пос' },
  { line: 4, text: '        for i from 0 to V - 1:' },
  { line: 5, text: '            for j from 0 to V - 1' },
  { line: 6, text: '                if dist[i][k] + d' },
  { line: 7, text: '                    dist[i][j] = ' },
  { line: 8, text: '    return dist' },
];
```

```
useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist = initialMatrix.map(row => [...row]);

  simulationSteps.push({
    k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: '    Инициализация матрицы прямых путей, ∞ (999)',
    activeCodeLine: 2,
  });
});
```

```
const N = 7;
for (let k = 0; k < N; k++) {
  simulationSteps.push({
    k, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: `+ Внешний цикл. Разрешаем пути через пром` ,
    activeCodeLine: 3,
  });
};
```

```
for (let i = 0; i < N; i++) {
  for (let j = 0; j < N; j++) {
    if (i === j || i === k || j === k) continue;

    const oldVal = dist[i][j];
    const viaVal = dist[i][k] + dist[k][j];
    if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal)
      dist[i][j] = viaVal;
  }
}
```

```

    k: currentStep ? (currentStep.k >= 0 ? currentStep.k : 0) : 0,
    i: currentStep ? (currentStep.i >= 0 ? currentStep.i : 0) : 0,
    j: currentStep ? (currentStep.j >= 0 ? currentStep.j : 0) : 0,
  }, currentStepIndex);
useVizControl("floyd", {
  onStep: (line) => { setIsPlaying(false); if (typeof CurrentStepIndex((v) => Math.min(v + 1, steps.length)) !== undefined) {
    onReset: () => { setIsPlaying(false); setCurrentStepIndex(0);
    onPlay: () => setIsPlaying(true),
    onPause: () => setIsPlaying(false),
  });
// подсветка из компилятора: k,i,j = 0,0,0 → шаг
useEffect(() => {
  const h = (e: Event) => {
    const d = (e as CustomEvent).detail;
    if (d.chapterId && d.chapterId !== "floyd") return;
    const v = d.vars as Record<string, any>;
    if (!v) return;
    if (v.k !== undefined && v.i !== undefined && v.j !== undefined) {
      const k = Number(v.k), i = Number(v.i), j = Number(v.j);
      vizObj.vars.set("k", k); vizObj.vars.set("i", i); vizObj.vars.set("j", j);
      const idx = steps.findIndex((s) => s.k === k && s.i === i && s.j === j);
      if (idx !== -1) { setIsPlaying(false); setCurrentStepIndex(idx);
    } else if (v.i !== undefined && v.j !== undefined) {
      const i = Number(v.i), j = Number(v.j);
      const idx = steps.findIndex((s) => s.i === i && s.j === j);
      if (idx !== -1) { setIsPlaying(false); setCurrentStepIndex(idx);
    }
  };
  window.addEventListener("viz:highlight", h as EventListener);
  return () => window.removeEventListener("viz:highlight", h as EventListener);
}, [steps, vizObj]);
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-2">
      <div className="flex items-center gap-3">

```

```

const u = parseInt(uStr);
return adjList[u].map((edge) => {
  const uNode = nodesSvg.find((n) => n.id === u);
  const vNode = nodesSvg.find((n) => n.id === v);
  const isTarget = currentStep.i === u && currentStep.j === v;
  const isVia = (currentStep.i === u && currentStep.j !== v) || (currentStep.i !== u && currentStep.j === v);
  let strokeColor = '#475569'; let marker = 'none';
  if (isTarget) { strokeColor = '#10b981'; marker = 'circle'; }
  else if (isVia) { strokeColor = '#818cf8'; marker = 'none'; }
  return (
    <g key={`-${u}-${v}`} >
      <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeDasharray={isTarget ? [4, 4] : [1, 0]} markerEnd={marker} />
      <circle cx={(uNode.x + vNode.x) / 2} cy={(uNode.y + vNode.y) / 2} r={isTarget ? 4 : 2} stroke={strokeColor} strokeDasharray={isTarget ? [4, 4] : [1, 0]} />
      <text x={(uNode.x + vNode.x) / 2} y={vNode.y + 5} fill={strokeColor} font-size="10" font-weight="bold" text-align="center">{v}</text>
    </g>
  );
});
}}}
{nodesSvg.map((node) => {
  const isK = currentStep.k === node.id;
  const isI = currentStep.i === node.id;
  const isJ = currentStep.j === node.id;
  let fill = '#1e293b'; let stroke = '#64748f';
  if (isK) { fill = '#4f46e5'; stroke = '#a6a6a6'; }
  else if (isI) { fill = '#b45309'; stroke = '#a6a6a6'; }
  else if (isJ) { fill = '#047857'; stroke = '#a6a6a6'; }
  return (
    <g key={node.id} className="transition-colors">
      <circle cx={node.x} cy={node.y} r={isK ? 4 : 2} className="transition-colors duration-200" />
      <text x={node.x} y={node.y + 5} fill={stroke} font-size="10" font-weight="bold" text-align="center">{node.id}</text>
      <text x={node.x} y={node.y + 32} fill={stroke} font-size="10" font-weight="bold" text-align="center">{node.name}</text>
    </g>
  );
});

```

```

        </div>
      </div>
    </div>

    <div className="flex flex-col lg:flex-row gap-6">
      /* Матрица расстояний */
      <div className="w-full lg:w-1/2 bg-rose-950/10">
        <div className="flex justify-between items-center">
          <h4 className="text-lg font-bold text-rose-950">
            Матрица расстояний
          </h4>
        </div>
        <div className="overflow-x-auto">
          <table className="w-full text-center border-collapse">
            <thead><tr className="bg-rose-950/40 text-rose-950">
              <th className="p-2 border-r border-rose-950">
                Шаг
              </th>
              {nodeNames.map((n, idx) => <th key={idx} className="p-2 border-r border-rose-950 font-bold" : ''>`>{n.split(' ')[0]} {idx + 1}</th>)}
            </tr></thead>
            <tbody className="text-xs font-mono">
              {currentStep.matrix.map((row, i) => (
                <tr key={i} className="border-b border-rose-950">
                  <td className={`p-2 bg-rose-950/40 font-weight-normal`} : ''>`>
                    {currentStep.k === i ? 'text-amber-400' : ''}>
                      {currentStep.k}
                    </td>
                    {nodeNames[i].split(' ')[0]} {i + 1}
                  </td>
                  {row.map((val, j) => {
                    const isUpd = currentStep.i === i
                    const isVia = currentStep.k !== currentStep.j
                    const isDiag = currentStep.i === currentStep.j
                    let bg = 'bg-slate-900/40 text-slate-50'
                    if (isUpd) bg = 'bg-emerald-900/80 text-emerald-50'
                    else if (isVia) bg = 'bg-indigo-900/80 text-indigo-50'
                    else if (isDiag) bg = 'bg-slate-900/40 text-slate-50'
                    return <td key={j} className={bg} : ''>`>
                      {val}
                    </td>
                  })}
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>

```

```

    { id: 'B', label: 'B', x: 100, y: 120 },
    { id: 'C', label: 'C', x: 300, y: 120 },
    { id: 'D', label: 'D', x: 50, y: 200 },
    { id: 'E', label: 'E', x: 150, y: 200 },
    { id: 'F', label: 'F', x: 250, y: 200 },
    { id: 'G', label: 'G', x: 350, y: 200 },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
    { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
    { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
  ];

  const adj: Record<string, string[]> = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C'],
    'G': ['C'],
  };

  type Action = {
    type: 'visit' | 'process' | 'backtrack';
    node: string;
    desc: string;
    queueOrStack?: string[];
    visitedNodes?: string[];
  };

  function generateDFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const stack: string[] = []; // for visualization, j

    function dfs(v: string) {

```



```

        for (const u of adj[v]) {
          if (!vis.has(u)) {
            vis.add(u);
            q.push(u);
            steps.push({ type: 'visit', node: u, desc: `04
            tack: [...q], visitedNodes: [...Array.from(
            }
          }
        }
      }

      steps.push({ type: 'backtrack', node: '', desc: `04
      om(vis)] });

      return steps;
    }
  }

```

```

export function GraphTraversalViz() {
  const [mode, setMode] = useState<"dfs" | "bfs">("df
  const [autoPlay, setAutoPlay] = useState(false);
  const [stepIdx, setStepIdx] = useState(0);

  const steps = useMemo(() => mode === 'dfs' ? genera

  useEffect(() => {
    setStepIdx(0);
    setAutoPlay(false);
  }, [mode]);

  useEffect(() => {
    if (autoPlay) {
      const t = setInterval(() => {
        setStepIdx(p => {
          if (p >= steps.length - 1) {
            setAutoPlay(false);
            return p;
          }
          return p + 1;
        }
      }, 1000);
    }
  }, [autoPlay, steps]);
}

```

```

        return (
          <line key={i} x1={u.x}
            className={`stroke-
500' : 'stroke-emerald-500'}) : 'stroke-slate
        );
      }}}

  {/* Nodes */}
  {nodes.map(n => {
    const visited = isVisited(n)
    const current = isCurrent(n)

    let fillColor = "fill-slate
    let strokeColor = "stroke-s
    let textColor = "fill-slate
    let radius = 18;

    if (visited) {
      fillColor = mode === 'd
      strokeColor = mode ===
      textColor = "fill-white
    }

    if (current && step.type !==
      strokeColor = "stroke-a
      textColor = "fill-amber
      radius = 22;
    }

    return (
      <g key={n.id} className=
        <circle cx={n.x} cy=
          className={`stro
        <text x={n.x} y={n.
          className={`tex
            {n.label}
          </text>

```

```

        </div>
    </div>

    <div className="flex flex-col sm:flex-row gap-2" style={{width: '100%'}}>
        { /* Controls */ }
        <div className="flex bg-slate-900 border border-slate-800 rounded-md p-2 text-slate-400 hover:text-white hover:bg-slate-800" style={{width: '100%'}}>
            <Undo strokeWidth={3} size={16} />
            </div>
            <div className="flex gap-2" style={{width: '100%'}}>
                <button onClick={() => setAutoPlay(!autoPlay)} style={{width: '50%'}}>
                    {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
                </button>
                <button onClick={() => {setAutoPlay(!autoPlay); setSteps(steps.length - 1)}} className="p-2 text-slate-400 hover:bg-transparent" style={{width: '50%'}}>
                    <SkipForward strokeWidth={3} size={16} />
                </button>
                <button onClick={() => {setAutoPlay(!autoPlay); setSteps(steps.length + 1)}} className="p-2 text-slate-400 hover:bg-transparent" style={{width: '50%'}}>
                    <SkipBackward strokeWidth={3} size={16} />
                </button>
                <button onClick={() => {setAutoPlay(!autoPlay); setSteps(steps.length + 1)}} className="p-2 text-slate-400 hover:bg-transparent" style={{width: '50%'}}>
                    <RefreshCw strokeWidth={3} size={16} />
                </button>
            </div>
        </div>

        { /* Description Log */ }
        <div className="flex-1 bg-slate-900/50 border border-slate-800 rounded-md p-2" style={{width: '100%'}}>
            <div className="text-[10px] text-slate-400" style={{width: '100%'}}>
                {steps.length}
            </div>
            <div className="text-sm text-slate-400" style={{width: '100%'}}>
                {step.desc}
            </div>
        </div>
    </div>
</div>
);

```

```

while (true) {
  let largest = idx;
  const l = 2 * idx + 1;
  const r = 2 * idx + 2;
  if (l < n && a[l].priority > a[largest].priority) largest = l;
  if (r < n && a[r].priority > a[largest].priority) largest = r;
  if (largest !== idx) {
    [a[largest], a[idx]] = [a[idx], a[largest]];
    idx = largest;
  } else break;
}
return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient {
  return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
  const level = Math.floor(Math.log2(index + 1));
  const posInLevel = index - (Math.pow(2, level) - 1);
  const count = Math.pow(2, level);
  const x = ((posInLevel + 0.5) / count) * 100;
  const y = 10 + level * 24;
  return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
  { name: 'Иван (Инфаркт)', symptom: 'Болит сердце', emoji: '👤', priority: 100 },
  { name: 'Маша (Перелом)', symptom: 'Открытый перелом', emoji: '👤', priority: 80 },
  { name: 'Кот Барсик (Укус)', symptom: 'Укусил шмеля', emoji: '🐈', priority: 60 },
  { name: 'Семён (Температура)', symptom: 'Жар 39.5', emoji: '👤', priority: 40 },
  { name: 'Оля (Порез)', symptom: 'Глубокий порез', emoji: '👤', priority: 20 },
  { name: 'Пётр (Кашель)', symptom: 'Простуда', emoji: '👤', priority: 10 },
];

```

```

    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const finalName = name || preset.name.split(' ')[0];
    const finalEmoji = preset.emoji;
    const finalSymptom = preset.symptom;

    const newPatient: Patient = {
      id: `p${uid++}`,
      name: finalName,
      emoji: finalEmoji,
      symptom: finalSymptom,
      priority,
      animState: 'in',
    };

    const nextHeap = heapInsert(heap, newPatient).map(x => x);
    // Mark specifically this new patient as 'in' in the heap
    const target = nextHeap.find(x => x.priority === priority);
    const marked = nextHeap.map(x => x.id === (target?.id));

    setHeap(marked);
    addLog(` Поступил пациент: ${finalName} с тяжестью ${priority}`);

    setTimeout(() => {
      setHeap(nextHeap);
      setBusy(false);
    }, 500);
  }, [busy, heap]);

  const handleRandomInsert = () => {
    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
    const randomShift = Math.floor(Math.random() * 10);
    const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
    insertPatient(preset.name, prio);
  };

  const handleCustomInsert = (e: React.FormEvent) => {
    e.preventDefault();

```

```

        setHealing(prev => prev ? { ...prev, animStat
        addLog(` Успех! Пациент ${topPatient.name} п
        setTimeout(() => {
            setHealing(null);
            setBusy(false);
        }, 600);
    }, 800);

    }, 350);
    }, 650);
}, [busy, heap]));

const reset = () => {
    setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
    setLog(` База данных скорой помощи перезагружена.
    setHealing(null);
    setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
    const res: React.ReactElement[] = [];
    const p = nodePos(idx);
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < heap.length) {
        const lp = nodePos(l);
        res.push(
            <line key={`el${idx}`}
                x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
                x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }
                stroke="#475569" strokeWidth="2"
            />
        );
    }
    if (r < heap.length) {
        const rp = nodePos(r);
        res.push(

```

```
    <span> Привезти по Скорой (INSERT)</span>
  </button>

  {/* Добавить кастомного */}
  <form onSubmit={handleCustomInsert} className="flex flex-col items-center min-h-100px gap-4" >
    <input
      type="text"
      required
      value={customName}
      onChange={e => setCustomName(e.target.value)}
      placeholder="Имя пациента"
      className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
    />
    <div className="flex flex-col items-center min-h-100px gap-4" >
      <span className="text-[9px] font-mono text-slate-400" > Приоритет
      <input
        type="range"
        min="10"
        max="99"
        value={customPriority}
        onChange={e => setCustomPriority(Number(e.target.value))}
        className="w-20 accent-emerald-500 cursor-pointer"
      />
    </div>
    <button
      type="submit"
      disabled={busy || !customName.trim() || !customPriority}
      className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 px-4 rounded-xl shadow-lg active:scale-95 transition-colors"
    >
      Везти
    </button>
  </form>

  {/* Забрать самого тяжелого */}
  <button
    onClick={extractMax}
    className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 px-4 rounded-xl shadow-lg active:scale-95 transition-colors"
  >
    Забрать
  </button>
</div>
</div>
</div>
```

```

        className={`
          absolute flex flex-col items-center
          -translate-x-1/2 -translate-y-1/2 t
          ${patient.animState === 'in' ? 'ani
          ${patient.animState === 'winner' ?
          ${patient.animState === 'out' ? 'an
          ${isRoot ? 'bg-rose-950/80 border-r
        `}
      style={{
        left: `${pos.x}%`,
        top: `${pos.y}%`,
        width: isRoot ? 60 : 52,
        height: isRoot ? 60 : 52,
      }}
    >
      <span className="text-2xl leading-non
      <span className="text-[10px] font-mon
        {patient.priority}%
      </span>

      {/* Tooltip */}
      <div className="absolute bottom-full
        <div className="bg-slate-950 border
0 shadow-xl">
        <div className="font-bold text-wh
        <div className="text-emerald-400"
        </div>
      </div>
    </div>
  );
  }}}
</div>

  <div className="w-full h-3 bg-gradient-to-r f
</div>

  {/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
  <div className="lg:col-span-4 bg-slate-900 roun

```



```
    {/* ЛОГ ДЕЙСТВИЙ */}
    <div className="bg-slate-900 border border-slate-800" >
      <div className="text-[10px] font-mono text-slate-100" >
        {log.map((entry, idx) => (
          <div
            key={idx}
            className={`text-xs font-mono flex items-center justify-between gap-2`} >
            {idx === 0 ? <span className="text-emerald-400 font-weight-bold">LOG</span> : null}
            <span>{entry}</span>
          </div>
        ))}
      </div>
    </div>
  );
};
```

ФАЙЛ 46/87: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```

    </svg>
    {caption && (
      <div className="text-[10px] text-slate-400 text-center">
        {caption}
      </div>
    )}
    <div className="text-[9px] text-slate-600 text-center">
      {paused ? "пауза – курсор на карточке" : "наведи"}
    </div>
  </div>
);
};

```

```

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
      {label !== undefined && (
        <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
          {label}
        </text>
      )}
    </g>
  );

```

```

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (

```

```

    <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
    <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
    <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
  </Svg>
);
};

```

```

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][ ] = [
    [ "A", "B"], [ "B", "C"],
    [ "D", "E"],
    [ "F", "G"], [ "G", "H"], [ "F", "H"],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return {

```

```

        y={({a[1] + b[1]) / 2 - 4}
        textAnchor="middle"
        fontSize={10}
        fontWeight="700"
        fill={C.warn}
      >
        {w}
      </text>
    )}
  </g>
);
}}}
{Object.entries(pos).map(([k, [x, y]]) => {
  <Node key={k} x={x} y={y} label={k} fill={C.idl}
  >>
})}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => {

```

```

ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
  const step = useLoop(frames.length, ms);
  const f = frames[step];
  const color = (id: string) =>
    id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id
  return (
    <Svg caption={captions[step]}>
      {f.edges.map(([a, b], i) => (
        <line
          key={i}
          x1={f.pos[a][0]}
          y1={f.pos[a][1]}
          x2={f.pos[b][0]}
          y2={f.pos[b][1]}
          stroke={C.edge}
          strokeWidth={1.8}
          style={{ transition: MOVE }}
        />
      ))}
      {Object.entries(f.pos).map(([id, [x, y]]) => (
        <g key={id} style={{ transform: `translate(${x}, ${y})` }>
          <circle r={13} fill={color(id)} stroke={C.idle} />
          <text textAnchor="middle" dominantBaseline="bottom">
            {id}
          </text>
        </g>
      ))}
    </Svg>
  );
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: "]}
    frames={

```

```

const GRAPH_POS: Record<string, [number, number]> = {
  A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step,
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}}
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.h
        stroke={wave.includes(k) ? "#a5b4fc" : undefi
    }}}
  </Svg>
  );
};

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(i

```

```

    }}}
    <text x={224} y={62} textAnchor="middle" fontSize=
    <text x={224} y={74} textAnchor="middle" fontSize=
  </Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь  

      <Edge a={[60, 70]} b={[200, 70]} color={improved  

      <text x={130} y={54} textAnchor="middle" fontSize=
      <Node x={60} y={70} r={20} fill={C.active} label=
      <Node x={200} y={70} r={20} fill={improved ? C.do  

      <text x={60} y={106} textAnchor="middle" fontSize=
      <text x={200} y={106} textAnchor="middle" fontSize=
      <text x={130} y={22} textAnchor="middle" fontSize=
    </Svg>
  );
};

```

```

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на  

      {edges.map(([u, v], i) => {
        const isBridge = u === "C" && v === "D";
        if (isBridge && cut) return null;

```

```

    ];
    const chosen = ["A-D", "A-B", "B-C", "D-E"];
    const step = useLoop(chosen.length + 1, 800);
    const taken = new Set(chosen.slice(0, step));
    return (
      <Svg caption={`Жадно берём самые лёгкие рёбра без ц
        {edges.map((e, i) => {
          const on = taken.has(`${e.u}-${e.v}`);
          return (
            <g key={i}>
              <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
              <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
                textAnchor="middle" fontSize={9} fontWeigh
            </g>
          );
        })}
      {Object.entries(pos).map(([k, [x, y]]) => (<Node
    </Svg>
  );
};

```

```

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
  };
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step :
      })}
    </Svg>
  );
};

```



```

    <g key={v} style={{ transition: "all .3s" }}>
      <rect x={94} y={100 - i * 26} width={72} height={26}
        fill={i === items.length - 1 ? (popping ? C.done : C.dim) : C.dim} />
      <text x={130} y={111 - i * 26} textAnchor="middle" font-size={16}>{v}</text>
    </g>
  )}}
  <text x={130} y={16} textAnchor="middle" font-size={16}>{C.done ? "Выход" : "Вход"}</text>
</Svg>
);
};

export const QueueDemo: React.FC = () => {
  const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
  const step = useLoop(states.length, 800);
  const items = states[step];
  return (
    <Svg caption="pop_front слева, push_back справа – к концу" width={240} height={100}>
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6" viewBox="0 0 6 6">
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} font-size={9} fill={C.dim}>Выход</text>
      <text x={206} y={44} font-size={9} fill={C.dim}>Вход</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={26}
            fill={i === items.length - 1 ? (popping ? C.done : C.dim) : C.dim} />
          <text x={53 + i * 54} y={71} textAnchor="middle" font-size={16}>{v}</text>
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};

export const DsuDemo: React.FC = () => {
  const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
  const step = useLoop(parents.length, 800);
  const items = parents[step];
  return (
    <Svg caption="Дисъюнктные множества" width={240} height={100}>
      <defs>
        <marker id="mdArrow" markerWidth="6" markerHeight="6" viewBox="0 0 6 6">
          <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
        </marker>
      </defs>
      <text x={12} y={44} font-size={9} fill={C.dim}>Выход</text>
      <text x={206} y={44} font-size={9} fill={C.dim}>Вход</text>
      {items.map((v, i) => (
        <g key={v} style={{ transition: "all .3s" }}>
          <rect x={30 + i * 54} y={56} width={46} height={26}
            fill={i === items.length - 1 ? (popping ? C.done : C.dim) : C.dim} />
          <text x={53 + i * 54} y={71} textAnchor="middle" font-size={16}>{v}</text>
        </g>
      ))}
      <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
    </Svg>
  );
};

```

```
};
```

```
export const TrieBfsDemo: React.FC = () => {
  const levels = [[0], [1, 2], [3, 4, 5]];
  const step = useLoop(levels.length + 1, 950);
  const doneIds = new Set(levels.slice(0, step).flat());
  const wave = step > 0 && step <= levels.length ? levels[step - 1] : null;
  return (
    <Svg caption={`BFS по бору: уровень ${Math.max(0, Math.min(step - 1, levels.length - 1))}`}
      {TRIE_NODES.filter((n) => n.parent !== null).map((n) => {
        const p = TRIE_NODES[n.parent as number];
        return <Edge key={n.id} a={[p.x, p.y]} b={[n.x, n.y]} fill={wave?.includes(n.id) ? C.active : C.dim} />
      })}
      {TRIE_NODES.map((n) => {
        <Node key={n.id} x={n.x} y={n.y} label={n.ch} fill={wave?.includes(n.id) ? C.active : C.dim} />
      })}
      <text x={6} y={12} fontSize={9} fill={C.dim}>очередной уровень
    </Svg>
  );
};
```

```
export const SuffixLinkDemo: React.FC = () => {
  const links: [number, number][] = [[1, 0], [2, 0], [3, 0], [4, 1], [5, 1], [6, 2], [7, 2], [8, 3], [9, 3], [10, 4], [11, 4], [12, 5], [13, 5], [14, 6], [15, 6], [16, 7], [17, 7], [18, 8], [19, 8], [20, 9], [21, 9], [22, 10], [23, 10], [24, 11], [25, 11], [26, 12], [27, 12], [28, 13], [29, 13], [30, 14], [31, 14], [32, 15], [33, 15], [34, 16], [35, 16], [36, 17], [37, 17], [38, 18], [39, 18], [40, 19], [41, 19], [42, 20], [43, 20], [44, 21], [45, 21], [46, 22], [47, 22], [48, 23], [49, 23], [50, 24], [51, 24], [52, 25], [53, 25], [54, 26], [55, 26], [56, 27], [57, 27], [58, 28], [59, 28], [60, 29], [61, 29], [62, 30], [63, 30], [64, 31], [65, 31], [66, 32], [67, 32], [68, 33], [69, 33], [70, 34], [71, 34], [72, 35], [73, 35], [74, 36], [75, 36], [76, 37], [77, 37], [78, 38], [79, 38], [80, 39], [81, 39], [82, 40], [83, 40], [84, 41], [85, 41], [86, 42], [87, 42], [88, 43], [89, 43], [90, 44], [91, 44], [92, 45], [93, 45], [94, 46], [95, 46], [96, 47], [97, 47], [98, 48], [99, 48], [100, 49], [101, 49], [102, 50], [103, 50], [104, 51], [105, 51], [106, 52], [107, 52], [108, 53], [109, 53], [110, 54], [111, 54], [112, 55], [113, 55], [114, 56], [115, 56], [116, 57], [117, 57], [118, 58], [119, 58], [120, 59], [121, 59], [122, 60], [123, 60], [124, 61], [125, 61], [126, 62], [127, 62], [128, 63], [129, 63], [130, 64], [131, 64], [132, 65], [133, 65], [134, 66], [135, 66], [136, 67], [137, 67], [138, 68], [139, 68], [140, 69], [141, 69], [142, 70], [143, 70], [144, 71], [145, 71], [146, 72], [147, 72], [148, 73], [149, 73], [150, 74], [151, 74], [152, 75], [153, 75], [154, 76], [155, 76], [156, 77], [157, 77], [158, 78], [159, 78], [160, 79], [161, 79], [162, 80], [163, 80], [164, 81], [165, 81], [166, 82], [167, 82], [168, 83], [169, 83], [170, 84], [171, 84], [172, 85], [173, 85], [174, 86], [175, 86], [176, 87], [177, 87], [178, 88], [179, 88], [180, 89], [181, 89], [182, 90], [183, 90], [184, 91], [185, 91], [186, 92], [187, 92], [188, 93], [189, 93], [190, 94], [191, 94], [192, 95], [193, 95], [194, 96], [195, 96], [196, 97], [197, 97], [198, 98], [199, 98], [200, 99], [201, 99], [202, 100], [203, 100], [204, 101], [205, 101], [206, 102], [207, 102], [208, 103], [209, 103], [210, 104], [211, 104], [212, 105], [213, 105], [214, 106], [215, 106], [216, 107], [217, 107], [218, 108], [219, 108], [220, 109], [221, 109], [222, 110], [223, 110], [224, 111], [225, 111], [226, 112], [227, 112], [228, 113], [229, 113], [230, 114], [231, 114], [232, 115], [233, 115], [234, 116], [235, 116], [236, 117], [237, 117], [238, 118], [239, 118], [240, 119], [241, 119], [242, 120], [243, 120], [244, 121], [245, 121], [246, 122], [247, 122], [248, 123], [249, 123], [250, 124], [251, 124], [252, 125], [253, 125], [254, 126], [255, 126], [256, 127], [257, 127], [258, 128], [259, 128], [260, 129], [261, 129], [262, 130], [263, 130], [264, 131], [265, 131], [266, 132], [267, 132], [268, 133], [269, 133], [270, 134], [271, 134], [272, 135], [273, 135], [274, 136], [275, 136], [276, 137], [277, 137], [278, 138], [279, 138], [280, 139], [281, 139], [282, 140], [283, 140], [284, 141], [285, 141], [286, 142], [287, 142], [288, 143], [289, 143], [290, 144], [291, 144], [292, 145], [293, 145], [294, 146], [295, 146], [296, 147], [297, 147], [298, 148], [299, 148], [300, 149], [301, 149], [302, 150], [303, 150], [304, 151], [305, 151], [306, 152], [307, 152], [308, 153], [309, 153], [310, 154], [311, 154], [312, 155], [313, 155], [314, 156], [315, 156], [316, 157], [317, 157], [318, 158], [319, 158], [320, 159], [321, 159], [322, 160], [323, 160], [324, 161], [325, 161], [326, 162], [327, 162], [328, 163], [329, 163], [330, 164], [331, 164], [332, 165], [333, 165], [334, 166], [335, 166], [336, 167], [337, 167], [338, 168], [339, 168], [340, 169], [341, 169], [342, 170], [343, 170], [344, 171], [345, 171], [346, 172], [347, 172], [348, 173], [349, 173], [350, 174], [351, 174], [352, 175], [353, 175], [354, 176], [355, 176], [356, 177], [357, 177], [358, 178], [359, 178], [360, 179], [361, 179], [362, 180], [363, 180], [364, 181], [365, 181], [366, 182], [367, 182], [368, 183], [369, 183], [370, 184], [371, 184], [372, 185], [373, 185], [374, 186], [375, 186], [376, 187], [377, 187], [378, 188], [379, 188], [380, 189], [381, 189], [382, 190], [383, 190], [384, 191], [385, 191], [386, 192], [387, 192], [388, 193], [389, 193], [390, 194], [391, 194], [392, 195], [393, 195], [394, 196], [395, 196], [396, 197], [397, 197], [398, 198], [399, 198], [400, 199], [401, 199], [402, 200], [403, 200], [404, 201], [405, 201], [406, 202], [407, 202], [408, 203], [409, 203], [410, 204], [411, 204], [412, 205], [413, 205], [414, 206], [415, 206], [416, 207], [417, 207], [418, 208], [419, 208], [420, 209], [421, 209], [422, 210], [423, 210], [424, 211], [425, 211], [426, 212], [427, 212], [428, 213], [429, 213], [430, 214], [431, 214], [432, 215], [433, 215], [434, 216], [435, 216], [436, 217], [437, 217], [438, 218], [439, 218], [440, 219], [441, 219], [442, 220], [443, 220], [444, 221], [445, 221], [446, 222], [447, 222], [448, 223], [449, 223], [450, 224], [451, 224], [452, 225], [453, 225], [454, 226], [455, 226], [456, 227], [457, 227], [458, 228], [459, 228], [460, 229], [461, 229], [462, 230], [463, 230], [464, 231], [465, 231], [466, 232], [467, 232], [468, 233], [469, 233], [470, 234], [471, 234], [472, 235], [473, 235], [474, 236], [475, 236], [476, 237], [477, 237], [478, 238], [479, 238], [480, 239], [481, 239], [482, 240], [483, 240], [484, 241], [485, 241], [486, 242], [487, 242], [488, 243], [489, 243], [490, 244], [491, 244], [492, 245], [493, 245], [494, 246], [495, 246], [496, 247], [497, 247], [498, 248], [499, 248], [500, 249], [501, 249], [502, 250], [503, 250], [504, 251], [505, 251], [506, 252], [507, 252], [508, 253], [509, 253], [510, 254], [511, 254], [512, 255], [513, 255], [514, 256], [515, 256], [516, 257], [517, 257], [518, 258], [519, 258], [520, 259], [521, 259], [522, 260], [523, 260], [524, 261], [525, 261], [526, 262], [527, 262], [528, 263], [529, 263], [530, 264], [531, 264], [532, 265], [533, 265], [534, 266], [535, 266], [536, 267], [537, 267], [538, 268], [539, 268], [540, 269], [541, 269], [542, 270], [543, 270], [544, 271], [545, 271], [546, 272], [547, 272], [548, 273], [549, 273], [550, 274], [551, 274], [552, 275], [553, 275], [554, 276], [555, 276], [556, 277], [557, 277], [558, 278], [559, 278], [560, 279], [561, 279], [562, 280], [563, 280], [564, 281], [565, 281], [566, 282], [567, 282], [568, 283], [569, 283], [570, 284], [571, 284], [572, 285], [573, 285], [574, 286], [575, 286], [576, 287], [577, 287], [578, 288], [579, 288], [580, 289], [581, 289], [582, 290], [583, 290], [584, 291], [585, 291], [586, 292], [587, 292], [588, 293], [589, 293], [590, 294], [591, 294], [592, 295], [593, 295], [594, 296], [595, 296], [596, 297], [597, 297], [598, 298], [599, 298], [600, 299], [601, 299], [602, 300], [603, 300], [604, 301], [605, 301], [606, 302], [607, 302], [608, 303], [609, 303], [610, 304], [611, 304], [612, 305], [613, 305], [614, 306], [615, 306], [616, 307], [617, 307], [618, 308], [619, 308], [620, 309], [621, 309], [622, 310], [623, 310], [624, 311], [625, 311], [626, 312], [627, 312], [628, 313], [629, 313], [630, 314], [631, 314], [632, 315], [633, 315], [634, 316], [635, 316], [636, 317], [637, 317], [638, 318], [639, 318], [640, 319], [641, 319], [642, 320], [643, 320], [644, 321], [645, 321], [646, 322], [647, 322], [648, 323], [649, 323], [650, 324], [651, 324], [652, 325], [653, 325], [654, 326], [655, 326], [656, 327], [657, 327], [658, 328], [659, 328], [660, 329], [661, 329], [662, 330], [663, 330], [664, 331], [665, 331], [666, 332], [667, 332], [668, 333], [669, 333], [670, 334], [671, 334], [672, 335], [673, 335], [674, 336], [675, 336], [676, 337], [677, 337], [678, 338], [679, 338], [680, 339], [681, 339], [682, 340], [683, 340], [684, 341], [685, 341], [686, 342], [687, 342], [688, 343], [689, 343], [690, 344], [691, 344], [692, 345], [693, 345], [694, 346], [695, 346], [696, 347], [697, 347], [698, 348], [699, 348], [700, 349], [701, 349], [702, 350], [703, 350], [704, 351], [705, 351], [706, 352], [707, 352], [708, 353], [709, 353], [710, 354], [711, 354], [712, 355], [713, 355], [714, 356], [715, 356], [716, 357], [717, 357], [718, 358], [719, 358], [720, 359], [721, 359], [722, 360], [723, 360], [724, 361], [725, 361], [726, 362], [727, 362], [728, 363], [729, 363], [730, 364], [731, 364], [732, 365], [733, 365], [734, 366], [735, 366], [736, 367], [737, 367], [738, 368], [739, 368], [740, 369], [741, 369], [742, 370], [743, 370], [744, 371], [745, 371], [746, 372], [747, 372], [748, 373], [749, 373], [750, 374], [751, 374], [752, 375], [753, 375], [754, 376], [755, 376], [756, 377], [757, 377], [758, 378], [759, 378], [760, 379], [761, 379], [762, 380], [763, 380], [764, 381], [765, 381], [766, 382], [767, 382], [768, 383], [769, 383], [770, 384], [771, 384], [772, 385], [773, 385], [774, 386], [775, 386], [776, 387], [777, 387], [778, 388], [779, 388], [780, 389], [781, 389], [782, 390], [783, 390], [784, 391], [785, 391], [786, 392], [787, 392], [788, 393], [789, 393], [790, 394], [791, 394], [792, 395], [793, 395], [794, 396], [795, 396], [796, 397], [797, 397], [798, 398], [799, 398], [800, 399], [801, 399], [802, 400], [803, 400], [804, 401], [805, 401], [806, 402], [807, 402], [808, 403], [809, 403], [810, 404], [811, 404], [812, 405], [813, 405], [814, 406], [815, 406], [816, 407], [817, 407], [818, 408], [819, 408], [820, 409], [821, 409], [822, 410], [823, 410], [824, 411], [825, 411], [826, 412], [827, 412], [828, 413], [829, 413], [830, 414], [831, 414], [832, 415], [833, 415], [834, 416], [835, 416], [836, 417], [837, 417], [838, 418], [839, 418], [840, 419], [841, 419], [842, 420], [843, 420], [844, 421], [845, 421], [846, 422], [847, 422], [848, 423], [849, 423], [850, 424], [851, 424], [852, 425], [853, 425], [854, 426], [855, 426], [856, 427], [857, 427], [858, 428], [859, 428], [860, 429], [861, 429], [862, 430], [863, 430], [864, 431], [865, 431], [866, 432], [867, 432], [868, 433], [869, 433], [870, 434], [871, 434], [872, 435], [873, 435], [874, 436], [875, 436], [876, 437], [877, 437], [878, 438], [879, 438], [880, 439], [881, 439], [882, 440], [883, 440], [884, 441], [885, 441], [886, 442], [887, 442], [888, 443], [889, 443], [890, 444], [891, 444], [892, 445], [893, 445], [894, 446], [895, 446], [896, 447], [897, 447], [898, 448], [899, 448], [900, 449], [901, 449], [902, 450], [903, 450], [904, 451], [905, 451], [906, 452], [907, 452], [908, 453], [909, 453], [910, 454], [911, 454], [912, 455], [913, 455], [914, 456], [915, 456], [916, 457], [917, 457], [918, 458], [919, 458], [920, 459], [921, 459], [922, 460], [923, 460], [924, 461], [925, 461], [926, 462], [927, 462], [928, 463], [929, 463], [930, 464], [931, 464], [932, 465], [933, 465], [934, 466], [935, 466], [936, 467], [937, 467], [938, 468], [939, 468], [940, 469], [941, 469], [942, 470], [943, 470], [944, 471], [945, 471], [946, 472], [947, 472], [948, 473], [949, 473], [950, 474], [951, 474], [952, 475], [953, 475], [954, 476], [955, 476], [956, 477], [957, 477], [958, 478], [959, 478], [960, 479], [961, 479], [962, 480], [963, 480], [964, 481], [965, 481], [966, 482], [967, 482], [968, 483], [969, 483], [970, 484], [971, 484], [972, 485], [973, 485], [974, 486], [975, 486], [976, 487], [977, 487], [978, 488], [979, 488], [980, 489], [981, 489], [982, 490], [983, 490], [984, 491], [985, 491], [986, 492], [987, 492], [988, 493], [989, 493], [990, 494], [991, 494], [992, 495], [993, 495], [994, 496], [995, 496], [996, 497], [997, 497], [998, 498], [999, 498], [1000, 499], [1001, 499], [1002, 500], [1003, 500], [1004, 501], [1005, 501], [1006, 502], [1007, 502], [1008, 503], [1009, 503], [1010, 504], [1011, 504], [1012, 505], [1013, 505], [1014, 506], [1015, 506], [1016, 507], [1017, 507], [1018, 508], [1019, 508], [1020, 509], [1021, 509], [1022, 510], [1023, 510], [1024, 511], [1025, 511], [1026, 512], [1027, 512], [1028, 513], [1029, 513], [1030, 514], [1031, 514], [1032, 515], [1033, 515], [1034, 516], [1035, 516], [1036, 517], [1037, 517], [1038, 518], [1039, 518], [1040, 519], [1041, 519], [1042, 520], [1043, 520], [1044, 521], [1045, 521], [1046, 522], [1047, 522], [1048, 523], [1049, 523], [1050, 524], [1051, 524], [1052, 525], [1053, 525], [1054, 526], [1055, 526], [1056, 527], [1057, 527], [1058, 528], [1059, 528], [1060, 529], [1061, 529], [1062, 530], [1063, 530], [1064, 531], [1065, 531], [1066, 532], [1067, 532], [1068, 533], [1069, 533], [1070, 534], [1071, 534], [1072, 535], [1073, 535], [1074, 536], [1075, 536], [1076, 537], [1077, 537], [1078, 538], [1079, 538], [1080, 539], [1081, 539], [1082, 540], [1083, 540], [1084, 541], [1085, 541], [1086, 542], [1087, 542], [1088, 543], [1089, 543], [1090, 544], [1091, 544], [1092, 545], [1093, 545], [1094, 546], [1095, 546], [1096, 547], [1097, 547], [1098, 548], [1099, 548], [1100, 549], [1101, 549], [1102, 550], [1103, 550], [1104, 551], [1105, 551], [1106, 552], [1107, 552], [1108, 553], [1109, 553], [1110, 554], [1111, 554], [1112, 555], [1113, 555], [1114, 556], [1115, 556], [1116, 557], [1117, 557], [1118, 558], [1119, 558], [1120, 559], [1121, 559], [1122, 560], [1123, 560], [1124, 561], [1125, 561], [1126, 562], [1127, 562], [1128, 563], [1129, 563], [1130, 564], [1131, 564], [1132, 565], [1133, 565], [1134, 566], [1135, 566], [1136, 567], [1137, 567], [1138, 568], [1139, 568], [1140, 569], [1141, 569], [1142, 570], [1143, 570], [1144, 571], [1145, 571], [1146, 572], [1147, 572], [1148, 573], [1149, 573], [1150, 574], [1151, 574], [1152, 575], [1153, 575], [1154, 576], [1155, 576], [1156, 577], [1157, 577], [1158, 578], [1159, 578], [1160, 579], [1161, 579], [1162, 580], [1163, 580], [1164, 581], [1165, 581], [1166, 582], [1167, 582], [1168, 583], [1169, 583], [1170, 584], [1171, 584], [1172, 585], [1173, 585], [1174, 586], [1175, 586], [1176, 587], [1177, 587], [1178, 588], [1179, 588], [1180, 589], [1181, 589], [1182, 590], [1183, 590], [1184, 591], [1185, 591], [1186, 592], [1187, 592], [1188, 593], [1189, 593], [1190, 594], [1191, 594], [1192, 595], [1193, 595], [1194, 596], [1195, 596], [1196, 597], [1197, 597], [1198, 598], [1199, 598], [1200, 
```

```

return (
  <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -`
    о префиксные суммы P">
    <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
    {a.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 39} y={24} width={34} height={12}
          fill={show && i >= l && i <= r ? C.active : C.dim}>
        <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
      </g>
    ))}
    <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
    {pref.map((v, i) => (
      <g key={i}>
        <rect x={20 + i * 33} y={78} width={29} height={12}
          fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
          stroke={C.idleStroke} style={{ transition: "stroke 0.2s" }}>
        <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
      </g>
    ))}
  </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для`
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.idle}>
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
    </Svg>
  );
};

```

```

    });
};

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} height={44}
            fill={step > 0 && (idx < len || (idx > i - len)) ? C.fill : C.dim}
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">π</text>
          <text x={32 + idx * 33} y={100} textAnchor="middle">{idx}</text>
        </g>
      ))}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" font-size={12}>
      π[i] — длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
  );
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово ${step}`
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {

```

```
export type DemoKind =  
  | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"  
  | "suffix-link" | "toposort" | "relax" | "prefix-sum"  
  | "segment-tree" | "bridge" | "articulation" | "mst"  
  | "prefix-function" | "dp" | "floyd" | "components" |  
  | "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {  
  bfs: BfsDemo,  
  dfs: DfsDemo,  
  heap: HeapDemo,  
  stack: StackDemo,  
  queue: QueueDemo,  
  dsu: DsuDemo,  
  trie: TrieDemo,  
  "trie-bfs": TrieBfsDemo,  
  "suffix-link": SuffixLinkDemo,  
  toposort: ToposortDemo,  
  relax: RelaxDemo,  
  "prefix-sum": PrefixSumDemo,  
  "sparse-table": SparseTableDemo,  
  "segment-tree": SegmentTreeDemo,  
  bridge: BridgeDemo,  
  articulation: ArticulationDemo,  
  mst: MstDemo,  
  amortized: AmortizedDemo,  
  np: NpDemo,  
  scc: SccDemo,  
  "prefix-function": PrefixFunctionDemo,  
  dp: DpDemo,  
  floyd: FloydDemo,  
  components: ComponentsDemo,  
  graph: GraphBasicsDemo,  
  "coord-compress": CoordCompressDemo,  
  zig: ZigDemo,  
  "zig-zig": ZigZigDemo,  
  "zig-zag": ZigZagDemo,
```

```

    anchor: HintAnchor | null;
    onClose: () => void;
    onOpenSimulator: (vizId: string) => void;
    onCardEnter: () => void;
    onCardLeave: () => void;
  }

const CARD_W = 340;
const GAP = 10;

export const TermPopover: React.FC<Props> = ({ anchor,
const cardRef = useRef<HTMLDivElement>(null);
const [pos, setPos] = useState<{ top: number; left: number}>({
  top: 0, left: 0 });

const recompute = React.useCallback(() => {
  if (!anchor || !cardRef.current) return;
  const vw = window.innerWidth;
  const vh = window.innerHeight;
  const width = Math.min(CARD_W, vw - 16);
  const height = cardRef.current.offsetHeight || 280;

  let left = anchor.rect.left + anchor.rect.width / 2;
  left = Math.max(8, Math.min(left, vw - width - 8));

  const spaceBelow = vh - anchor.rect.bottom;
  const spaceAbove = anchor.rect.top;
  const below = spaceBelow > spaceAbove ? true : false;

  let top: number;
  if (below) {
    top = anchor.rect.bottom + GAP;
    if (top + height > vh - 8) top = Math.max(8, vh - height - 8);
  } else {
    top = anchor.rect.top - height - GAP;
    if (top < 8) top = anchor.rect.bottom + GAP;
  }

  top = Math.max(8, Math.min(top, vh - height - 8));

```

```

const viz = getViz(entry.simulator);

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
    aria-label={entry.title}
  >
    <div className="bg-slate-900 border border-indigo-500" style={styles.card} >
      <div className="flex items-start gap-2 px-3 pt-3" >
        <div className="bg-indigo-600 p-1 rounded-md" style={styles.bulb} >
          <Lightbulb className="w-3.5 h-3.5 text-white" />
        </div>
        <h4 className="text-sm font-bold text-indigo-100" >{entry.title}</h4>
        <button
          onClick={onClose}
          className="text-slate-500 hover:text-white"
          aria-label="Заккрыть подсказку"
        >
          <X className="w-4 h-4" />
        </button>
      </div>
      <div className="p-3 space-y-2.5 max-h-[60vh] overflow-y-auto" >
        <p className="text-[13px] leading-relaxed text-slate-300" >{entry.demo} && <MiniDemo kind={entry.demo} />
        <p className="text-slate-300" >{entry.formal} && (

```

ФАЙЛ 52/87: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
```

```
export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;
```

```
  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];
```

```
  const edges = [
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0 },
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0 },
    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0 },
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0 },
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0 },
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0 },
  ];
```

```
  const isNodeVisible = (id: string) => {
    if (id === 'S' && (step === 0 || step === 7)) return true;
    return false;
  };
```

```
  const getPotential = (id: string) => {
    if (step >= 2 && step < 7) {
      if (id === 'S') return 0;
      if (id === 'A') return 0;
    }
    return 0;
  };
```



```

        if (e.id === 'AB' && step === 4) return "url(#a";
        if (e.id === 'AB' && step > 4) return "url(#arr";
        if (e.id === 'BC' && step === 5) return "url(#a";
        if (e.id === 'BC' && step > 5) return "url(#arr";
        if (e.id === 'CA' && step === 6) return "url(#a";
        if (e.id === 'CA' && step > 6) return "url(#arr";

        if (step >= 4) return "url(#arrow-slate-dim)";
        return "url(#arrow-slate)";
    };

    const getEdgeContent = (e: any) => {
        if (e.u === 'S') return e.weight;

        if (step < 4) return e.weight;

        if (e.id === 'AB') {
            if (step === 4) return `${e.weight} + (${0}`;
            return e.newWeight;
        }
        if (e.id === 'BC') {
            if (step < 5) return e.weight;
            if (step === 5) return `${e.weight} + (${-2}`;
            return e.newWeight;
        }
        if (e.id === 'CA') {
            if (step < 6) return e.weight;
            if (step === 6) return `${e.weight} + (${-1}`;
            return e.newWeight;
        }
        return e.weight;
    };
};

```

```

const isEdgeTextHighlighted = (e: any) => {
    if (e.id === 'AB' && step === 4) return true;
    if (e.id === 'BC' && step === 5) return true;
    if (e.id === 'CA' && step === 6) return true;
    return false;
};

```

```

        Это повышает или понижает вес ребра
    </div>
    );
    case 4: return (
        <div>
            <b>Шаг 3.1: Пересчет ребра A → B.</b>
            Старый вес: <span className="text-emerald-300">0</span>
            Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-emerald-300">-2</span>
        </div>
    );
    case 5: return (
        <div>
            <b>Шаг 3.2: Пересчет ребра B → C.</b>
            Старый вес: 1. <br/>
            Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-emerald-300">-2</span>
        </div>
    );
    case 6: return (
        <div>
            <b>Шаг 3.3: Пересчет ребра C → A.</b>
            Старый вес: 2. <br/>
            Вычисляем:  $2 + (-1) - 0 =$  <span className="text-emerald-300">1</span>
        </div>
    );
    case 7: return (
        <div className="text-emerald-300">
            <b>Готово!</b> Фиктивную вершину S мы добавили в граф.
            Все новые веса ребер в графе стали кратчайшими!
            При этом старые кратчайшие пути остались кратчайшими!
        </div>
    );
    default: return "";
}

};

return (
    <div className="bg-slate-800 p-4 rounded-xl border">

```

```

        if (isS && (step === 0 || s

const midX = (u.x + v.x) /
const midY = (u.y + v.y) /

const edgeColorClass = getE
const textHigh = isEdgeText
const textEmer = isEdgeText

return (
  <g key={i}>
    <line
      x1={u.x} y1={u.y}
      className={`str
      markerEnd={getM
    />

    <text
      x={midX} y={midY}
      textAnchor="mid
      dominantBaseline
      className={`tex
        ${textHigh
        textEmer
        isS ? 'fi
    `}>
      {getEdgeContent
    </text>
  </g>
)
}})

{/* Nodes */}
{nodes.map(n => {
  if (!isNodeVisible(n.id)) r
  const pot = getPotential(n.

  return (

```

```

        <div className="flex bg-slate-900 b
          <button onClick={() => setStep(1
            className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
            <RotateCcw strokeWidth={3}
          </button>
          <button onClick={() => setStep(1
            className="p-3 bg-slate-800
-slate-400 transition-colors" title="Шаг на
            <Undo strokeWidth={3} size=
          </button>
          <button onClick={() => setStep(
            className={`flex-1 font-bol
              ${step === MAX_STEP
                ? 'bg-emerald-600/5
                : 'bg-indigo-600 ho
              {step === MAX_STEP ? "Готов
              {step !== MAX_STEP && <Skip
            </button>
          </div>
        </div>

        {/* Progress Dots */}
        <div className="flex justify-center
          {Array.from({length: MAX_STEP +
            <div key={i} className={`w-1
125' : i < step ? 'bg-indigo-900' : 'bg-sla
            )}}
          </div>
        </div>
      </div>
    </div>
  </div>
);
}

```

```
interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini";
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    // Generate step-by-step Kosaraju steps
    const generatedSteps: AlgoStep[] = [];
    const visited = new Set<number>();
    const order: number[] = [];
    const sccMap: Record<number, number> = {};

    const recordStep = (
      phase: AlgoStep["phase"],
      desc: string,
      currentNode: number | null,
      transposed: boolean,
      currentSccId: number,
    ) => {
      generatedSteps.push({
        phase,
        desc,
        visited: new Set(visited),
        currentNode,
        order: [...order],
      });
    };
  });
}
```

```

    order.push(u);
    recordStep(
      "dfs1",
      `DFS 1: вершина ${u} полностью обработана. Запи-
      у,
      false,
      -1,
    );
  };
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    dfs1(i);
  }
}

const topoOrder = [...order].reverse();
recordStep(
  "transpose",
  `Первый проход завершен! Стек выхода в топологиче-
  ебра.`,
  null,
  true,
  -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u)));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
  visited.add(u);
  sccMap[u] = currentScc;

```

```

    recordStep(
      "finished",
      `Алгоритм успешно завершен! Найдено ${sccId} компонент,
      null,
      true,
      sccId,
    );

    setSteps(generatedSteps);
    setCurrentStepIdx(0);
  }, []);

const step = steps[currentStepIdx] || {
  phase: "init",
  desc: "Загрузка...",
  visited: new Set(),
  currentNode: null,
  order: [],
  transposed: false,
  sccMap: {},
  currentSccId: -1,
  activeEdges: initialEdges,
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);

```

```

        <RotateCcw className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-12" >
    {/* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] relative" >
      <svg className="w-full h-full max-w-[650px]" >
        {/* Markers for directed arrows */}
        <defs>
          <marker
            id="arrow"
            viewBox="0 0 10 10"
            refX="22"
            refY="5"
            markerWidth="6"
            markerHeight="6"
            orient="auto-start-reverse"
          >
            <path d="M 0 1 L 10 5 L 0 9 z" fill="#66BB6A" />
          </marker>
          <marker
            id="arrow-active"
            viewBox="0 0 10 10"
            refX="22"
            refY="5"
            markerWidth="6"
            markerHeight="6"
            orient="auto-start-reverse"
          >
            <path d="M 0 1 L 10 5 L 0 9 z" fill="#FFC107" />
          </marker>
          <marker
            id="arrow-transposed"
            viewBox="0 0 10 10"
            refX="22"
            refY="5"

```



```

// Adjust slightly for bidirectional link
const isBidi =
  (edge.u === 3 && edge.v === 4) ||
  (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
  <line
    key={`edge-${idx}`}
    x1={x1}
    y1={y1 + dy}
    x2={x2}
    y2={y2 + dy}
    stroke={stroke}
    strokeWidth="2.5"
    markerEnd={`url(#${markerId})`}
    className="transition-all duration-300"
  />
);
}}}

{/* Nodes */}
{initialNodes.map((node) => {
  const isCurrent = step.currentNode === node.id;
  const isVisited =
    step.visited.has(node.id) ||
    (step.phase === "dfs1" && step.currentNode === node.id);
  const sccId = step.sccMap[node.id];

  let classes = "fill-slate-800 stroke-slate-800";
  if (sccId) {
    classes = getSccColor(sccId);
  } else if (isCurrent) {
    classes = "fill-amber-600 stroke-amber-600";
  } else if (isVisited && step.phase === "dfs1") {
    classes = "fill-emerald-700 stroke-emerald-700";
  }
  return (
    <div
      key={node.id}
      className={classes}
      style={{width: 20px, height: 20px, display: flex, align-items: center, justify-content: center; border-radius: 50%; border: 1px solid black; margin: 5px; position: relative; overflow: hidden; background-color: inherit; color: inherit; font-size: 10px; font-weight: bold; text-align: center; line-height: 1; vertical-align: middle;}}
    >
      {node.id}
    </div>
  );
})}

```

```

    </svg>

    {step.transposed && (
      <div className="absolute top-4 left-4 bg-indigo-950 rounded"
        1 rounded">
        Граф транспонирован (Ребра обратные)
      </div>
    )}
  </div>

  {/* Logs & Stacks */}
  <div className="lg:col-span-4 bg-slate-950 p-5 rounded"
    00 overflow-y-auto">
    <h4 className="text-xs font-bold text-slate-400">
      Порядок обхода и стек
    </h4>

    <div className="bg-slate-900 border border-slate-800 p-4">
      <p className="text-xs text-white leading-relaxed">
        {step.desc}
      </p>
    </div>

    <div className="flex-1 space-y-4 overflow-y-auto">
      {/* Top-Sort output / Order stack representation */}
      <div>
        <span className="text-[10px] text-slate-500">
          СТЕК ВЫХОДА (DFS 1 ORDER):
        </span>
        <div className="flex flex-wrap gap-1 bg-slate-900 p-4">
          {step.order.length === 0 ? (
            <span className="text-slate-600 text-sm">
              Нет элементов
            </span>
          ) : (
            [...step.order].reverse().map((val, idx) => (
              <div
                key={idx}
                className="bg-indigo-950 border border-indigo-800
                  px-2 py-1 text-xs text-white flex items-center gap-1"
              >

```

```

        className="bg-slate-900 border border-slate-800"
      >
        Назад
      </button>
      <button
        disabled={currentStepIdx >= steps.length}
        onClick={() => setCurrentStepIdx((prev) => prev - 1)}
        className="bg-slate-900 border border-slate-800"
      >
        Вперед
      </button>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 54/87: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Check } from 'lucide-react';

interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'magenta'
}

interface Edge {

```



```

        title: 'Успех: Ребро В-С в осто́ве',
        description: 'Вершина С перекрашена в красный ц
        type: 'success'
    }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Берем ребро A-D (вес 5)',
        description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
        ю – мы перекрашиваем всё в один цвет!"',
        type: 'warning'
    }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
    setVertices(prev => prev.map(v => v.color === 'bl
    setEdges(prev => prev.map(e => e.status === 'incl
    setLogs(prev => [{
        title: 'Слияние кланов: Ребро A-D добавлено',
        description: 'Мы объединили красный и синий клан
        type: 'success'
    }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Берем ребро B-E (вес 6)',
        description: 'Н0: это ребро соединяет вершины B
        type: 'warning'
    }, ...prev]));
},
// Step 10: Reject B-E
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{

```

```

        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.',
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Берем ребро C-F (вес 9)',
        description: 'Оно соединяет вершины C и F, кото
        type: 'warning'
    }, ...prev]));
},
// Step 16: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро C-F.',
        type: 'error'
    }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 9: Берем ребро G-H (вес 10)',
        description: 'Соединяет фиолетовую G и бесцветн
        type: 'info'
    }, ...prev]));
},
// Step 18: Include G-H
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро G-H в осто́ве',

```



```

    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (
    setLogs(prev => [{
      title: 'Успех: Вершина B захвачена плесенью',
      description: 'В кучу добавлено B-C(4). Ребро B-
      type: 'success'
    }, ...prev]));
  },
  // Step 8: Pop B-C (4)
  () => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
      title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
      description: 'Плесень тянется к вершине C.',
      type: 'info'
    }, ...prev]));
  },
  // Step 9: Include B-C, add C's edges
  () => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
      eap' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (
    setLogs(prev => [{
      title: 'Успех: Вершина C захвачена',
      description: 'Ребро C-F(9) добавлено в кучу.',
      type: 'success'
    }, ...prev]));
  },
  // Step 10: Pop B-E (6) - Cycle!
  () => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
      title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
      description: 'ВНИМАНИЕ! Плесень проверяет верши
      type: 'warning'
    }, ...prev]));
  },
  // Step 11: Reject B-E

```



```

    },
    // Step 15: Include D-G, add G's edges
    () => {
        setVertices(prev => prev.map(v => v.id === 6 ? {
            setEdges(prev => prev.map(e => e.id === '3-6' ? {
                eap' } : e));
        setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
        setLogs(prev => [{
            title: 'Успех: Вершина G захвачена',
            description: 'В кучу добавлено G-H(10).',
            type: 'success'
        }, ...prev]));
    },
    // Step 16: Pop C-F (9) - Cycle!
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
        setLogs(prev => [{
            title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
            description: 'Вершины C и F уже заражены плесенью',
            type: 'warning'
        }, ...prev]));
    },
    // Step 17: Reject C-F
    () => {
        setEdges(prev => prev.map(e => e.id === '2-5' ? {
        setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
        setLogs(prev => [{
            title: 'Отказ: Игнорируем ребро C-F',
            description: 'Выбрасываем из кучи.',
            type: 'error'
        }, ...prev]));
    },
    // Step 18: Pop G-H (10)
    () => {
        setEdges(prev => prev.map(e => e.id === '6-7' ? {
        setLogs(prev => [{
            title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
            description: 'Плесень ползет к вершине H.',

```

```

        title: 'Шаг 12: Куча выдает H-I (вес 12)',
        description: 'Плесень тянется к последней чистой вершине',
        type: 'info'
    }, ...prev]);
},
// Step 23: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setHeapQueue(['F-I (13 - цикл)']));
    setLogs(prev => [{
        title: '      Успех: Плесень захватила весь граф
        description: 'Все 9 вершин поглощены. Итоговый граф
        иной – мы разрастались из одной точки с помощью
        type: 'success'
    }, ...prev]);
},
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
    // Step 1: Five-Year Plan 1 (Inspecting outgoing)
    () => {
        // Highlight the cheapest outgoing edges for EACH
        // Node 0(A) -> A-B(2)
        // Node 1(B) -> A-B(2)
        // Node 2(C) -> B-C(4)
        // Node 3(D) -> D-E(3)
        // Node 4(E) -> D-E(3)
        // Node 5(F) -> E-F(7)
        // Node 6(G) -> D-G(8)
        // Node 7(H) -> G-H(10)
        // Node 8(I) -> H-I(12)
        setEdges(prev => prev.map(e =>
            ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
        ]));
        setLogs(prev => [{
            title: 'Первая пятилетка: Каждая деревня выбирает

```

```

        title: 'Вторая пятилетка: Колхозы выбирают мост
description: 'Теперь Красный и Синий колхозы од
        -D с весом 5. Остальные мосты B-E(6) и C-F(9)
        type: 'warning'
    }, ...prev]);
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
    setVertices(prev => prev.map(v => ({ ...v, color:
    setEdges(prev => prev.map(e =>
        e.status === 'included' || e.id === '0-3' ? { .
    ));
    setLogs(prev => [{
        title: 'Объединение завершено: Один гигантский
        description: 'Оба колхоза объединились по мосту
        type: 'success'
    }, ...prev]);
},
// Step 5: Check remaining edges & mark as rejected
() => {
    setEdges(prev => prev.map(e => e.status === 'pend
    setLogs(prev => [{
        title: '    Успех: MST построен Борувкой за 2 шаг
        description: 'Все лишние ребра (B-E, E-H, C-F,
            вес: 51. Благодаря параллельности, мы потрати
        type: 'success'
    }, ...prev]);
}
];

```

```

const currentSteps =
    activeAlgo === 'kruskal' ? kruskalSteps :
    activeAlgo === 'prim' ? primSteps :
    boruvkaSteps;

```

```

const handleNextStep = () => {
    if (stepIndex < currentSteps.length) {
        currentSteps[stepIndex]();
    }
}

```

```
const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 text-white';
    case 'blue': return 'bg-blue-500 border-blue-300 text-white';
    case 'purple': return 'bg-purple-600 border-purple-300 text-white';
    case 'mold': return 'bg-emerald-500 border-emerald-300 text-white';
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn)
        return 'bg-slate-700 border-emerald-500 text-white';
      return 'bg-slate-700 border-slate-500 text-slate-200';
  }
};
```

```
const getEdgeStroke = (status: string, color: string) => {
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
  return '#475569';
};
```

```
return (
  <div className="space-y-12">
    { /* 3-in-1 MST Simulator Box */ }
    <div className="bg-slate-900/90 border border-slate-800 p-4">
      <div className="flex flex-col lg:flex-row justify-between">
        <div>
          <div className="flex items-center gap-2 mb-2">
            <span className="bg-gradient-to-r from-indigo-500 to-purple-500 text-white font-weight-bold uppercase tracking-wider">

```

```

    <button
      onClick={() => handleReset('boruvka')}
      className={
        "flex items-center gap-1.5 px-3 py-2
+
        (activeAlgo === 'boruvka'
          ? 'bg-rose-600 text-white shadow-md
          : 'text-slate-400 hover:text-slate-1
        )
      >
      <Users className="w-3.5 h-3.5" />
      <span>Борувка (Билет 20)</span>
    </button>
  </div>

  <button
    onClick={() => handleReset(activeAlgo)}
    className="flex items-center justify-center gap-2
-200 font-semibold text-xs rounded-xl border
  >
    <RotateCcw className="w-3.5 h-3.5" />
    <span>Сбросить</span>
  </button>

  <button
    onClick={handleNextStep}
    disabled={stepIndex >= currentSteps.length}
    className={
      "flex items-center justify-center gap-2
initial cursor-pointer " +
      (stepIndex >= currentSteps.length
        ? "bg-slate-800 text-slate-500 border
        : activeAlgo === 'kruskal'
        ? "bg-indigo-600 hover:bg-indigo-500
        : activeAlgo === 'prim'
        ? "bg-emerald-600 hover:bg-emerald-500
        : "bg-rose-600 hover:bg-rose-500 text
      )
  >
    <span>Следующий шаг</span>
  </button>

```

```

        </>
      ) : (
        <>
          <div className="flex items-center gap-2">
            <div className="w-3 h-3 rounded-full bg-slate-300">
            </div>
            <div className="flex items-center gap-2">
              <div className="w-3 h-3 rounded-full bg-slate-300">
              </div>
              <div className="flex items-center gap-2">
                <div className="w-3 h-3 rounded-full bg-slate-300">
                </div>
              </div>
            </div>
          </>
        )}
      </div>

    <div className="absolute top-4 right-4 bg-slate-300 p-4 text-slate-300">
      {stepIndex} / {currentSteps.length}
    </div>

    {/* SVG Edges */}
    <svg className="absolute inset-0 w-full h-full" style={{
      aspectRatio: "none"
    }}>
      {edges.map(edge => {
        const u = vertices[edge.u];
        const v = vertices[edge.v];
        const strokeColor = getEdgeStroke(edge);
        const isInspecting = edge.status === 'inspect';
        const isRejected = edge.status === 'rejected';
        const isInHeap = edge.status === 'in_heap';

        return (
          <g key={edge.id}>
            <line
              x1={u.x + "%"}
              y1={u.y + "%"}
              x2={v.x + "%"}

```

```

        <span className="text-[8px] block -
      }}
    </div>
  )})
</div>

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom
  shadow-xl z-20">
    <p className={"font-bold text-base " +
    'ld-300' : 'text-rose-300'}>
      MST построено с помощью {activeAlgo
    </p>
    <p className="text-slate-300 text-xs mt
      Общий вес минимального остова: 51. Вс
    </p>
  </div>
  )}
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl bord
  <div className="p-4 bg-slate-900 border-b b
    <h4 className="font-bold text-slate-200 t
      Ход выполнения
    </h4>
  </div>

  {activeAlgo === 'prim' && (
    <div className="p-3 bg-slate-900/60 borde
      <p className="text-[11px] font-bold tex
        ⚡ Приоритетная очередь (Min-Heap):
      </p>
      <div className="flex flex-wrap gap-1.5"
        {heapQueue.length === 0 ? (
          <span className="text-xs text-slate
        ) : (
          heapQueue.map((item, idx) => (

```

```
</div>
```

```
{/* Cheat Sheet: Complexity & Code for all 3 algo
```

```
<div className="bg-slate-900/80 border border-sla
```

```
  <div className="flex flex-col sm:flex-row items
```

```
    <div className="flex items-center gap-2.5">
```

```
      <BookOpen className="w-5 h-5 text-indigo-400
```

```
      <h4 className="text-xl font-bold text-white
```

```
    </div>
```

```
<div className="flex bg-slate-950 p-1 rounded
```

```
  <button
```

```
    onClick={() => setActiveCheatTab('kruskal
```

```
    className={"px-3 py-1.5 rounded text-xs f
```

```
    == 'kruskal' ? "bg-indigo-600 text-white" : "
```

```
  >
```

```
    Краскал (Билет 18)
```

```
  </button>
```

```
  <button
```

```
    onClick={() => setActiveCheatTab('prim')}
```

```
    className={"px-3 py-1.5 rounded text-xs f
```

```
    == 'prim' ? "bg-emerald-600 text-white" : "
```

```
  >
```

```
    Прима (Билет 19)
```

```
  </button>
```

```
  <button
```

```
    onClick={() => setActiveCheatTab('boruvka
```

```
    className={"px-3 py-1.5 rounded text-xs f
```

```
    == 'boruvka' ? "bg-rose-600 text-white" : "
```

```
  >
```

```
    Борувка (Билет 20)
```

```
  </button>
```

```
</div>
```

```
</div>
```

```
{activeCheatTab === 'kruskal' && (
```

```
  <div className="grid grid-cols-1 lg:grid-cols
```

```
    <div className="lg:col-span-1 space-y-4">
```



```

        <p className="text-2xl font-black text-white">
        <p className="text-slate-400 text-xs mt-2">
    p>
    </div>
    <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
            <Award className="w-3.5 h-3.5 text-rose-500">
        </p>
        <p className="text-2xl font-black text-white">
        <p className="text-slate-400 text-xs mt-2">
    </div>
    <div className="bg-slate-950 p-4 rounded-3xl">
        <p className="text-slate-400 text-xs font-medium">
            ении:</p>
            <p className="text-xs text-slate-300 leading-5">
                , организуя массовое распараллеленное слияние
            </p>
        </div>
    </div>
    <div className="lg:col-span-2">
        <div className="bg-slate-950 p-4 rounded-3xl">
            <p className="text-slate-400 text-xs font-medium">
                <Code className="w-3.5 h-3.5 text-rose-500">
            </p>
            <pre className="text-xs text-emerald-400">
                80 flex-1">
{`def boruvka(n, edges):
    parent = list(range(n))
    mst = []
    while len(mst) < n - 1:
        # Для каждого колхоза ищем самый дешёвый мост на
        cheapest = [-1] * n
        for u, v, w in edges:
            ru, rv = find(u), find(v)
            if ru != rv:
                if cheapest[ru] == -1 or cheapest[ru][2] > w:
                    if cheapest[rv] == -1 or cheapest[rv][2] > w:
                        # Объединяем все колхозы по выбранным мостам
                        for bridge in cheapest:

```

```

    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый – зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/60',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Все-ко-Все Флойд',
    title: '    Все ко Все (Флойд)',
    desc: 'Матрица + три цикла {',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: '}'. Ищет пути от всех ко всем.',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald-400/60',
    titleColor: 'text-emerald-400',
    badge: 'text-emerald-400/70 bg-emerald-950/40',
  },
];

```

```

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt}
              className="w-full h-full object-cover grid-item">
          </div>
          <div className="p-4">
            <h3 className={`text-lg font-bold mb-1 ${c.highlightColor}`>

```

```

    document.body.style.overflow = open ? "hidden" : ""
    return () => {
      document.body.style.overflow = ""
    }
  }, [open]));

useEffect(() => {
  const onKey = (e: KeyboardEvent) => e.key === "Escape";
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, []);

return (
  <>
    <div className="lg:hidden sticky top-[57px] z-40">
      <button
        onClick={() => setOpen(true)}
        className="w-full flex items-center gap-2.5 p-2"
        aria-label="Открыть содержание"
      >
        <span className="p-1.5 rounded-lg bg-indigo-600 text-white">
          <List className="w-4 h-4" />
        </span>
        <span className="min-w-0 flex-1">
          <span className="block text-[10px] uppercase">
            Содержание • {index + 1} из {total}
          </span>
          <span className="block text-[13px] font-bold">
            {title}
          </span>
        </span>
      </button>
    </div>

    {open && (
      <div className="lg:hidden fixed inset-0 z-[90]">
        <div className="absolute inset-0 bg-slate-950">
          <div className="absolute inset-y-0 left-0 w-[100%] h-[100%]
            te-[tocIn_.18s_ease-out]">
            <div className="flex items-center justify-between">

```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {
  const [busy, setBusy] = useState(false);

  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };

  return (
    <header className="bg-slate-900 border-b border-slate-800">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center gap-3 w-full">
          <div className="bg-gradient-to-tr from-indigo-500 to-purple-600 w-6 h-6">
            <Sparkles className="w-6 h-6" />
          </div>
          <div className="min-w-0 flex-1">
            <h1 className="text-xl font-extrabold text-white">
              Универсальное пособие
            </h1>
            <p className="text-xs text-indigo-400 font-medium">
              Подготовка к экзаменам: Алгоритмы, Структуры данных
            </p>
          </div>
        </div>
      </div>

      <div className="flex items-center w-full lg:w-auto">
        <button
          id="tab-guide-btn"
          onClick={() => setActiveTab('guide')}
          className={`flex-1 lg:flex-none flex items-center`>
```

```
        className="flex items-center justify-center
        :bg-sky-600/20 border border-sky-500/30 tra
        title="Скачать полный TXT файл со всем кодом
      >
      <FileText className="w-4 h-4" /> TXT
    </a>
    <button
      id="download-html-btn"
      onClick={saveBook}
      disabled={busy}
      className="flex items-center justify-center
      er:bg-amber-600/20 border border-amber-500/
      title="Скачать всё пособие одним автономным
    >
      {busy ? <Loader2 className="w-4 h-4 animate
    </button>
  </div>
</div>
</header>
);
};
```

ФАЙЛ 58/87: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотри
    </div>
    <div className="grid grid-cols-1 md:grid-cols-2 g
      { /* K5 */ }
      <div className="bg-slate-950 rounded-xl p-4 bor
        <div className="absolute top-2 left-2 bg-slate
```

```

        const xing = crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x}
            stroke={xing ? "#f43f5e" : "#4f46e5"}
        >);
    }
    const labels = ["A", "B", "C", "D", "E"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill=
            <text x={p.x} y={p.y+3} textAnchor="m
            </g>);
    }
    return <>{lines}{circles}</>;
  })()}
</svg>
<div className="absolute bottom-2 left-2 right
  z-20">
  V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 bor
  <div className="absolute top-2 left-2 bg-slate
    00/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
  <svg className="w-full h-full absolute inset-0
    {(() => {
      const pts = [
        {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
        {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
      ];
      const edges = [
        [0,3],[0,4],[0,5],
        [1,3],[1,4],[1,5],
        [2,3],[2,4],[2,5]
      ];
      function findPt(id:number) { return pts.f
      function intersect(a:number,b:number,c:nu
        if (a===c||a===d||b===c||b===d) return

```

```
    <div className="bg-amber-950/40 border border-amber-950">
      ▲ Запомни: K5 и K3,3 – два кита непланарности.
      гомеоморфный K5 или K3,3 – он непланарен. Теорема Кёрши.
    </div>
  </div>
);
}
```

ФАЙЛ 59/87: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useCallback, useEffect, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";
import { getFallbackMeta, getMinimalStarter } from "../utils";
```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
```

```
const STARTER_CODE = `# Первый запуск загрузит Python в интерпретатор
name = "алгоритмы"`;
```

```
for number in range(1, 4):
    print(f"{number}. Учим {name}!")
`;
```

```
type PyodideRuntime = {
  runPythonAsync: (source: string) => Promise<unknown>;
  setStdout: (options: { batched: (message: string) => void }) => void;
  setStderr: (options: { batched: (message: string) => void }) => void;
  setStdin: (options: { stdin: () => string | number | null }) => void;
};
```

```
type PyodideModule = {
```



```

const onSync = (e: Event) => {
  const d = (e as CustomEvent).detail;
  if (!d?.vars) return;
  if (chapterId && d.chapterId && d.chapterId !== chapterId) {
    setLiveVars(d.vars);
  }
  window.addEventListener("viz:sync", onSync as EventListener);
  return () => window.removeEventListener("viz:sync", onSync);
}, [chapterId]);

useEffect(() => {
  setCode(syncedStarter);
  setLiveVars(null);
}, [chapterId]);

const runCode = useCallback(async () => {
  if (running) return;
  setRunning(true);
  setOutput("");
  setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово");
  const stdout: string[] = [];
  const stderr: string[] = [];
  const inputLines = stdin.split(/\r?\n/);
  try {
    const runtime = await getPythonRuntime();
    setRuntimeReady(true);
    setRuntimeMessage("Python готов – код выполняется");
    runtime.setStdout({ batched: (message) => stdout.push(message) });
    runtime.setStderr({ batched: (message) => stderr.push(message) });
    runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
    await runtime.runPythonAsync(code);
    const text = [...stdout, ...stderr].join("");
    setOutput(text || "Готово: программа ничего не вывела");
  } catch (error) {
    const captured = [...stdout, ...stderr].join("");
    setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
    setRuntimeMessage("Не удалось выполнить код – подробности в консоли");
  } finally {
    setRunning(false);
  }
}, [code, stdin]);

```

```

00 hover:text-white transition-colors">
    Как это работает <ExternalLink className="w
</a>
</div>

{liveVars && (
  <div className="mb-4 rounded-xl border border
    mono text-xs">
    <span className="inline-flex items-center g
      <span className="w-2 h-2 rounded-full bg-
    </span>
    {Object.entries(liveVars).map(([k, v]) => (
      <span key={k} className="bg-slate-900 bor
        {k}={String(v)}
      </span>
    ))}
    <span className="ml-auto text-[10px] text-s
  </div>
)}

<div className="grid xl:grid-cols-[minmax(0,1.3
  <section className="rounded-2xl border border
    <div className="flex flex-wrap items-center
      <div className="flex items-center gap-2">
        <span className="w-2.5 h-2.5 rounded-fu
        <span className="text-sm font-bold text
        <span className="text-[11px] text-slate
          {meta && <span className="text-[10px] b
        </div>
      <div className="flex items-center gap-2">
        {meta && (
          <button type="button" onClick={insertI
            text-indigo-300 hover:text-white hover:bg-
              <Feather className="w-3.5 h-3.5" />
            </button>
          )}
          <button type="button" onClick={() => na
            ter gap-1 px-2 py-1.5 rounded-lg text-xs te

```

```
      <div className="mt-5 grid md:grid-cols-3 gap-3">
        <div className="flex gap-2 rounded-xl border k-0 text-indigo-400 mt-0.5" /><span>Ctrl/Cm
        <div className="flex gap-2 rounded-xl border k-0 text-amber-400 mt-0.5" /><span>Первый э
        <div className="flex gap-2 rounded-xl border k-0 text-emerald-400 mt-0.5" /><span>Код в
      </div>
    </div>
  </main>
);
}
```

ФАЙЛ 60/87: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';

interface Customer {
  id: string;
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
  { name: 'Студент Вася', emoji: '👨‍🎓', color: 'from-blue-500 to-violet-500', bubbleText: 'Матана!' },
  { name: 'Кодер Семён', emoji: '👨‍💻', color: 'from-violet-500 to-purple-500', bubbleText: 'Порцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber-500 to-orange-500', bubbleText: 'Плиз.' },
];
```

```

    counter++;

    const newGuest: Customer = {
      id: Date.now().toString(),
      ...preset,
      animState: 'in',
    };

    setQueue(prev => [...prev, newGuest]);
    addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoji}`);

    setTimeout(() => {
      setQueue(prev => prev.map(c => c.id === newGuest.id ? { ...c, animState: 'out' } : c), 450);
    }, [queue, isServing]);

    // DEQUEUE: Выдать суп первому (голова / Head, FIFO)
    const dequeue = useCallback(() => {
      if (isServing) return;
      if (queue.length === 0) {
        setShake(true);
        addLog(`⚠ DEQUEUE: В очереди никого нет! Повар скучает`);
        setTimeout(() => setShake(false), 400);
        return;
      }

      setIsServing(true);
      const headGuest = queue[0];

      addLog(`- DEQUEUE: Повар наливает суп первому – ${headGuest.name} ${headGuest.emoji}`);

      // Step 1: Head guest gets eating animation
      setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'eat' } : c));

      setTimeout(() => {
        // Step 2: Guest leaves with happy face, queue shake
        setQueue(prev => prev.map((c, idx) => idx === 0 ? { ...c, animState: 'happy' } : c));
      }, 400);
    }, [queue, isServing]);
  }, [queue, isServing]);
}

```

Здесь нет обхода и блата – только строгий п

```
</p>
</div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap"
  <button
    onClick={enqueue}
    disabled={isServing}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-800 to-slate-700 text-white
      font-medium rounded-lg px-4 py-3
      disabled:opacity-40 disabled:cursor-not-allowed transition-all
      hover:scale-95 transition-all cursor-pointer"
  >
    <span> Встать в очередь (ENQUEUE)</span>
  </button>

  <button
    onClick={dequeue}
    disabled={isServing || queue.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-800 to-slate-700 text-white
      font-medium rounded-lg px-4 py-3
      disabled:opacity-40 disabled:cursor-not-allowed transition-all
      hover:scale-95 transition-all cursor-pointer flex"
  >
    <span> Налить суп первому (DEQUEUE)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800
      text-white font-medium cursor-pointer border border-slate-700"
  >
    Сброс
  </button>
</div>
```

```
{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12">
```

```

<div className="flex-1 flex flex-col" style={
  <span className="text-4xl mb-1"> </
  <p className="text-xs font-bold font
  <p className="text-[10px]">Все сыты
</div>
) : (
  queue.map((customer, idx) => {
    const isHead = idx === 0;
    const isTail = idx === queue.length - 1;
    return (
      <div
        key={customer.id}
        className={`
          flex flex-col items-center gap-2
          ${customer.animState === 'in' ? 'animate-in' : ''}
          ${customer.animState === 'eat' ? 'animate-eat' : ''}
          ${customer.animState === 'out' ? 'animate-out' : ''}
        `
      >
        {/* Облачко мыслей */}
        <div className={`
          absolute -top-16 left-1/2 -translate-x-50%
          8 text-center text-slate-200 leading-tight
          ${isHead ? 'opacity-100 border' : ''}
        `}>
          <div className="font-bold text-slate-200">
            {customer.bubbleText}
          <div className="absolute -bottom-16 left-1/2 -translate-x-50%
            it rotate-45"></div>
          </div>

          {/* Персонаж */}
          <div className={`
            w-14 h-14 rounded-2xl border-2
            relative
            ${customer.color}
            ${isHead ? 'ring-4 ring-amber' : ''}
          `}>

```

```

    {servedHistory.length === 0 ? (
      <div className="flex-1 flex flex-col item">
        <span className="text-5xl mb-2"> </span>
        <p className="text-xs font-bold font-mono">
          <p className="text-[10px] mt-1">Здесь 6
        </div>
      ) : (
        <div className="flex flex-col gap-1.5 w-full">
          {servedHistory.map((item, idx) => (
            <div
              key={idx}
              className="w-full py-1.5 px-3 rounded-
0 flex items-center gap-2 text-[11px] text-
            >
              <span>{item}</span>
            </div>
          ))}
        </div>
      )}
    </div>

    {/* Пол */}
    <div className="w-full h-3 bg-gradient-to-r f
    </div>
  </div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-
    <div className="text-[10px] font-mono text-slate
      {log.map((entry, idx) => (
        <div
          key={idx}
          className={`text-xs font-mono flex items-ce
        >
          {idx === 0 ? <span className="text-indigo-5
          <span>{entry}</span>
        </div>
      )}}
    )}}
  )}}

```

```

    );
    const [isTalking, setIsTalking] = useState<boolean>(false);
    const [isBlinking, setIsBlinking] = useState<boolean>(false);

    // Регулярное моргание рыбы
    useEffect(() => {
        const blinkInterval = setInterval(() => {
            setIsBlinking(true);
            setTimeout(() => setIsBlinking(false), 200);
        }, 4000);
        return () => clearInterval(blinkInterval);
    }, []);

    // Эффект разговора при смене текста
    useEffect(() => {
        setIsTalking(true);
        const timer = setTimeout(() => setIsTalking(false), 1000);
        return () => clearTimeout(timer);
    }, [speechText]);

    // Построение начального бора (Trie) без fail-ссылок
    const buildInitialTrie = (wordList: string[]) => {
        const newNodes: { [key: number]: TrieNode } = {
            0: { id: 0, char: 'ROOT', parent: 0, children: {} }
        };
        let count = 0;

        wordList.forEach((word) => {
            let curr = 0;
            for (let i = 0; i < word.length; i++) {
                const c = word[i].toUpperCase();
                if (newNodes[curr].children[c] === undefined) {
                    count++;
                    newNodes[curr].children[c] = count;
                    newNodes[count] = {
                        id: count,
                        char: c,
                        parent: curr,
                    };
                }
                curr = newNodes[curr].children[c];
            }
        });
    };

```



```

    };

    layoutDFS(0, 0);

    setNodes(newNodes);
    setSimulationStep(0);
    setBfsQueue([]);
    setCurrentNode(null);
    setSpeechText(
      `Отлично! Я построил базовое префиксное дерево (БПД)
      S, чтобы расставить fail-ссылки!`
    );
  };
};

useEffect(() => {
  buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();
  const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
  if (!cleanWord) return;
  if (words.includes(cleanWord)) {
    setSpeechText(`Слово «${cleanWord}» уже есть в слове!`);
    return;
  }
  const updated = [...words, cleanWord];
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
  if (words.length <= 1) {
    setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем работать!');
    return;
  }
  const updated = words.filter((w) => w !== wordToRemove);
  setWords(updated);
  setNewWord('');
  buildInitialTrie(updated);
};

```

```

const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.char})`;

for (const [char, u] of Object.entries(rNode.children)) {
  newQueue.push(u);
  let v = rNode.fail;
  while (v !== 0 && updatedNodes[v].children[char]) {
    v = updatedNodes[v].fail;
  }
  const failTarget = updatedNodes[v].children[char];
  updatedNodes[u].fail = failTarget;

  // Рассчитываем терминальную ссылку (term_link)
  if (updatedNodes[failTarget].is_terminal) {
    updatedNodes[u].term_link = failTarget;
  } else {
    updatedNodes[u].term_link = updatedNodes[failTarget].term_link;
  }

  logs += `Для #${u} ('${char}') fail = #${failTarget}`;
}

if (childrenIds.length === 0) {
  logs += 'У этой вершины нет детей, просто переходим к следующему';
}

setNodes(updatedNodes);
setBfsQueue(newQueue);
setCurrentNode(r);
setSpeechText(logs);
};

return (
  <div className="bg-slate-800/90 rounded-2xl p-6 border" >
    {/* Шапка виджета */}
    <div className="flex flex-wrap items-center justify-between" >

```

```
<path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85 Z" fill="#f87171" stroke="#f87171" stroke-width="2px"
  className="origin-[40px_85px] animate-spin" />
```

```
{/* Хвост */}
```

```
<path
  d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
  fill="#be123c"
  className="origin-[35px_100px] animate-spin"
/>
```

```
{/* Плавники */}
```

```
<path d="M 90 55 L 70 30 L 110 55 Z" fill="#4682b4" stroke="#4682b4" stroke-width="2px"
  className="origin-[90px_55px] animate-spin" />
```

```
<path d="M 90 145 L 70 170 L 110 145 Z" fill="#4682b4" stroke="#4682b4" stroke-width="2px"
  className="origin-[90px_145px] animate-spin" />
```

```
<path
  d="M 120 115 L 100 135 L 130 130 Z"
  fill="#f87171"
  className="origin-[120px_115px] animate-spin"
/>
```

```
{/* Глаз (с морганием) */}
```

```
<circle cx="145" cy="85" r="12" fill="#fff" stroke="#000" stroke-width="2px"
  {isBlinking ? (
    <line x1="133" y1="85" x2="157" y2="85" stroke="#000" stroke-width="2px"
  ) : (
    <circle cx="148" cy="85" r="6" fill="#fff" stroke="#000" stroke-width="2px"
    <circle cx="151" cy="82" r="2" fill="#fff" stroke="#000" stroke-width="2px"
  )}
/>
  )}
```

```
{/* Рот (анимация разговора) */}
```

```
{isTalking ? (
  <path d="M 165 105 Q 155 120 165 125 Q 165 110 155 115 168 118" stroke="#000" stroke-width="2px"
) : (
  <path d="M 165 110 Q 155 115 168 118" stroke="#000" stroke-width="2px"
)}
  )}
```

```
{/* Улыбка / жабры */}
```

```
<path d="M 130 80 Q 125 100 130 120" stroke="#000" stroke-width="2px"
  className="origin-[130px_80px] animate-spin" />
```

```

        onClick={startBFS}
        className="bg-emerald-600 hover:bg-emerald-700 shadow-lg shadow-emerald-900/40 transition-transform"
      >
        <Play className="w-4 h-4" /> Запустит
      </button>
    )}
    {simulationStep === 1 && (
      <button
        onClick={stepBFS}
        className="bg-blue-600 hover:bg-blue-700 shadow-lg shadow-blue-900/40 transition-transform"
      >
        <span>Шаг BFS</span>
        <span className="bg-blue-800 px-1.5 py-0.5" />
      </button>
    )}
    {simulationStep === 2 && (
      <button
        onClick={() => buildInitialTrie(words)}
        className="bg-indigo-600 hover:bg-indigo-700 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
      >
        <RotateCcw className="w-4 h-4" /> Перезагрузит
      </button>
    )}
  </div>
</div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800 p-4" >
  <h5 className="text-white font-bold mb-3 flex items-center" >
    <span>Дерево и Суффиксные (fail) ссылки</span>
  </h5>

  {(() => {

```

```

    />
    <text
      x={({parent.x + node.x) / 2 - 10}
      y={({parent.y + node.y) / 2}
      fill="#94a3b8"
      fontSize="12"
      fontWeight="bold"
    >
      {node.char}
    </text>
  </g>
);
}})

/* Draw Fail Links */
[simulationStep > 0 && Object.values(nodes)
  .filter(node => node.id !== 0 || node.fail !== 0)
  .map(node => {
    // Wait, all nodes get fail link. Let's
    // step >= 1 and computed it
    // To avoid too many lines, let's draw
    const target = nodes[node.fail];
    if (!target || !node.x || !node.y || !target.x || !target.y) return;

    // Simple bezier curve for fail link
    const isCurrent = currentNode === node;

    return (
      <path
        key={`fail-${node.id}`}
        d={`M ${node.x} ${node.y - 10} Q ${
          node.x + 16, node.y - 16
        }`}
        fill="none"
        stroke={isCurrent ? '#f43f5e' : 'rgb(100, 100, 100)'}
        strokeWidth={isCurrent ? 2 : 1.5}
        strokeDasharray="4 4"
        markerEnd="url(#fail-arrow)"
      />
    );
  })
];

```

```

        fontSize={isRoot ? "10" : "12"}
        fontWeight="bold"
        textAnchor="middle"
      >
        {isRoot ? 'R' : node.char}
      </text>
      {!isRoot && {
        <text
          x={node.x + 12}
          y={node.y - 12}
          fill="#94a3b8"
          fontSize="9"
        >
          {node.id}
        </text>
      }}
    </g>
  );
  }}}
</svg>
);
}}())}
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
  <div className="bg-slate-900/50 p-5 rounded-xl">
    <div>
      <h5 className="text-white font-bold mb-3">
        <span> </span> Словарь (Паттерны)
      </h5>
      <div className="flex flex-wrap gap-2 mb-4">
        {words.map((w) => {
          <span
            key={w}
            className="bg-slate-800 border border-slate-700 padding-2 2px"
            style={{ display: inline-block, text-align: center, width: 100px, height: 30px; vertical-align: middle; font-size: 0.8em; font-weight: bold; color: white; line-height: 1.2; border-radius: 4px; border: 1px solid #444; }}>
            {w}
          </span>
        )}
      </div>
    </div>
  </div>
</div>

```

```

    </span>
    <span className="text-xs text-slate-400 font-mono">
      Всего вершин: {Object.keys(nodes).length}
    </span>
  </h5>

  <div className="max-h-[220px] overflow-y-auto">
    <table className="w-full text-left border-collapse">
      <thead>
        <tr className="border-b border-slate-700">
          <th className="py-2 px-3">ID Вершины</th>
          <th className="py-2 px-3">Символ</th>
          <th className="py-2 px-3">Родитель</th>
          <th className="py-2 px-3">fail-ссылка</th>
          <th className="py-2 px-3">term_link</th>
          <th className="py-2 px-3">Дети</th>
          <th className="py-2 px-3">Слова (dict)</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-800">
        {Object.values(nodes).map((node) => {
          const isCurrent = currentNode === node.id
          const isInQueue = bfsQueue.includes(node.id)

          return (
            <tr>
              <td>
                {node.id}
              </td>
              <td>
                {node.symbol}
              </td>
              <td>
                {node.parent}
              </td>
              <td>
                {node.fail}
              </td>
              <td>
                {node.term_link}
              </td>
              <td>
                {node.children}
              </td>
              <td>
                {node.words}
              </td>
            </tr>
          )
        })}
      </tbody>
    </table>
  </div>

```

```

        </td>
        <td className="py-2.5 px-3 text-slate-600">
          {Object.keys(node.children).length === 0 ? '∅' : Object.entries(node.children)
            .map(([c, id]) => `${c}→#${id}`)
            .join(', ')}
        </td>
        <td className="py-2.5 px-3">
          {node.words.length === 0 ? (
            <span className="text-slate-600">
              ∅
            </span>
          ) : (
            <span className="bg-emerald-900 text-white">
              {node.words.join(', ')}
            </span>
          )}
        </td>
      </tr>
    </tbody>
  </table>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 62/87: src/components/SegmentTreeVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

```



```

"    if (l === r) {" , // 2
"        tree[v] = A[l]; return" , // 3
"    }" , // 4
"    mid = (l + r) >> 1" , // 5
"    build(2v, l, mid)" , // 6
"    build(2v+1, mid+1, r)" , // 7
"    tree[v] = tree[2v]+tree[2v+1]" , // 8
"    }" , // 9
];
void CODE;

function go(v: number, l: number, r: number) {
    if (l === r) {
        steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
            , highlightNodes: [v], arrayHighlight: [l, r] });
        tree[v] = arr[l];
        steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
            l, r] });
        return;
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
        { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v, l, m);
    steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
        lightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]`,
        tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: tree[1]`,
    Highlight: [0, n - 1] });
return steps;
}

// 2. Top-Down Query steps

```

```

for (let i = 0; i < n; i++) tree[n + i] = arr[i];
for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]);

const steps: Step[] = [];
let id = 0;
let l = qL + n;
let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`, codeLine: 3, treeState: tree, highlightNodes: [l, r] });

while (l < r) {
  const highlights: number[] = [];
  if (l & 1) {
    res += tree[l] ?? 0;
    steps.push({ id: id++, desc: `l=${l} нечётный → l++`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l] });
    highlights.push(l);
    l++;
  }
  if (r & 1) {
    r--;
    res += tree[r] ?? 0;
    steps.push({ id: id++, desc: `r=${r + 1} нечётный → r--`, codeLine: 6, treeState: tree, highlightNodes: [r], arrayHighlight: [r] });
    highlights.push(r);
  }
  l >>= 1;
  r >>= 1;
  if (l < r) {
    steps.push({ id: id++, desc: `Поднимаемся: l = ${l}, r = ${r}`, codeLine: 7, treeState: tree, highlightNodes: [l, r], arrayHighlight: [qL, qR], result: res });
  }
}

steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`, codeLine: 8, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });

return steps;

```

```

"  l = ql + n; r = qr + n + 1;",
"  // -----",
"  while (l < r) {",
"    if (l & 1) res += tree[l++];",
"    // -----",
"    if (r & 1) res += tree[--r];",
"    l >>= 1; r >>= 1;",
"    return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(700);

  useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

  useEffect(() => {
    if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
      const t = setTimeout(() => setIdx(i => i + 1), speed);
      return () => clearTimeout(t);
    }
  }, [playing, idx, steps, speed]);

  const step = steps[idx] || steps[0];
  if (!step) return null;

  const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-900' } };

  return { step, idx, setIdx, playing, setPlaying, speed, clr };
}

```

```

    );
  }, [step, clr]);

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'
    /* Header */
    <div className={`px-4 py-2.5 flex items-center ju
      -emerald-950/50' : 'bg-amber-950/50'} border-b
      <h4 className="font-bold text-sm text-white fle
      <span className="text-[10px] font-mono text-sla
    </div>

    /* Array strip */
    <div className="px-3 py-2 bg-slate-950 border-b b
      {arr.map((v, i) => {
        const inRange = step.arrayHighlight && i >= s
        return (
          <div key={i} className={`flex flex-col item
            go' ? 'bg-indigo-950 border-indigo-500' : c
            er-500'} -translate-y-0.5` : 'bg-slate-900
            <span className="text-[7px] text-slate-50
            <span className="text-[11px] font-bold for
          </div>
        );
      })}
    </div>

    /* Tree */
    <div className="flex-1 p-2 overflow-x-auto bg-sla
      <div className="scale-[0.85] origin-top min-w-m
    </div>

    /* Code */
    <pre className="px-3 py-2 text-[9px] md:text-[10px]
      x">
      {code.map((line, i) => {
        const ln = i + 1;

```

```

    </div>
    <select value={speed} onChange={e => setSpeed(N
      text-[9px] font-bold rounded px-1.5 py-1 outl
    <option value={1200}>Медл.</option>
    <option value={700}>Норма</option>
    <option value={350}>Быстро</option>
    <option value={120}>Турбо</option>
  </select>
</div>

  {/* Progress bar */}
  <div className="h-1 bg-slate-950"><div className=
    + 1) / total) * 100 : 0}%` }} /></div>
</div>
  );
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12
  const [customInput, setCustomInput] = useState('5, 8,
  const [qL, setQL] = useState(1);
  const [qR, setQR] = useState(4);
  const [view, setView] = useState<'build' | 'queries'

  // Clamp query range when array changes
  useEffect(() => {
    setQL(prev => Math.min(prev, arr.length - 1));
    setQR(prev => Math.min(prev, arr.length - 1));
  }, [arr]);

  const handleApply = (e: React.FormEvent) => {
    e.preventDefault();
    const parsed = customInput.split(',').map(s => pars
    if (parsed.length >= 2 && parsed.length <= 16) {
      setArr(parsed);
    }
  }
}

```

```

        ution-colors">Применить</button>
      </div>
      <div className="flex flex-wrap gap-1.5 pt-1" >
        {arr.map((v, i) => (
          <span key={i} className="bg-slate-900 b
            <span className="text-slate-500">A[{i
          </span>
        ))}
        <span className="text-xs text-slate-500 f
      </div>
    </form>

    {/* Query range */}
    <div className="bg-slate-950 p-4 rounded-xl b
      <label className="text-[10px] font-bold upp
      <div className="flex items-center gap-2">
        <div className="flex items-center gap-1">
          <span className="text-xs text-slate-400
          <input
            type="number"
            min={0}
            max={arr.length - 1}
            value={qL}
            onChange={e => setQL(Math.min(Number(
            className="w-14 bg-slate-900 border b
        focus:border-indigo-500"
      />
    </div>
    <div className="flex items-center gap-1">
      <span className="text-xs text-slate-400
      <input
        type="number"
        min={0}
        max={arr.length - 1}
        value={qR}
        onChange={e => setQR(Math.min(Number(
        className="w-14 bg-slate-900 border b
    focus:border-indigo-500"
  
```

```

        arr={arr}
    />
  })

{view === 'queries' && {
  <div className="grid grid-cols-1 xl:grid-cols-2"
    <SimPanel
      key={`qtd-${arr.join(' - ')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Сверху`}
      icon="↑"
      steps={queryTDSteps}
      code={QUERY_TD_CODE}
      color="emerald"
      arr={arr}
    />
    <SimPanel
      key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
      title={`Запрос sum([${qL}..${qR}]) – Снизу`}
      icon="↓"
      steps={queryBUSteps}
      code={QUERY_BU_CODE}
      color="amber"
      arr={arr}
    />
  </div>
})

{view === 'theory' && {
  <div className="space-y-5">
    /* Why it splits this way */
    <div className="bg-slate-950 p-5 rounded-xl border"
      <h4 className="text-base font-bold text-white">
        Почему разбиение именно так для N={arr.length}?
      </h4>
      <p className="text-sm text-slate-300 leading-relaxed">
        Каждый узел берёт <code className="bg-slate-900 px-1">
        и получает <code className="bg-slate-900 px-1">
        -slate-900 px-1 rounded font-mono text-emerald-400
        евая часть забирает на 1 больше (округление

```



```

        />
        {query && (
          <button
            onClick={() => setQuery("")}
            className="absolute right-2 top-1/2 -tran
            aria-label="Очистить поиск"
          >
            <X className="w-3.5 h-3.5" />
          </button>
        )}
      </div>
    </div>

    <div className="flex-1 overflow-y-auto px-3 pb-4
      {groups.length === 0 && <p className="text-sm t

    {groups.map(([category, items]) => (
      <div key={category}>
        <div className="text-[10px] font-bold upperc
        <div className="space-y-1">
          {items.map((chapter) => {
            const isSelected = chapter.id === selec
            const hasViz = Boolean(VIZ_REGISTRY[cha
            return (
              <button
                key={chapter.id}
                onClick={() => {
                  setSelectedChapterId(chapter.id);
                  onNavigate?.();
                  window.scrollTo({ top: 0, behavio
                }}
                className={`w-full text-left px-3 p
                  isSelected
                    ? "bg-indigo-600/15 border-indi
                    : "bg-slate-800/40 border-transp
                `}
              >
            </button>
          )}
        >
        {hasViz && (

```

```
import { createPortal } from "react-dom";
import { CompilerPanelContent } from "../CompilerPanelContent";

interface Props {
  chapterId: string;
  open: boolean;
  onClose: () => void;
  onOpenFull: () => void;
}

/**
 * Мобильный drawer – на десктопе используется inline п
 * здесь – только оверлей для телефонов (lg:hidden в Ар
 * Контент переиспользует CompilerPanelContent – минима
 */
export function SideCompilerDrawer({ chapterId, open, on
  if (!open) return null;
  return createPortal(
    <div className="fixed inset-0 z-[90] flex justify-e
      <div className="absolute inset-0 bg-black/40 back
        <div className="relative w-[92vw] max-w-[400px] h
          <CompilerPanelContent chapterId={chapterId} onC
        </div>
      </div>,
    document.body
  );
}
```

ФАЙЛ 65/87: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar
```

```

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный"', },
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', },
  { id: '3', text: 'Кандидат: "Привет, я испанский ин"', },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.1);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: '🥞' },
    { name: 'Кастрюля от супа', icon: '🍲' },
    { name: 'Чашка от кофе', icon: '☕' },
    { name: 'Миска с остатками салата', icon: '🥗' },
    { name: 'Тарелка от тако', icon: '🌮' },
  ];
  const randomPreset = presets[Math.floor(Math.random() * presets.length)];
  const newPlate = {
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setStack(prev => [...prev, newPlate]);
  setPopMessage(`Положили поверх стопки: ${newPlate.name} 🍽️`);
};

const popPlate = () => {
  if (stack.length === 0) {
    setPopMessage("⚠️ Стопка пуста! Все тарелки вымыты");
    return;
  }
  const topPlate = stack[stack.length - 1];
  setStack(prev => prev.slice(0, prev.length - 1));
  setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name} 🍽️`);
};

```

```

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },
    { id: '3', name: 'Кот Борис', icon: ' ' },
  ]);
  setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');
  setHeapMessage(` Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
  if (heap.length === 0) {
    setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
    return;
  }
  // Find highest probability
  let maxIdx = 0;
  for (let i = 1; i < heap.length; i++) {
    if (heap[i].probability > heap[maxIdx].probability) {
      maxIdx = i;
    }
  }
  const best = heap[maxIdx];
  setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
  setHeapMessage(` Выбран самый "тяжелый" кандидат:

```

```

        <div className="text-left">
          <div className="font-bold text-sm text-wh
            <div className="text-xs opacity-80">LIFO
          </div>
        </div>
        <span className="text-xs bg-blue-500/20 text-l
          DFS
        </span>
      </button>

    <button
      onClick={() => setActiveTab('queue')}
      className={`flex items-center justify-between
        activeTab === 'queue'
          ? 'bg-indigo-600/20 border-indigo-500 tex
            : 'bg-slate-800/60 border-slate-700/60 te
        }`}
    >
      <div className="flex items-center gap-3">
        <AlignLeft className={`w-5 h-5 ${activeTab :
          <div className="text-left">
            <div className="font-bold text-sm text-wh
              <div className="text-xs opacity-80">FIFO
            </div>
          </div>
        </div>
        <span className="text-xs bg-indigo-500/20 tex
          BFS
        </span>
      </button>

    <button
      onClick={() => setActiveTab('heap')}
      className={`flex items-center justify-between
        activeTab === 'heap'
          ? 'bg-emerald-600/20 border-emerald-500 t
            : 'bg-slate-800/60 border-slate-700/60 te
        }`}
    >

```

```

border-blue-500/40 px-4 py-2.5 rounded-xl
>
    Помыть верхнюю
</button>
<button
  onClick={resetStack}
  title="Сбросить"
  className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
>
  <RotateCcw className="w-5 h-5" />
</button>
</div>
</div>

{popMessage && {
  <div className="bg-blue-950/60 border borde
imate-fade-in">
    <span className="inline-block w-2 h-2 roun
    {popMessage}
  </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
>
  {stack.length === 0 ? {
    <div className="text-center py-12 text-sl
      <div className="text-5xl mb-3"> </div>
      <p className="text-base font-bold">Стон
      <p className="text-xs mt-1">Добавьте гр
    </div>
  ) : {
    <div className="w-full max-w-sm flex flex
      <div className="absolute top-0 text-xs
full flex items-center gap-1">
      <span>ВЕРШИНА СТЕКА (TOP)</span>
      <ArrowDown className="w-3.5 h-3.5 ani

```

```

{ /* QUEUE TAB */
{activeTab === 'queue' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-in
            /span>
          </h3>
          <p className="text-slate-400 text-xs mt-1
            Кто первым пришел, тот первым получил с
          </p>
        </div>
        <div className="flex flex-wrap items-center
          <button
            onClick={addPerson}
            className="flex-1 md:flex-none flex ite
              -2.5 rounded-xl text-sm font-bold shadow-lg
            >
              <Plus className="w-4 h-4" /> Встать в о
            </button>
            <button
              onClick={servePerson}
              className="flex-1 md:flex-none flex ite
                er border-indigo-500/40 px-4 py-2.5 rounded
              >
                Налить суп первому
            </button>
            <button
              onClick={resetQueue}
              title="Сбросить"
              className="p-2.5 bg-slate-800 hover:bg-
                tion-colors cursor-pointer"
            >
              <RotateCcw className="w-5 h-5" />
            </button>

```

```

        isFirst
        ? 'bg-gradient-to-b from-indigo-500 to-white'
        : 'bg-slate-900/90 border-slate-900'
      }` }
    >
    {isFirst && {
      <span className="absolute -top-10 left-10 text-white se tracking-wider shadow">
        Первый (Head)
      </span>
    }}
    <span className="text-4xl mb-2">{
      <span className={`font-bold text-white`}
        {person.name}
      </span>
      <span className={`text-[10px] font-mono`}
        {isFirst ? 'bg-indigo-500/30 text-white' : 'bg-white text-indigo-500'}
      </span>
      {isFirst ? 'Получает сун' : 'Входит'}
    }`}>
  </div>
);
}}

<div className="flex flex-col items-center justify-center gap-600 min-w-[120px] h-[120px]">
  <Plus className="w-6 h-6 mb-1" />
  <span className="text-xs font-mono uppercase">
    </div>
</div>
)}
<div className="mt-6 text-xs text-slate-500">
  ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ
</div>
</div>
</div>
})

```



```

<div className="md:col-span-6">
  <label className="block text-xs font-bold">
    <input
      type="text"
      value={newCustomText}
      onChange={e => setNewCustomText(e.target.value)}
      placeholder="например: 'Привет, я лучший!'"
      className="w-full bg-slate-900 border border-emerald-500 transition-colors"
    />
  </div>
<div className="md:col-span-3">
  <label className="block text-xs font-bold">
    Вероятность: <span className="text-emerald-500">
  </label>
  <input
    type="range"
    min="0.01"
    max="0.99"
    step="0.01"
    value={newCustomProb}
    onChange={e => setNewCustomProb(Number(e.target.value))}
    className="w-full accent-emerald-500 cursor-pointer"
  />
</div>
<div className="md:col-span-3">
  <button
    type="submit"
    disabled={!newCustomText.trim()}
    className="w-full flex items-center justify-center gap-2 border border-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
  >
    <Plus className="w-4 h-4" /> Добавить в список
  </button>
</div>
</form>

```

```

        <span className={`flex items-center
            isTop ? 'bg-emerald-500 text-
        }`}>
            #{index + 1}
        </span>
        <div>
            <div className={`font-bold te
                {item.text}
            </div>
            {isTop && (
                <span className="text-[10px
-0.5 rounded inline-block mt-1">
                    Вершина Кучи (Root) • Буд
                </span>
            )}
        </div>
    </div>

    <div className="flex items-center
        /* Custom progress bar */
        <div className="hidden sm:block
            <div
                className={`h-full rounded-
                style={{ width: `${percent}%
            ></div>
        </div>
        <span className={`font-mono font
            isTop ? 'text-emerald-400 tex
        }`}>
            {percent}%
        </span>
    </div>
</div>
    );
    }}}}
</div>
))
<div className="mt-6 text-xs text-slate-500

```

```

-----
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
  {
    name: "Линия влево",
    hintCase: "Zig-Zig",
    target: 1,
    build: () => {
      const n1 = leaf(1);
      const n2: TNode = { key: 2, left: n1, right: leaf
      const n4: TNode = { key: 4, left: n2, right: leaf
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Змейка",
    hintCase: "Zig-Zag",
    target: 3,
    build: () => {
      const n3 = leaf(3);
      const n2: TNode = { key: 2, left: leaf(1), right:
      const n4: TNode = { key: 4, left: n2, right: leaf
      return { key: 6, left: n4, right: leaf(7) };
    },
  },
  {
    name: "Один уровень",
    hintCase: "Zig",
    target: 2,
    build: () => ({ key: 4, left: { key: 2, left: leaf(
  },
  {
    name: "Смешанный",
    hintCase: "Zig-Zag, затем Zig-Zig",
    target: 5,
    build: () => {
      const n5 = leaf(5);
      const n6: TNode = { key: 6, left: n5, right: leaf
      const n4: TNode = { key: 4, left: leaf(3), right:

```

```

    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const g = path.length >= 3 ? path[path.length - 3] : null;

    if (!g) return { node: x.key, kind: "Zig" };

    const xLeft = p.left === x;
    const pLeft = g.left === p;
    if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
    return { node: x.key, kind: "Zig-Zag (сначала нижняя)" };
}

/* ----- раскладка -----

interface Placed {
    key: number;
    x: number;
    y: number;
    depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
    const nodes: Placed[] = [];
    const edges: [number, number][] = [];
    let order = 0;

    const walk = (n: TNode | null, depth: number) => {
        if (!n) return;
        walk(n.left, depth + 1);
        nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
        if (n.left) edges.push([n.key, n.left.key]);
    };

```

```

    },
    [presetIdx]
  );

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)
const expected = useMemo(() => (solved ? null : canonicalNext(
tree, target));
setHistory((h) => [...h, clone(tree) as TNode]);
setMoves((m) => [...m, { node: key, correct: want?.length
setTree((t) => rotateUp(clone(t) as TNode, key));
});

const undo = () => {
  if (history.length === 0) return;
  setTree(history[history.length - 1]);
  setHistory((h) => h.slice(0, -1));
  setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target)

return (
  <div className="w-full bg-slate-950 rounded-2xl border
    <div className="flex items-start justify-between
      <div>
        <h4 className="text-sm sm:text-base font-bold
        <p className="text-[12px] text-slate-400 lead
          Клик по узлу = один поворот с его родителем
          <span className="font-mono font-bold text-in
            поворотов выбираешь сам, разбор покажем в к

```

```

        <line
          key={i}
          x1={na.x}
          y1={na.y}
          x2={nb.x}
          y2={nb.y}
          stroke={onPath ? "#6366f1" : "#475569"}
          strokeWidth={onPath ? 3 : 1.8}
          style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
        />
      );
    }
  }

  {nodes.map((n) => {
    const isTarget = n.key === target;
    const clickable = !solved && n.depth > 0;
    return (
      <g
        key={n.key}
        onClick={() => clickable && clickNode(n)}
        style={{
          transform: `translate(${n.x}px, ${n.y}px)`,
          transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
          cursor: clickable ? "pointer" : "default",
        }}
      >
        {isTarget && (
          <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
        )}
        <circle
          r={16}
          fill={isTarget ? "#6366f1" : path.has(n.key) ? "#a5b4fc" : "#475569"}
          stroke={isTarget ? "#a5b4fc" : "#475569"}
          strokeWidth={2}
        />
        <text textAnchor="middle" dominantBaseline="middle">
          {n.key}
        </text>
      </g>
    );
  })}

```

```

{showAnswer && !solved && expected && {
  <div className="mt-3 text-[12px] bg-amber-950/40" >
    Сейчас splay крутил бы узел{" "}
    <span className="font-mono font-bold">{expected}
  </div>
}}

{solved && {
  <div
    className={`mt-3 rounded-lg border px-3 py-2`}
    wrong === 0
      ? "bg-emerald-950/40 border-emerald-700/60"
      : "bg-rose-950/30 border-rose-800/60 text-rose-50"
  >
    <div className="flex items-center gap-2 font-mono" >
      {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
      {wrong === 0
        ? `Узел ${target} в корне за ${moves.length} ходов`
        : `Узел ${target} в корне за ${moves.length} ходов`
      }
    </div>
    {wrong > 0 && {
      <ul className="list-disc pl-5 space-y-0.5">
        {moves.map(
          (m, i) =>
            !m.correct && {
              <li key={i}>
                ход {i + 1}: крутили <span className="font-mono">{m.expected}
              </li>
            }
          )
        )}
      </ul>
    }}
  </div>
}}
{wrong === 0 && moves.length > minimal && {
  <div className="opacity-80">Порядок верный,
}}

```

```

text: string;
/** Пара, которую крутим на этом шаге: кадр-«прицел».
focus?: [NodeId, NodeId];
/** Кто переехал на этом кадре – рисуем призрак и стрелку.
moved?: NodeId[];
ms: number;
}

```

```

interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах.
inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
  frames: Frame[];
}

```

```

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

```

```

/* ----- Zig -----
const ZIG_BEFORE = {
  pos: { p: [195, 50], x: [105, 130], C: [280, 130], A: [370, 130],
  edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"], ["C", "A"], ["C", "B"], ["A", "B"]],
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIG: Case = {
  key: "zig",
  label: "Zig",
  when: "родитель x – уже корень, деда нет",
  rotations: "1 поворот",

```



```

const ZZ_S1 = {
  pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
  edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
  key: "zig-zig",
  label: "Zig-Zig",
  when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
  rotations: "2 поворота, первым – верхний",
  keys: { x: 20, p: 40, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZZ_S0,
      kind: "start",
      title: "Было: прямая линия 60 → 40 → 20",
      text: "Три узла выстроились в «бамбук»: каждый следующий узел  
зглаживает.",
      ms: RESULT_MS,
    },
    {
      ...ZZ_S0,
      kind: "aim",
      title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
      text: "Главная хитрость этого случая: первым крутит  
фокус: ["g", "p"],",
      ms: AIM_MS,
    },
    {
      ...ZZ_S1,
      kind: "result",
      title: "Поворот 1: 40 поднялось над 60",
      text: "40 встало в корень, 60 опустилось к нему по  
убине 1, а не 2.",
      moved: ["p", "g", "C"],
      ms: RESULT_MS,
    }
  ]
}

```

```

keys: { x: 40, p: 20, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
  {
    ...ZG_S0,
    kind: "start",
    title: "Было: 60 → влево на 20 → вправо на 40",
    text: "Узлы идут «змейкой»: сначала шаг влево, по",
    ms: RESULT_MS,
  },
  {
    ...ZG_S0,
    kind: "aim",
    title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
    text: "Дед (60) стоит на месте и в этом повороте",
    focus: ["p", "x"],
    ms: AIM_MS,
  },
  {
    ...ZG_S1,
    kind: "result",
    title: "Поворот 1: змейка выпрямилась",
    text: "40 заняло место 20 и забрало его к себе ле",
    moved: ["x", "p", "C"],
    ms: RESULT_MS,
  },
  {
    ...ZG_S1,
    kind: "aim",
    title: "Прицел 2: ребро 60 – 40",
    text: "Остался одиночный поворот вокруг деда – и",
    focus: ["g", "x"],
    ms: AIM_MS,
  },
  {
    pos: { x: [180, 50], p: [90, 130], g: [275, 130],

```

```

    </marker>
  </defs>

  { /* призраки: где узел стоял до поворота */
  {ghosts.map((id) => {
    const from = prev[id]!;
    const to = pos[id]!;
    const r = isSubtree(id) ? 15 : 20;
    const dx = to[0] - from[0];
    const dy = to[1] - from[1];
    const len = Math.hypot(dx, dy) || 1;
    const pad = r + 8;
    return (
      <g key={`ghost-${id}`} opacity={0.55}>
        <circle cx={from[0]} cy={from[1]} r={r} fill="none" />
        {len > pad * 2 + 6 && (
          <line
            x1={from[0] + (dx / len) * pad}
            y1={from[1] + (dy / len) * pad}
            x2={to[0] - (dx / len) * pad}
            y2={to[1] - (dy / len) * pad}
            stroke="#a5b4fc"
            strokeWidth={1.5}
            strokeDasharray="4 3"
            markerEnd="url(#splay-arrow)"
          />
        )}
      </g>
    );
  })}

  {edges.map(([a, b], i) => {
    const pa = pos[a];
    const pb = pos[b];
    if (!pa || !pb) return null;
    const hot = Boolean(focus && focus.includes(a));
    return (
      <line

```

```

        {sub ? id : keys[id]}
      </text>
      {!sub && (
        <text textAnchor="middle" y={-r - 7} font
          {id}
        </text>
      )}
    </g>
  );
  }}}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const [caseIdx, setCaseIdx] = useState(0);
  const [frame, setFrame] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [slow, setSlow] = useState(false);

  const active = CASES[caseIdx];
  const total = active.frames.length;
  const idx = Math.min(frame, total - 1);
  const current = active.frames[idx];
  const prevPos = active.frames[Math.max(idx - 1, 0)].p;
  const duration = useMemo(() => Math.round(current.ms - prevPos), [current]);
  const last = idx === total - 1;

  useEffect(() => {
    setFrame(0);
  }, [caseIdx]);

  useEffect(() => {
    if (!playing) return;
    const t = setTimeout(() => setFrame((f) => (f + 1) % total), duration);
    return () => clearTimeout(t);
  }, [playing, frame, duration, total]);

```

```

    </div>
    <p className="text-[13px] text-slate-400 leading-4">
  </div>

  <div className="bg-slate-900 rounded-xl border border-slate-800">
    <Tree frame={current} prev={prevPos} keys={activeKeys}>
      {/* обход слева направо – одинаковый на всех кадрах */}
      <div className="mt-2 pt-2 border-t border-slate-800">
        <div className="flex items-center gap-1.5 flex-wrap">
          <span className="text-slate-500 mr-1">Обход
            {active.inorder.map((t, i) => {
              <span
                key={i}
                className={`px-1.5 py-0.5 rounded font-mono text-sm
                  /\d/.test(t) ? "bg-emerald-500/15 text-emerald-400" : ""
                }`}
              >
                {t}
              </span>
            ))}
          <span className="text-emerald-400/80 ml-1">
        </div>
        <p className="text-[11px] text-slate-500 mt-1">
      </div>
    </div>

    {/* индикатор автопрокрутки */}
    <div className="h-1 w-full bg-slate-800 rounded-full">
      <div
        key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
        className="h-full bg-indigo-500"
        style={
          playing
            ? { animation: `demoProgress ${duration}ms linear 0s 1` }
            : { width: "100%", background: "#334155" }
        }
      </div>
    </div>
  </div>

```

```

      : "bg-slate-800 border-slate-700 text-slate-500"
    }` }
  >
    <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.2"}
  </button>
  <button
    onClick={() => {
      setFrame(0);
      setPlaying(false);
    }}
    className="flex items-center gap-1.5 px-3 py-3 text-sm font-medium transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" /> Сброс
  </button>

  <div className="flex items-center gap-3 ml-auto"
    {[["x", "p", "g"] as const].map((id) =>
      active.keys[id] === undefined ? null : (
        <span key={id} className="flex items-center justify-center"
          <span className="w-3 h-3 rounded-full" style={styles[id]} />
        </span>
      )
    )}
  </div>
</div>

<p className="mt-2 text-[11px] text-slate-500">
  Листай кнопкой «Дальше» в своём темпе – кадры не пропустят
</p>
</div>
);
};

```

```

const leftRotate = (x: SplayNode): SplayNode => {
  const y = x.right;
  if (!y) return x;
  x.right = y.left;
  y.left = x;
  return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
  if (!node) return null;
  return {
    key: node.key,
    left: cloneTree(node.left),
    right: cloneTree(node.right),
    x: node.x,
    y: node.y,
  };
};

export const SplayTreeViz: React.FC = () => {
  // Let's create an elegant initial unbalanced or semi-balanced tree
  // Values: 10, 20, 30, 40, 50, 60
  const createInitialTree = (): SplayNode => {
    let r: SplayNode | null = null;
    const initialKeys = [40, 20, 50, 10, 30, 60];
    initialKeys.forEach((key) => {
      r = insertBST(r, key);
    });
    return r!;
  };

  const [tree, setTree] = useState<SplayNode>(createInitialTree());
  const [inputValue, setInputValue] = useState<string>('');
  const [log, setLog] = useState<string[]>([
    'Дерево инициализировано. Попробуйте кликнуть на узел.',
  ]);
  const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

```

```

    } else {
      steps.push(
        `Поиск ${key}: элемент найден! Запускаем подъем
      );
    }
  }

// Now let's implement standard Splay logic recursively
const splayAction = (
  node: SplayNode | null,
  k: number,
): SplayNode | null => {
  if (!node || node.key === k) return node;

  if (k < node.key) {
    if (!node.left) return node;

    if (k < node.left.key) {
      // Zig-Zig (Left-Left)
      node.left.left = splayAction(node.left.left,
      node = rightRotate(node);
      steps.push(
        `Вращение Zig-Zig (Правый поворот родителя
      );
    } else if (k > node.left.key) {
      // Zig-Zag (Left-Right)
      node.left.right = splayAction(node.left.right
      if (node.left.right) {
        node.left = leftRotate(node.left);
        steps.push(`Компонент Zig-Zag: левый поворо
      }
    }
  }

  if (!node.left) return node;
  steps.push(
    `Финальный поворот Zig (Правое вращение корня
  );
  return rightRotate(node);
} else {

```



```
// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
  e.preventDefault();
  const val = parseInt(inputValue);
  if (isNaN(val)) return;

  // Standard BST insert, then splay!
  let newTree = cloneTree(tree);
  newTree = insertBST(newTree, val);
  const { newRoot, steps } = splayStepByStep(newTree,
  if (newRoot) {
    setTree(newRoot);
  }
  setLog([`Вставили вершину [${val}].`, ...steps]);
  setInputValue("");
  setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
  e.preventDefault();
  const val = parseInt(inputValue);
  if (isNaN(val)) return;
  handleSplay(val);
  setInputValue("");
};

const resetTree = () => {
  setTree(createInitialTree());
  setHighlightedNode(null);
  setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive
const computeCoordinates = (
  node: SplayNode | null,
  x: number,
  y: number,
```

```

        x1={node.x}
        y1={node.y}
        x2={node.right.x}
        y2={node.right.y}
        stroke="#475569"
        strokeWidth="2"
      />,
    );
    lines = lines.concat(renderLines(node.right));
  }
  return lines;
};

const renderNodes = (node: SplayNode | null): React.R
  if (!node) return [];
  let circles: React.ReactNode[] = [];
  const isHighlighted = highlightedNode === node.key;

  circles.push(
    <g
      key={`n-${node.key}`}
      className="cursor-pointer group"
      onClick={() => handleSplay(node.key)}
      data-title={`Кликните, чтобы выполнить Splay(${
    >
    <circle
      cx={node.x}
      cy={node.y}
      r="18"
      fill={isHighlighted ? "#4f46e5" : "#1e293b"}
      stroke={isHighlighted ? "#818cf8" : "#64748b"}
      strokeWidth={isHighlighted ? "3" : "2"}
      className="transition-all duration-300 group-l
    />
    <text
      x={node.x}
      y={node.y}
      dy=".3em"

```

```
</svg>
```

```
{/* Quick interactive controls */}
```

```
<div className="absolute bottom-4 left-4 right-
```

```
  <form onSubmit={handleFind} className="flex
```

```
    <input
```

```
      type="number"
```

```
      placeholder="Поиск..."
```

```
      value={inputValue}
```

```
      onChange={(e) => setInputValue(e.target
```

```
      className="w-24 bg-slate-950 text-white
```

```
00 focus:outline-none"
```

```
  />
```

```
  <button
```

```
    type="submit"
```

```
    onClick={handleFind}
```

```
    className="bg-indigo-600 hover:bg-indigo-
```

```
  >
```

```
    Поиск / Splay
```

```
</button>
```

```
<button
```

```
  type="button"
```

```
  onClick={(e) => {
```

```
    e.preventDefault();
```

```
    const val = parseInt(inputValue);
```

```
    if (!isNaN(val)) {
```

```
      let newTree = cloneTree(tree);
```

```
      newTree = insertBST(newTree, val);
```

```
      const { newRoot, steps } = splaySte
```

```
      if (newRoot) setTree(newRoot);
```

```
      setLog([`Вставили [${val}].`, ...st
```

```
      setInputValue("");
```

```
      setHighlightedNode(val);
```

```
    }
```

```
  }}
```

```
  className="bg-indigo-950 hover:bg-slate
```

```
0"
```

```
>
```

```

        </p>
        <p>
            <span className="font-semibold text-slate-300">
                оба левые или оба правые. Вращаем деда, поворачиваем
            </p>
        </p>
        <p>
            <span className="font-semibold text-slate-300">
                колено. Вращаем узел в разные стороны.
            </p>
        </div>
    </div>
</div>

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
    <h4 className="text-indigo-400 font-bold text-slate-300">
        <HelpCircle className="w-4 h-4 text-indigo-400"/>
        Разбор экзаменационных вопросов (Без нытья):
    </h4>

    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
        <div className="bg-slate-900/60 p-4 rounded-lg">
            <div>
                <p className="font-bold text-slate-300 mb-2">
                    1. Отсутствующий элемент
                </p>
                <p className="text-slate-400 leading-relaxed">
                    Допустим, мы ищем{" "}
                    <code className="text-indigo-400 font-sans">
                        его нет. Алгоритм спустится к листу, на котором
                        неудачей (например, {" "})
                    <code className="text-indigo-400">[20]</code>
                    <code className="text-indigo-400">[30]</code>
                    поднимет в корень{" "}
                    <strong className="text-white">
                        последний успешно просмотренный узел
                    </strong>
                    , на котором он споткнулся! Это оставляе

```

```

        всю структуру вдвое более плоской за счет
        при двойных вращениях {
        <code className="text-emerald-400 font-
        }. Дорогая операция инвестирует в дешевые
        операций. Поэтому амортизированное время
        <strong className="text-white"> $O(\log n)$ 
      </p>
    </div>
    <div className="mt-3 text-[10px] text-emerald-400">
      Каждое "растяжение" компенсируется "сжатием"
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 69/87: `src/components/StackViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

```

```

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color:

```

```

    counter++;

    const newPlate: Plate = {
      id: Date.now().toString(),
      ...preset,
      isDirty: true,
      animState: 'in',
    };

    setDirtyStack(prev => [...prev, newPlate]);
    addLog(`  PUSH: Положили ${newPlate.emoji} сверху с

    setTimeout(() => {
      setDirtyStack(prev => prev.map(p => p.id !== newP
    }, 500);
  }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${top

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id !== topPla
setShowSoap(true);

setTimeout(() => {
  // Step 2: Wash complete. Plate becomes clean and

```

```

    <h3 className="font-black text-blue-400 text-
    <p className="text-xs text-slate-300 mt-1 lea
      Ты складываешь грязные тарелки друг на друга.
      А когда приходит время мыть, ты берёшь именн
      Попытаешься вытащить нижнюю – всё рухнет!
    </p>
  </div>
</div>

```

```

{/* УПРАВЛЕНИЕ */}
<div className="flex items-center gap-3 flex-wrap
  <button
    onClick={pushPlate}
    disabled={isWashing}
    className="flex-1 min-w-[150px] bg-gradient-t
      acity-40 disabled:cursor-not-allowed text-w
      ale-95 transition-all cursor-pointer flex i
  >
    <span> Положить грязную (PUSH)</span>
  </button>

  <button
    onClick={popPlate}
    disabled={isWashing || dirtyStack.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-t
      opacity-40 disabled:cursor-not-allowed text
      ve:scale-95 transition-all cursor-pointer f
  >
    <span> Помыть верхнюю (POP)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800
      r-pointer border border-slate-700"
  >
    Сброс
  </button>

```

```

) : (
  <div className={`flex-1 flex flex-col justify-between`} >

    {/* Указатель TOP */}
    <div
      className="absolute right-8 flex items-center"
      style={{ bottom: `${24 + topIndex * 44}px` }}
    >
      <span className="text-xs font-mono text-slate-400" style="display: inline-block; width: 100px; text-align: right; vertical-align: middle;">
        er gap-1">
        <ArrowUp className="w-3.5 h-3.5" />
        <span>ВЕРШИНА (TOP)</span>
      </span>
      <span className="text-blue-400 animate-pulse" style="display: inline-block; width: 100px; text-align: right; vertical-align: middle;">
        </div>

    {/* Тарелки (отрисовываем в обратном порядке) */}
    {dirtyStack.slice().reverse().map((plate, idx) => {
      const isTop = idx === topIndex;
      return (
        <div
          key={plate.id}
          className={`
            w-full max-w-[280px] h-10 rounded-md
            shadow-md select-none transition-all duration-200
            ${plate.color}
            ${plate.animState === 'in' ? 'animate-in' : ''}
            ${plate.animState === 'washing' ? 'animate-washing' : ''}
            ${isTop ? 'ring-4 ring-blue-500/50' : ''}
          `}
        >
          <span className="text-lg">{plate.name}</span>
          <span className="text-slate-800 text-sm" style="display: inline-block; width: 100px; text-align: right; vertical-align: middle;">
            {isTop && (
              <span className="text-[9px] bg-blue-500/50" style="display: inline-block; width: 100px; text-align: right; vertical-align: middle;">
                LIFO
              </span>
            )}
          </span>
        </div>
      )
    })}
  </div>
)

```



```

        <span className="text-sm"> {plate.n}
        <span className="text-[10px] text-emerald-500"> {plate.n}
      </div>
    )}
  </div>
)}
</div>

  {/* Столешница сушилки */}
  <div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-emerald-400">
  </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4">
  <div className="text-[10px] font-mono text-slate-200">
    {log.map((entry, idx) => (
      <div
        key={idx}
        className={`text-xs font-mono flex items-center justify-between p-2`}
      >
        {idx === 0 ? <span className="text-blue-500">История действий</span> : null}
        <span>{entry}</span>
      </div>
    ))}
  </div>
</div>
);
};

```

ФАЙЛ 70/87: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useRef, useMemo } from "react";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";

```

```

-----
    return { s, steps, patternLength: pattern.length };
}

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$` });

        if (i <= r) {
            steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
            z[i] = Math.min(r - i + 1, z[i - l]);
            steps.push({ i, l, r, z: [...z], desc: `Продолжение. Копипаста сработала.` });
        }

        steps.push({ i, l, r, zTarget: z[i], zSource: i });
        while (i + z[i] < n && s[z[i]] === s[i + z[i]])
            steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i] });
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i], desc: `[i]]}' разные. Стоп.` });
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Успех!` });
    }
}

```

```
// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { set
        return prev + 1;
      });
    }, 1200);
  }
  return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

    <div className="flex items-center gap-2
```

```

    }
    if (step.zTarget === index
        borderColor = "border-a
        bgColor = step.match ===
-900/40");
    }
}

return (
  <div key={index} className=
    {/* L/R markers for Z */}
    {mode === "z" && step.l
      <div className="abs
    }}
    {mode === "z" && step.r
      <div className="abs
    }}

    {/* Index */}
    <span className={`text-

    {/* Char Box */}
    <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
      {char}
    </div>

    {/* Value array box */}
    <div className="text-[1
slate-700/50 font-mono font-bold">
      {mode === "kmp" ? s
    </div>

    {/* KMP fingers */}
    {mode === "kmp" && step
      <div className="abs
z-20">

    <span classNam

```

```

        <div className="absolute top-0 right-0 z-10
        ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_rgba(34,197,94,0.5)]
        })
    </div>
  )
  }
}
</div>
</div>

<div className="flex flex-col sm:flex-row gap-2" style={{
  /* Controls */
  <div className="flex bg-slate-900 border border-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
    <button onClick={prevStep} disabled={isFirstStep} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={playPause} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
      <button onClick={nextStep} disabled={isLastStep} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
        <SkipForward strokeWidth={3} size={16} />
      </button>
      <button onClick={reset} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-100" style={{
        border-slate-800">
        <RefreshCw strokeWidth={3} size={16} />
      </button>
    </div>

    /* Description Log */
    <div className="flex-1 bg-slate-900/50 border border-slate-800 overflow-hidden">
      <div className="absolute top-0 right-0 z-10 text-white font-mono text-sm">
        <div className="text-[10px] text-slate-400">
          {steps.length}</div>
      </div>
    </div>
  }}

```

```
        r = i + z[i] - 1;
    }
}

</pre>
</div>
</div>
);
}
```

ФАЙЛ 71/87: `src/components/TopologicalSortViz.tsx`

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from "react";
import {
  Play,
  Pause,
  RotateCcw,
  Clock,
} from "lucide-react";
```

```
interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}
```

```
interface TEdge {
  u: number;
  v: number;
}
```

```
const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (0" }
```

```

        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
        dfsStack: [...dfsStack],
    });
};

recordStep(
    "Начало топологической сортировки. Используем кла
    null,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
dagEdges.forEach((e) => adj[e.u].push(e.v));

const dfs = (u: number) => {
    visited.add(u);
    dfsStack.push(u);
    recordStep(
        `DFS: зашли в вершину ${u} ("${dagNodes[u].name
        u,
    );

    for (const to of adj[u]) {
        if (!visited.has(to)) {
            dfs(to);
        } else if (!fullyProcessed.has(to)) {
            // Explaining potential cycle detected (not in
            recordStep(
                `⚠ Внимание: Рёбра в серую вершину ${to} о
                u,
            );
        }
    }
}

fullyProcessed.add(u);

```

```

    let timer: NodeJS.Timeout;
    if (isPlaying && currentStepIdx < steps.length - 1)
      timer = setInterval(() => {
        setCurrentStepIdx((prev) => prev + 1);
      }, 1600);
    } else {
      setIsPlaying(false);
    }
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)
    .join(" → ");
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

    <div className="flex items-center gap-2 bg-slate-
      <button
        onClick={togglePlay}
        className="flex items-center gap-2 px-3 py-
          500 text-white"
      >

```



```

        refY="5"
        markerWidth="6"
        markerHeight="6"
        orient="auto-start-reverse"
    >
        <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
    </marker>
</defs>

{/* Edges */}
{dagEdges.map((edge, idx) => {
    const u = dagNodes.find((n) => n.id === e
    const v = dagNodes.find((n) => n.id === e

    const isActive =
        (step.currentNode === edge.u || step.cu
        step.visited.has(edge.v));

    return (
        <line
            key={`edge-${idx}`}
            x1={u.x}
            y1={u.y}
            x2={v.x}
            y2={v.y}
            stroke={isActive ? "#818cf8" : "#334155"}
            strokeWidth={isActive ? "3" : "2"}
            markerEnd={`url(#${isActive ? "dag-ar
            className="transition-all duration-300
        />
    );
}})

{/* Nodes */}
{dagNodes.map((node) => {
    const isCurrent = step.currentNode === no
    const isVisited = step.visited.has(node.i
    const isProcessed = step.fullyProcessed.h

```

```

        fontSize="11px"
        fontWeight="bold"
        className="pointer-events-none"
      >
        ({node.label}) {node.name.split(" "
      </text>
      <text
        x={node.x}
        y={node.y + 12}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="9px"
        className="pointer-events-none"
      >
        {node.name.split(" ").slice(1).join
      </text>
    </g>
  );
  }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800">
    <p className="text-xs text-indigo-300 min-h-12">
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto">
    {/* Topological List output */}
  </div>

```

```

        </span>
        <div className="flex gap-1">
          <button
            disabled={currentStepIdx === 0}
            onClick={() => setCurrentStepIdx((prev)
            className="bg-slate-900 border border-s
t-white"
          >
            Назад
          </button>
          <button
            disabled={currentStepIdx >= steps.length
            onClick={() => setCurrentStepIdx((prev)
            className="bg-slate-900 border border-s
t-white"
          >
            Вперед
          </button>
        </div>
      </div>
    </div>
  </div>
</div>
);
};

```

ФАЙЛ 72/87: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React from "react";
```

```
import { ArticulationPointsViz } from "../ArticulationPo
```

```
import BellmanFordViz from "../BellmanFordViz";
```

```
import BfsViz from "../BfsViz";
```

```
import { DPVisualizer } from "../DPVisualizer";
```

```

        <span>
          Разобрался — проверь себя: ниже то же самое дерево
          не решишь.
        </span>
      </p>
      <SplayRotationSandbox />
    </div>
  );

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
  "stack-dfs": {
    title: "Стек и обход в глубину",
    hint: "Кладите и снимайте тарелки — это ровно то, что нужно",
    Component: StackViz,
  },
  "queue-bfs": {
    title: "Очередь и обход в ширину",
    hint: "Очередь слева, живой BFS по графу справа.",
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
};

```

```

    Component: () => <SplayRotationsPair />,
  },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: () => <GraphDfsBfs /> },
  "top-sort": { title: "Топологическая сортировка", Component: () => <TopSort /> },
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: () => <SccKosaraju /> },
  "graph-articulation": { title: "Точки сочленения", Component: () => <GraphArticulation /> },
  "bridges-code": { title: "Мосты: DFS + tin/low", Component: () => <BridgesCode /> },
  "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: () => <EulerPathVsCycle /> },
  "planarity-euler-formula": { title: "Планарность: K5 и K3,3", Component: () => <PlanarityEulerFormula /> },
  dijkstra: {
    title: "Дейкстра",
    hint: "Жадно забираем ближайшую вершину и релаксируем",
    Component: () => (
      <>
        <ChapterImage vizType="dijkstra" />
        <DijkstraViz />
      </>
    ),
  },
  "bellman-ford": {
    title: "Форд-Беллман",
    Component: () => (
      <>
        <ChapterImage vizType="bellman-ford" />
        <BellmanFordViz />
      </>
    ),
  },
  floyd: {
    title: "Флойд-Уоршелл",
    Component: () => (
      <>
        <ChapterImage vizType="floyd" />
        <FloydViz />
      </>
    ),
  },
  "johnson-algo": { title: "Алгоритм Джонсона", Component: () => <JohnsonAlgo /> },

```

```
label: string; // путь от корня
char: string;
x: number; // Координаты для SVG водопада
y: number;
depth: number;
fail: number;
term_link: number;
is_terminal: boolean;
words: string[];
}

const WATERFALL_NODES: { [key: number]: WNode } = {
  0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0 },
  // Ветка H -> E -> R -> S
  1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1 },
  2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2 },
  3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3 },
  4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4 },
  // Ветка H -> I -> S
  5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2 },
  6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3 },
  // Ветка S -> H -> E
  7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1 },
  8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2 },
  9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3 },
  // Терминальные узлы
  10: { id: 10, label: 'HES', char: 'S', x: 230, y: 460, depth: 4, is_terminal: true },
  11: { id: 11, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4, is_terminal: true },
  12: { id: 12, label: 'HERSE', char: 'E', x: 130, y: 560, depth: 5, is_terminal: true },
  13: { id: 13, label: 'HERSES', char: 'S', x: 80, y: 660, depth: 6, is_terminal: true },
  14: { id: 14, label: 'HERSEHS', char: 'H', x: 110, y: 760, depth: 7, is_terminal: true },
  15: { id: 15, label: 'HERSEHS', char: 'S', x: 110, y: 860, depth: 8, is_terminal: true },
  16: { id: 16, label: 'HERSEHS', char: 'E', x: 110, y: 960, depth: 9, is_terminal: true },
  17: { id: 17, label: 'HERSEHS', char: 'S', x: 110, y: 1060, depth: 10, is_terminal: true },
  18: { id: 18, label: 'HERSEHS', char: 'H', x: 110, y: 1160, depth: 11, is_terminal: true },
  19: { id: 19, label: 'HERSEHS', char: 'S', x: 110, y: 1260, depth: 12, is_terminal: true },
  20: { id: 20, label: 'HERSEHS', char: 'E', x: 110, y: 1360, depth: 13, is_terminal: true },
  21: { id: 21, label: 'HERSEHS', char: 'S', x: 110, y: 1460, depth: 14, is_terminal: true },
  22: { id: 22, label: 'HERSEHS', char: 'H', x: 110, y: 1560, depth: 15, is_terminal: true },
  23: { id: 23, label: 'HERSEHS', char: 'S', x: 110, y: 1660, depth: 16, is_terminal: true },
  24: { id: 24, label: 'HERSEHS', char: 'E', x: 110, y: 1760, depth: 17, is_terminal: true },
  25: { id: 25, label: 'HERSEHS', char: 'S', x: 110, y: 1860, depth: 18, is_terminal: true },
  26: { id: 26, label: 'HERSEHS', char: 'H', x: 110, y: 1960, depth: 19, is_terminal: true },
  27: { id: 27, label: 'HERSEHS', char: 'S', x: 110, y: 2060, depth: 20, is_terminal: true },
  28: { id: 28, label: 'HERSEHS', char: 'E', x: 110, y: 2160, depth: 21, is_terminal: true },
  29: { id: 29, label: 'HERSEHS', char: 'S', x: 110, y: 2260, depth: 22, is_terminal: true },
  30: { id: 30, label: 'HERSEHS', char: 'H', x: 110, y: 2360, depth: 23, is_terminal: true },
  31: { id: 31, label: 'HERSEHS', char: 'S', x: 110, y: 2460, depth: 24, is_terminal: true },
  32: { id: 32, label: 'HERSEHS', char: 'E', x: 110, y: 2560, depth: 25, is_terminal: true },
  33: { id: 33, label: 'HERSEHS', char: 'S', x: 110, y: 2660, depth: 26, is_terminal: true },
  34: { id: 34, label: 'HERSEHS', char: 'H', x: 110, y: 2760, depth: 27, is_terminal: true },
  35: { id: 35, label: 'HERSEHS', char: 'S', x: 110, y: 2860, depth: 28, is_terminal: true },
  36: { id: 36, label: 'HERSEHS', char: 'E', x: 110, y: 2960, depth: 29, is_terminal: true },
  37: { id: 37, label: 'HERSEHS', char: 'S', x: 110, y: 3060, depth: 30, is_terminal: true },
  38: { id: 38, label: 'HERSEHS', char: 'H', x: 110, y: 3160, depth: 31, is_terminal: true },
  39: { id: 39, label: 'HERSEHS', char: 'S', x: 110, y: 3260, depth: 32, is_terminal: true },
  40: { id: 40, label: 'HERSEHS', char: 'E', x: 110, y: 3360, depth: 33, is_terminal: true },
  41: { id: 41, label: 'HERSEHS', char: 'S', x: 110, y: 3460, depth: 34, is_terminal: true },
  42: { id: 42, label: 'HERSEHS', char: 'H', x: 110, y: 3560, depth: 35, is_terminal: true },
  43: { id: 43, label: 'HERSEHS', char: 'S', x: 110, y: 3660, depth: 36, is_terminal: true },
  44: { id: 44, label: 'HERSEHS', char: 'E', x: 110, y: 3760, depth: 37, is_terminal: true },
  45: { id: 45, label: 'HERSEHS', char: 'S', x: 110, y: 3860, depth: 38, is_terminal: true },
  46: { id: 46, label: 'HERSEHS', char: 'H', x: 110, y: 3960, depth: 39, is_terminal: true },
  47: { id: 47, label: 'HERSEHS', char: 'S', x: 110, y: 4060, depth: 40, is_terminal: true },
  48: { id: 48, label: 'HERSEHS', char: 'E', x: 110, y: 4160, depth: 41, is_terminal: true },
  49: { id: 49, label: 'HERSEHS', char: 'S', x: 110, y: 4260, depth: 42, is_terminal: true },
  50: { id: 50, label: 'HERSEHS', char: 'H', x: 110, y: 4360, depth: 43, is_terminal: true },
  51: { id: 51, label: 'HERSEHS', char: 'S', x: 110, y: 4460, depth: 44, is_terminal: true },
  52: { id: 52, label: 'HERSEHS', char: 'E', x: 110, y: 4560, depth: 45, is_terminal: true },
  53: { id: 53, label: 'HERSEHS', char: 'S', x: 110, y: 4660, depth: 46, is_terminal: true },
  54: { id: 54, label: 'HERSEHS', char: 'H', x: 110, y: 4760, depth: 47, is_terminal: true },
  55: { id: 55, label: 'HERSEHS', char: 'S', x: 110, y: 4860, depth: 48, is_terminal: true },
  56: { id: 56, label: 'HERSEHS', char: 'E', x: 110, y: 4960, depth: 49, is_terminal: true },
  57: { id: 57, label: 'HERSEHS', char: 'S', x: 110, y: 5060, depth: 50, is_terminal: true },
  58: { id: 58, label: 'HERSEHS', char: 'H', x: 110, y: 5160, depth: 51, is_terminal: true },
  59: { id: 59, label: 'HERSEHS', char: 'S', x: 110, y: 5260, depth: 52, is_terminal: true },
  60: { id: 60, label: 'HERSEHS', char: 'E', x: 110, y: 5360, depth: 53, is_terminal: true },
  61: { id: 61, label: 'HERSEHS', char: 'S', x: 110, y: 5460, depth: 54, is_terminal: true },
  62: { id: 62, label: 'HERSEHS', char: 'H', x: 110, y: 5560, depth: 55, is_terminal: true },
  63: { id: 63, label: 'HERSEHS', char: 'S', x: 110, y: 5660, depth: 56, is_terminal: true },
  64: { id: 64, label: 'HERSEHS', char: 'E', x: 110, y: 5760, depth: 57, is_terminal: true },
  65: { id: 65, label: 'HERSEHS', char: 'S', x: 110, y: 5860, depth: 58, is_terminal: true },
  66: { id: 66, label: 'HERSEHS', char: 'H', x: 110, y: 5960, depth: 59, is_terminal: true },
  67: { id: 67, label: 'HERSEHS', char: 'S', x: 110, y: 6060, depth: 60, is_terminal: true },
  68: { id: 68, label: 'HERSEHS', char: 'E', x: 110, y: 6160, depth: 61, is_terminal: true },
  69: { id: 69, label: 'HERSEHS', char: 'S', x: 110, y: 6260, depth: 62, is_terminal: true },
  70: { id: 70, label: 'HERSEHS', char: 'H', x: 110, y: 6360, depth: 63, is_terminal: true },
  71: { id: 71, label: 'HERSEHS', char: 'S', x: 110, y: 6460, depth: 64, is_terminal: true },
  72: { id: 72, label: 'HERSEHS', char: 'E', x: 110, y: 6560, depth: 65, is_terminal: true },
  73: { id: 73, label: 'HERSEHS', char: 'S', x: 110, y: 6660, depth: 66, is_terminal: true },
  74: { id: 74, label: 'HERSEHS', char: 'H', x: 110, y: 6760, depth: 67, is_terminal: true },
  75: { id: 75, label: 'HERSEHS', char: 'S', x: 110, y: 6860, depth: 68, is_terminal: true },
  76: { id: 76, label: 'HERSEHS', char: 'E', x: 110, y: 6960, depth: 69, is_terminal: true },
  77: { id: 77, label: 'HERSEHS', char: 'S', x: 110, y: 7060, depth: 70, is_terminal: true },
  78: { id: 78, label: 'HERSEHS', char: 'H', x: 110, y: 7160, depth: 71, is_terminal: true },
  79: { id: 79, label: 'HERSEHS', char: 'S', x: 110, y: 7260, depth: 72, is_terminal: true },
```

```

};

const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен!');
    setActiveSearchCodeLine(4);
    return;
  }

  const char = textToScan[currentIndex];
  let curr = currentNode;
  let logMsg = `[Символ '${char}']`;

  let jumpType: 'normal' | 'fail' | null = null;
  void jumpType;
  let originalCurr = curr;

  if (TRIE_TRANSITIONS[curr][char] !== undefined) {
    const nextNode = TRIE_TRANSITIONS[curr][char];
    logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}).`;
    setJumpPath({ from: curr, to: nextNode, type: 'normal' });
    curr = nextNode;
    setActiveSearchCodeLine(3);
  } else {
    setActiveSearchCodeLine(2);
    let jumps = 0;
    while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
      curr = WATERFALL_NODES[curr].fail;
      jumps++;
    }
    const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
    if (originalCurr === 0) {
      logMsg += `В корне нет перехода по '${char}'. Лосось недоволен!`;
      setJumpPath(null);
    } else {
      logMsg += `Поток по '${char}' у вершины #${originalCurr} не найден!`;
    }
  }
}

```

```

    title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
    desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
        рние вершины сразу инициализируются с fail = RO
    codeLine: 1,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'Ø',
    checkingBranchesFrom: null,
},
{
    targetNodeId: 1,
    title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
    desc: 'Мы обрабатываем первую вершину из очереди -
        просто не существует. fail[H] автоматически нап
    codeLine: 5,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: null,
},
{
    targetNodeId: 7,
    title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
    desc: 'Обрабатываем дочернюю вершину #7 (S) на Dep
        ,
    codeLine: 5,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'S',
    checkingBranchesFrom: null,
},
{
    targetNodeId: 2,
    title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
    desc: 'Переходим на Depth 2 к вершине #2 (HE). Ро
        а-разведчик плывет в ROOT и ищет ветку по "E".
    codeLine: 3,
    scoutPos: 0,

```



```

    {
        targetNodeId: 6,
        title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
        desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
            но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
    {
        targetNodeId: 9,
        title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
        desc: '    МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
            ба плывет в #1 (H) и ищет ветку "E". У вершины :
        codeLine: 4,
        scoutPos: 1,
        inspectedParentFail: 1,
        checkTargetChar: 'E',
        checkingBranchesFrom: 1,
    },
    {
        targetNodeId: 4,
        title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
        desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
            #7 (S). fail[HERS] = #7 (S). Автомат полностью
        codeLine: 4,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'S',
        checkingBranchesFrom: 0,
    },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTra

```

```

        <Play className="w-4 h-4" />
        <span>Режим 2: Поиск в тексте</span>
      </button>
    </div>
  </div>

  { /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */ }
  { activeMode === 'training' ? {
    /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
    <div className="grid grid-cols-1 lg:grid-cols-3" >
      { /* Управление шагами обучения */ }
      <div className="bg-slate-900/90 p-5 rounded-x" >
        <div>
          <h5 className="text-indigo-300 font-bold" >
            <span> </span> Полный обход в ширину (В
          </h5>
          <p className="text-xs text-slate-300 mb-3" >
            Показаны все шаги очереди BFS (начиная с
            ар 8 (SHE → HE)</strong>, где наглядно видно
          </p>
          <div className="space-y-1.5 mb-6 overflow" >
            { TRAINING_STEPS_INFO.map((step, idx) =>
              <button
                key={idx}
                onClick={() => setTrainStep(idx)}
                className={`w-full text-left px-3 p-3
                  ${
                    idx === trainStep
                      ? 'bg-indigo-600 text-white font-medium'
                      : 'bg-slate-800 text-slate-400'
                  }` }
              >
                <span className="line-clamp-1">{step}
                {idx === trainStep && <span className="font-medium" >
                  </button>
                </div>
              </div>
            </div>
          </div>

```

```

        тут</span>}
    </div>
    <div className={`p-1.5 rounded flex item
-l-4 border-indigo-400 text-indigo-200 font
    <span>2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}</span>
    {currentTrainInfo.codeLine === 2 && <
к родителю</span>}
    </div>
    <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
    <span>3. while (v !== root && trie[v]
    {currentTrainInfo.codeLine === 3 && <
т, возврат в корень</span>}
    </div>
    <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
    <span>4. if (trie[v][char] !== undefin
/span>
    {currentTrainInfo.codeLine === 4 && <
ан IF!</span>}
    </div>
    <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
    <span>5. else fail[u] = root; // Bero
    {currentTrainInfo.codeLine === 5 && <
root</span>}
    </div>
    </div>
</div>

<div>
    <h5 className="text-slate-200 font-bold ml
    <Lightbulb className="w-4 h-4 text-ambe
    </h5>
    <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
        {currentTrainInfo.desc}
    </div>
</div>

```

```

        }` }
      >
      {char}
    </span>
  )))
  {currentIndex >= textToScan.length && (
    <span className="bg-emerald-600 text-white" >
      ГOTOBO ✓
    </span>
  )}
</div>

<button
  onClick={stepSearchSimulation}
  disabled={isSearchDone || textToScan.length === 0}
  className={`w-full py-2.5 rounded-lg font-medium
    ${isSearchDone || textToScan.length === 0
      ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
      : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
  }` }
>
  <span>{currentIndex === 0 ? 'Начать план' : 'Продолжить'}
  <ArrowRight className="w-4 h-4" />
</button>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4" >
  <div>
    <h5 className="text-white font-bold mb-3">
      <span>⚡</span> Текущее действие в коде
    </h5>
    <div className="bg-slate-950 p-4 rounded-lg">
      <div className={`p-1.5 rounded flex items-center border border-blue-500 text-blue-300 font-bold`} >
        <span>1. curr = state; char = text[i]
        {activeSearchCodeLine === 1 && <span>

```

```

-hidden shadow-2xl">
  { /* Водопадные фоновые эффекты */ }
  <div className="absolute inset-0 opacity-10 bg-
    o-900 to-transparent pointer-events-none"></div>
  <div className="absolute top-0 left-1/2 -transl
    <div className="w-8 bg-blue-400 h-full animat
    <div className="w-12 bg-blue-300 h-full anima
    <div className="w-6 bg-blue-500 h-full animat
  </div>

  <div className="flex items-center justify-betwe
    <h5 className="text-white font-bold text-base
      <span> </span> Ветвящийся синий водопад (Бо
    </h5>
    <div className="flex flex-wrap gap-4 text-xs"
      <span className="flex items-center gap-1 te
        <span className="w-3 h-0.5 bg-blue-500 in
      </span>
      <span className="flex items-center gap-1 te
        <span className="w-3 h-0.5 border-t borde
      </span>
    </div>
  </div>

  { /* Само SVG дерево водопада */ }
  <div className="w-full overflow-x-auto custom-s
    <svg viewBox="0 0 800 520" className="w-full

    { /* Горизонтальные линии и подписи уровней */ }
    {[
      { depth: 0, y: 60, label: 'Depth 0 (Корень
      { depth: 1, y: 160, label: 'Depth 1 (H, S
      { depth: 2, y: 260, label: 'Depth 2 (HE, I
      { depth: 3, y: 360, label: 'Depth 3 (HER,
      { depth: 4, y: 460, label: 'Depth 4 (HERS
    ]}.map((level) => {
      <g key={`depth-line-${level.depth}`}>
        <line

```

```

        y2={toNode.y}
        stroke={isHighlighted ? '#3b82f6'
        strokeWidth={isHighlighted || isTra
        className={isHighlighted || isTra
    />
    {/* Символ перехода */}
    <circle cx={({fromNode.x + toNode.x)
ined ? (isMatchBranch ? '#10b981' : '#f43f5
    <text
        x={({fromNode.x + toNode.x} / 2}
        y={({fromNode.y + toNode.y} / 2 +
        fill={isTrainingExamined ? (isMat
        fontSize="12"
        fontWeight="bold"
        textAnchor="middle"
    >
        {char}
    </text>

    {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
    {isTrainingExamined && (
        <g transform={`translate(${(fromN
.y) / 2})`}>
            <rect x="-15" y="-12" width="30
strokeWidth="2" />
            <text x="0" y="5" fontSize="14"
                {isMatchBranch ? ' ' : ' '}
            </text>
        </g>
    )}
    </g>
    );
    });
    })}

```

```

{/* Рендер fail-ссылок (пунктирные водопадн
Object.values(WATERFALL_NODES).map((node) :
    if (node.id === 0) return null;

```

```

    /* Рендер вершин (бассейнов водопада) */
    {Object.values(WATERFALL_NODES).map((node) :
      const isCurrent = activeFishPosNode.id === node.id;
      const isTargetChild = activeMode === 'target';
      const hasWords = node.words.length > 0;

      return (
        <g key={`node-${node.id}`} className="node"
          /* Внешнее свечение для активной вершины */
          {isCurrent && (
            <circle cx={node.x} cy={node.y} r={node.radius}
              fill="white" stroke="black" stroke-width={node.strokeWidth}
            )}
          {isTargetChild && (
            <circle cx={node.x} cy={node.y} r={node.radius}
              fill="white" stroke="black" stroke-width={node.strokeWidth}
            )}

          /* Основной круг вершины */
          <circle
            cx={node.x}
            cy={node.y}
            r={isCurrent || isTargetChild ? '24px' : '16px'}
            fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8719d' : 'white'}
            stroke={isCurrent ? 'white' : isTargetChild ? 'black' : 'white'}
            stroke-width={isCurrent || isTargetChild ? 2 : 1}
            className="transition-all duration-300"
          />

          /* Текст внутри вершины */
          <text
            x={node.x}
            y={node.y + 5}
            fill={isCurrent || isTargetChild ? 'white' : 'black'}
            font-size={isCurrent || isTargetChild ? 12 : 10}
            font-weight="bold"
            text-anchor="middle"
          >
            {node.label}
          </text>
        </g>
      );
    });
  }
}

```

```

        { /* Сама рыба */ }
        <text
          x={activeFishPosNode.x + 15}
          y={activeFishPosNode.y - 19}
          fontSize="18"
          textAnchor="middle"
          style={{ transition: 'x 0.8s cubic-bezier(0.25, 0.8, 0.25, 0.75)' }}
          className="animate-float select-none"
        >
          {activeMode === 'training' ? ' ♂ ' : ' ♀ '}
        </text>
      </g>

    </svg>
  </div>
</div>

{ /* Список найденных совпадений (только в режиме поиска) */ }
{activeMode === 'search' && (
  <div className="mt-6 bg-slate-900/50 p-5 rounded" >
    <h5 className="text-white font-bold mb-3 text-sm" >
      <span> </span> Улов лосося (Найденные слова)
    </h5>
    {foundMatches.length === 0 ? (
      <p className="text-sm text-slate-400 italic" >
        Пока ничего не поймали. Запустите шаги симуляции
      </p>
    ) : (
      <div className="flex flex-wrap gap-2" >
        {foundMatches.map((match, idx) => (
          <div
            key={idx}
            className="bg-emerald-950 border border-emerald-500 text-white text-sm font-bold shadow animate-[scaleUp_0.3s_ease-out]"
          >
            <CheckCircle2 className="w-4 h-4 text-emerald-400" />
            <span>{match.word}</span>
            <span className="text-[10px] bg-emerald-900 px-2" >{match.idx}</span>
          </div>
        ))}
      </div>
    )}
  </div>
) }

```



```

    [5, 6, 7, 8],
    [9, 1, 2, 3],
    [4, 5, 6, 7],
  ];

const cellBase =
  "flex items-center justify-center rounded-md font-monospace text-sm" +
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  return (
    <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
      <div className="flex gap-2 mb-4 overflow-x-auto">
        {
          [
            ["1d", "1. Одномерные суммы"],
            ["2d", "2. Двумерные суммы"],
          ] as const
        }.map(([key, label]) => (
          <button
            key={key}
            onClick={() => setMode(key)}
            className={`px-3 py-2 rounded-lg text-xs sm:text-sm
              ${mode === key ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"}
            `>
            {label}
          </button>
        ))
      </div>
      {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
    </div>
  );
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {

```

```

        <div
          key={i}
          onMouseEnter={() => setHover({ kind:
          onMouseLeave={() => setHover(null)}
          onClick={() => setRange({r} => (i < r
          className={` ${cellBase} cursor-pointe
            inCover
              ? "bg-indigo-500 text-white ring-1
              : inRange
                ? "bg-indigo-900/70 text-indigo
                : "bg-slate-800 text-slate-300
            }`}
          title={`a[${i}] = ${v}`}
        >
          {v}
        </div>
      );
    }}}
  </div>

```

```

  { /* массив P */ }
  <div className="flex gap-1.5 items-center">
    <div className="w-[46px] shrink-0 text-right
    {pref.map((v, i) => {
      const active = hover?.kind === "pref" &&
      const isL = i === range.l;
      const isR = i === range.r + 1;
      return (
        <div
          key={i}
          onMouseEnter={() => setHover({ kind:
          onMouseLeave={() => setHover(null)}
          className={` ${cellBase} cursor-help $
            active
              ? "bg-emerald-500 text-white ring
              : isR
                ? "bg-emerald-700 text-white"
                : isL

```

```

        <span className="font-mono text-emerald-300" style="font-size: 0.8em; margin-left: 10px;">
            подвинуть границу запроса.
        </span>
    </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор на любое число: у <b className="text-indigo-400">у</b>
            у <b className="text-indigo-400">а</b> – ку,
        </p>
    )}
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo-500 p-4" style="border-radius: 10px;">
    <div className="text-xs text-slate-400 mb-2">
        Запрос суммы на отрезке (кликайте по массиву .
    </div>
    <p className="font-mono text-sm sm:text-base text-slate-300">
        sum(a[{range.l}..{range.r}]) = P[{range.r + 1}..{range.l}]
        <span className="text-emerald-400">{pref[range.r + 1]}
        <span className="text-rose-400">{pref[range.l]}
        <span className="text-indigo-300 font-bold">{range.r - range.l + 1}
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
        Один вычитание вместо цикла: запрос за O(1) по префикс-сумме
    </p>
</div>
</div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
    const n = A2.length;
    const m = A2[0].length;

    const P = useMemo(() => {
        const p = Array.from({ length: n + 1 }, () => Array.from(
            for (let i = 1; i <= n; i++) {

```

```

        onClick={() =>
          setRect((r) =>
            i < r.r1 || j < r.c1 ? { r1:
              )
            }
          className={` ${cellBase} cursor-po
            inRect ? "bg-indigo-500 text-wh
          }`}
        >
        {v}
      </div>
    );
  }}}
</div>
)}}
</div>
</div>

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
  <div className="text-xs text-slate-400 mb-2">
    <div className="inline-block">
      {P.map((row, i) => {
        <div key={i} className="flex gap-1.5 mb-1">
          {row.map((v, j) => {
            const corner = cornerOf(i, j);
            const isHover = hover?.i === i && hov
            const cornerColor =
              corner === "A"
                ? "bg-emerald-600 text-white"
                : corner === "D"
                  ? "bg-sky-600 text-white"
                  : corner
                    ? "bg-rose-600 text-white"
                    : "bg-slate-900 border border
            return {
              <div
                key={j}

```

```

        <span className="text-rose-400">C</span> + <span
        <span className="text-indigo-300 font-bold">{
    </p>
    <p className="text-[11px] text-slate-500 mt-1">
      Вычли два «хвоста» и вернули дважды вычитенный
    </p>
  </div>
</div>
);
};

```

ФАЙЛ 75/87: src/components/visualizers/SparseTableViz.t
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРӨК: 786 | РАЗМЕР: 1000

```

import React, { useState, useEffect, useMemo } from "react";
import { motion, AnimatePresence } from "motion/react";
import { useVizSync } from "../../hooks/useVizSync";
import { useVizControl } from "../../hooks/useVizControl";
import { VizObject } from "../../viz/VizObject";
import {
  Play,
  RotateCcw,
  ChevronRight,
  ChevronLeft,
  Lock,
  ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => [0]);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];

```



```

// подсветка в визуализации = строка в компиляторе; o
useVizSync("sparse-table", { n: N, m: LOG, i: cur?.i
  ur.i][cur.j] : ext ? st1D[ext.i]?.[ext.j] ?? st1D[0
useVizControl("sparse-table", {
  onStep: (line) => {
    if (typeof line === "number") setStep(Math.min(li
    else setStep((s) => Math.min(s + 1, maxStep));
  },
  onReset: () => { setStep(0); setExt(null); setHover
  onPrev: () => setStep((s) => Math.max(0, s - 1)),
});
// слушает компилятор: i, j = 0, 0 → подсветка ячейки
useEffect(() => {
  const h = (e: Event) => {
    const d = (e as CustomEvent).detail;
    if (d.chapterId && d.chapterId !== "sparse-table"
    const vars = d.vars as Record<string, any>;
    if (!vars) return;
    if (vars.i !== undefined && vars.j !== undefined)
      const i = Number(vars.i), j = Number(vars.j);
      const idx = computeSteps.findIndex((s) => s.i =
      if (idx !== -1) { setStep(idx + 1); setHover({
      else { setHover({ i, j }); setExt({ i, j }); }
    } else if (vars.a !== undefined) {
      // 2D строка a,b,c,d – подсветим блок 2x2 в 1D
      setHover({ i: 0, j: 1 }); setExt({ i: 0, j: 1 }
    }
  };
  window.addEventListener("viz:highlight", h as Event
  return () => window.removeEventListener("viz:highli
}, []);

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-
          Построение Sparse Table (Min)

```

```

        w,44px)) h-[clamp(26px,8vw,44px)] text-[clamp(16px,8vw,26px)]
        inCover ? "bg-indigo-500 text-white" : "bg-white text-indigo-500"
      }` }
    >
      {v}
    </div>
    <span className="text-[9px] text-slate-100" v={v}>
  </div>
);
}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
  {hover && covered ? (
    <p className="text-slate-300">
      <b className="text-sky-400">
        ST[{hover.i}][{hover.j}] = {stID[hover.i][{hover.j}]}
      </b>{" "}
      – это минимум на отрезке{" "}
      <span className="font-mono text-sky-300">
        [{covered.from}, {covered.to}]
      </span>{" "}
      длины  $2^{\{hover.j\}}$  = {1 << hover.j}:{ " "}
      <span className="font-mono text-slate-400">
        min({arrID.slice(covered.from, covered.to)})
      </span>
    </p>
  ) : (
    <p className="text-slate-500">
      Наведите курсор (или тапните) на любое число
    </p>
  )}
</div>
</div>

<div className="overflow-x-auto pb-2">
  <div className="inline-block min-w-full bg-slate-100">
    <div className="grid grid-cols-[auto_repeat(4,1fr)]">
      { /* Headers */}
    </div>
  </div>
</div>

```



```

        }}
      </div>
    );
  }}}
</React.Fragment>
  }}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-
  {step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-l
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bo
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{st1
            <span className="text-amber-400">
              {st1D[c.i + half][c.j - 1]}
            </span>
          ) ={" "}
            <span className="text-indigo-400 font-b
              {st1D[c.i][c.j]}
            </span>

```

```

onReset: () => { setK(0); setHover(null); setLocked
});
useEffect(() => {
  const h = (e: Event) => {
    const d = (e as CustomEvent).detail;
    if (d.chapterId && d.chapterId !== "sparse-table")
    const vars = d.vars as Record<string, any>;
    if (!vars) return;
    if (vars.a !== undefined && vars.b !== undefined &&
      // 4→1 из компилятора: подсветить 2x2 блок
      setK(1);
      setLocked({ r: 0, c: 0 });
      setHover({ r: 0, c: 0 });
    } else if (vars.i !== undefined && vars.j !== undefined &&
      const r = Number(vars.i), c = Number(vars.j);
      setLocked({ r: Math.min(6, r), c: Math.min(6, c);
      setHover({ r: Math.min(6, r), c: Math.min(6, c);
    }
  };
  window.addEventListener("viz:highlight", h as EventListener);
  return () => window.removeEventListener("viz:highlight", h);
}, []);

return (
  <div className="flex flex-col xl:flex-row gap-6">
    <div className="flex-1 min-w-0 bg-slate-900 p-3 scroll-smooth">
      <h3 className="text-xl font-bold text-indigo-400">
        <p className="text-sm text-slate-400 mb-6 text-balance">
          Для наглядности показываем только <b className="text-rose-300">
            <br/>
            <span className="text-rose-300 animate-pulse">
              иц!</span>
            </p>

      <div className="flex bg-slate-950 p-1 rounded-lg">
        {[0, 1, 2, 3].map(step => (
          <button
            key={step}

```

```

const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

    if (isActive) {
        if (isCurr) {
            highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
            zIndex = 'z-10';
        } else {
            const prevL = 1 << (k - 1);
            const isTop = r < activeCel
            const isLeft = c < activeCe

            if (isTop && isLeft) {
                highlightClass = 'bg-ros
md:scale-[1.08]';
            } else if (isTop && !isLeft)
                highlightClass = 'bg-blur
] md:scale-[1.08]';
            } else if (!isTop && isLeft)
                highlightClass = 'bg-eme
9,0.3)] md:scale-[1.08]';
            } else {
                highlightClass = 'bg-amb
3)] md:scale-[1.08]';
            }
            zIndex = 'z-10';
        }
    }

return (
    <div
        key={idx}
        onMouseEnter={() => { if(is
        onMouseLeave={() => { if(is
        onClick={() => {
            if(isCurr) {
                if(locked && locked.

```

```

ate-300 shadow-inner overflow-x-auto">
  <span className="text-slate-500">// Те
  <span className="text-purple-400">int<
="text-amber-300">1</span>);<br/><br/>
  <span className="text-slate-500">// Кв
  <span className="text-indigo-400">ST</
  &nbsp;&nbsp;&nbsp;<span className="text-rose
e="text-amber-300">1</span>],      <sp
  &nbsp;&nbsp;&nbsp;<span className="text-blue
ext-amber-300">1</span>][k-<span className=
pan><br/>
  &nbsp;&nbsp;&nbsp;<span className="text-emer
="text-amber-300">1</span>][k-<span classNa
span><br/>
  &nbsp;&nbsp;&nbsp;<span className="text-ambe
white">c + pL</span>][k-<span className="te
ame="text-slate-500">// Правый Низ</span><b
  {'}'}));
</div>

{activeCell ? {
  <div className="bg-slate-950 p-5 round
.2)] mt-2 transition-all">
    <p className="text-slate-300 mb-3 t
      <span>Пример для ячейки:</span>
      {locked && <span className="text
ded border border-rose-500/20"><Lock size={
    </p>
    <p className="text-indigo-300 borde
      <span className="font-bold text-w
    }][{k}][{k}]</span>
      <span className="text-slate-400">
    </p>

    <div className="space-y-2 pt-3 pl-2
      {{{}} => {
        const prevL = 1 << (k - 1);
        return {

```

```

        <div className="bg-slate-950/40 border
            mt-2 flex items-center justify-center anim
                Наведите курсор на матрицу {L}x{L}
            </div>
        </div>
    </div>
  </div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
  const [r1, setR1] = useState(1);
  const [c1, setC1] = useState(1);
  const [r2, setR2] = useState(5);
  const [c2, setC2] = useState(6);

  const H = r2 - r1 + 1;
  const W = c2 - c1 + 1;
  const kx = Math.floor(Math.log2(H));
  const ky = Math.floor(Math.log2(W));
  const Lx = 1 << kx;
  const Ly = 1 << ky;

  const matRows = 8 - Lx + 1;
  const matCols = 8 - Ly + 1;

  return (
    <div className="flex flex-col xl:flex-row gap-6
      <div className="flex-1 min-w-0 bg-slate-900
        <h3 className="text-xl font-bold text-ro
          <p className="text-slate-400 text-[13px]
            еpx, r2, c2 - правый низ).</p>
          <div className="flex flex-col items-cent
            <div className="grid grid-cols-[repea

```

```

    <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(${(Ly / 8) * 100
</div>

```

```

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
    <label className="flex items-cen
    <span className="text-slate-
    <div className="flex gap-2">
    <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
    <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
    </div>
    </label>
    <label className="flex items-cen
    <span className="text-slate-
    <div className="flex gap-2">
    <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
    <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
    </div>
    </label>
    </div>
  </div>
</div>

```

```

<div className="flex-1 bg-slate-900 p-6 rou
  <h3 className="text-xl font-bold text-em
  <p className="text-sm font-sans text-sla
    Мы знаем, что максимальная степень дв
rounded text-white">{Lx}</span>. Аналогично
te">{Ly}</span>.
  </p>
  <p className="text-sm font-sans text-sla
    Чтобы покрыть весь прямоугольник, мы

```

```

        {'}'}));
    </div>

    <div className="w-full flex justify-center"
      <div className="flex flex-col items-center"
        <h4 className="text-slate-400 font-bold"
          Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][kx]][ky]
        </h4>
        <div className="grid gap-[2px] p-2.5"
Columns: `repeat(${matCols}, 1fr)`>}}>
      {Array.from({length: matRows * matCols}, (v, idx) => {
        const r = Math.floor(idx / matCols);
        const c = idx % matCols;
        let isTL = r === r1 && c === c1;
        let isTR = r === r1 && c === c2;
        let isBL = r === r2 - Lx + 1 && c === c1;
        let isBR = r === r2 - Lx + 1 && c === c2;

        let color = "bg-slate-800 text-white";
        let txt = ST2D[r][c][kx][ky];

        let badge = null;
        if (isTL) { color = "bg-rose-500 text-white";
0 border-2"; badge = "TL"; }
        else if (isTR) { color = "bg-rose-500 text-white";
ue-300 border-2"; badge = "TR"; }
        else if (isBL) { color = "bg-emerald-500 text-white";
-emerald-300 border-2"; badge = "BL"; }
        else if (isBR) { color = "bg-emerald-500 text-white";
mber-300 border-2"; badge = "BR"; }

        return (
          <div key={idx} className={color}
tion-all duration-300 relative shrink-0 ${c}
            {txt}
            {badge && <div className="border-2 border-slate-700 text-white px-1 font-bold rounded

```



```

        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').cla
            ? 'px-3 py-1 rounded text-sm font-bold l
            : 'px-3 py-1 rounded text-sm font-bold

        document.getElementById('btn-mode-wtf').cla
            ? 'px-3 py-1 rounded text-sm font-bold l
            : 'px-3 py-1 rounded text-sm font-bold

        const aside = document.querySelector('aside')
        const headerMain = document.querySelector('h1')
        const questionsContainer = document.getElementById('questions')
        const wtfContainer = document.getElementById('wtf')

        if (mode === 'wtf') {
            document.body.classList.add('wtf-mode')
            if (aside) aside.style.display = 'none'
            if (headerMain) headerMain.style.display = 'none'
            if (questionsContainer) questionsContainer.style.display = 'none'
            if (wtfContainer) wtfContainer.classList.remove('hidden')

            // Remove the URL hash to avoid scrolling
            history.pushState("", document.title, window.location.pathname + window.location.search)
            window.scrollTo(0, 0);
        } else {
            document.body.classList.remove('wtf-mode')
            if (aside) aside.style.display = '';
            if (headerMain) headerMain.style.display = '';
            if (questionsContainer) questionsContainer.style.display = '';
            if (wtfContainer) wtfContainer.classList.add('hidden')
        }
    }

```

```

function toggleDrawer() {
  const isClosed = mobileDrawer.classList.contains('closed')
  if (isClosed) {
    mobileDrawer.classList.remove('-translateX-100%')
    if (drawerOverlay) {
      drawerOverlay.classList.remove('opacity-0')
      // Timeout allows display:block to be applied
      setTimeout(() => drawerOverlay.classList.remove('opacity-0'), 0)
    }
    document.body.style.overflow = "hidden"
  } else {
    mobileDrawer.classList.add('-translateX-100%')
    if (drawerOverlay) {
      drawerOverlay.classList.add('opacity-100')
      setTimeout(() => drawerOverlay.classList.add('opacity-100'), 0)
    }
    document.body.style.overflow = "";
  }
}

if (mobileMenuBtn && mobileDrawer) {
  mobileMenuBtn.addEventListener("click", () => toggleDrawer())
}
if (closeDrawerBtn && mobileDrawer) {
  closeDrawerBtn.addEventListener("click", () => toggleDrawer())
}
if (drawerOverlay) {
  drawerOverlay.addEventListener("click", () => toggleDrawer())
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
  link.addEventListener("click", () => {
    if (mobileDrawer && !mobileDrawer.classList.contains('closed')) {
      toggleDrawer();
    }
  })
});
});
});

```

```
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']]
        displayMath: [['$$', '$$'], ['\\[', '\\
      }
    };
  </script>
  <style>
    @import url('https://fonts.googleapis.com/css2?
    @reference "./src/index.css";
    .math-font { font-family: 'Fira Code', monospac
    .uno-card {
      width: 70px; height: 105px; border-radius:
      box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
      align-items: center; justify-content: cente
      color: white; overflow: hidden; flex-shrink
      text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
      font-family: 'Arial Black', Impact, sans-se
    }
    .uno-card::before, .uno-card::after {
      content: attr(data-val); position: absolute
      text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
    }
    .uno-card::before { top: 2px; left: 4px; }
    .uno-card::after { bottom: 2px; right: 4px; tra
    .uno-card.mini { width: 45px !important; height
    .uno-card.mini::before, .uno-card.mini::after {

  /* Global formula scroll settings */
```

```

        .uno-equation .text-2xl { font-size: 1.2rem }
        .uno-equation.gap-4 { gap: 0.5rem !important }

        /* Allow horizontal scroll for equations in
        .formula-scroll { flex-wrap: nowrap !important;
        px; }
    }
    .uno-inner {
        position: absolute; width: 110%; height: 75%;
        border-radius: 50%; transform: rotate(-30deg);
        box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
    }
    .uno-val { position: relative; z-index: 2; }
    .card-red { background-color: #e52521; } .card-
    .card-blue { background-color: #004dcf; } .card-
    .card-green { background-color: #339e35; } .card-
    .card-yellow { background-color: #ffc400; } .ca
    .card-black { background-color: #222; } .card-b
    .card-black::before, .card-black::after { text-
    .info-block { @apply bg-slate-800 border-l-4
    .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
    ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
    king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
    r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
    background-color: #f8fafc !important;
    color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col

```

```

        animation: creepy-blink 4s infinite;
    }
    .creepy-eye-alt {
        transform-origin: center;
        transform-box: fill-box;
        animation: creepy-blink 5.2s infinite;
        animation-delay: 1.5s;
    }
    .creepy-emoji {
        display: inline-block;
        transform-origin: center;
        animation: creepy-blink 3.8s infinite;
    }
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

    <div class="sticky top-0 z-50 w-full bg-slate-950/90
        center items-center gap-4 sm:gap-8 transition-colors">
        <div class="flex items-center gap-3">
            <span class="text-sm font-bold uppercase text-slate-200">Тема</span>
            <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
                <button onclick="setMode('normal')" id="btn-normal"
                    transition-colors">Список тем</button>
                <button onclick="setMode('wtf')" id="btn-wtf"
                    transition-colors">WTF</button>
            </div>
        </div>
        <div class="flex items-center gap-3">
            <span class="text-sm font-bold uppercase text-slate-200">Тема</span>
            <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
                <button onclick="setTheme('dark')" id="btn-dark"
                    transition-colors">Темная</button>
                <button onclick="setTheme('light')" id="btn-light"
                    transition-colors">Светлая</button>
                <button onclick="setTheme('notebook')" id="btn-notebook"
                    transition-colors">Тетрадь</button>
            </div>
        </div>
    </div>

```

```

        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-rem">
        <rect x="0" y="0" width="40" height="10"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <!-- МЯСОРУБКА -->
    <g id="meat-grinder">
        <path d="М 10 40 L 15 20 Q 25 15 35 20
        <rect x="5" y="40" width="40" height="20"
        <!-- Ручка -->
        <path d="М 45 50 Q 60 50 60 30 L 75 30"
        <circle cx="75" cy="30" r="4" fill="#333"
        <!-- Выход -->
        <path d="М 5 45 L -5 45 L -5 55 L 5 55
        <circle cx="25" cy="50" r="15" fill="#f
    </g>
    <g id="lego-green">
        <rect x="0" y="10" width="20" height="10"
        <rect x="3" y="5" width="14" height="5"
    </g>
    <g id="lego-white">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="0" y1="2" x2="40" y2="2" stro
        <text x="20" y="14" fill="#475569" font
    </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
    0/50 z-50 hover:bg-blue-500 transition-transform
    <svg xmlns="http://www.w3.org/2000/svg" class="

```

```
        <a href="#section-1-3" class="b
        <a href="#section-1-4" class="b
</div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-2" class=
er-indigo-500 pl-3 font-bold text-indigo-300">
        2. Комплексные числа
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#section-2-1" class="b
        <a href="#section-2-2" class="b
        <a href="#section-2-3" class="b
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-3" class=
er-teal-500 pl-3 font-bold text-teal-300">
        3. Тригонометрическая форма
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#section-3-1" class="b
        <a href="#section-3-2" class="b
        <a href="#section-3-3" class="b
        <a href="#section-3-4" class="b
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-4" class=
er-amber-500 pl-3 font-bold text-amber-300">
```

```

        <a href="#q6-3" class="block ho
        <a href="#q6-4" class="block ho
        <a href="#q6-5" class="block ho
        <a href="#q6-6" class="block ho
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
            7. Взаимно простые числ
        </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q7-1" class="bloc
        <a href="#q7-2" class="bloc
        <a href="#q7-3" class="bloc
        <a href="#q7-4" class="bloc
        <a href="#q7-5" class="bloc
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-8" class=
er-violet-500 pl-3 font-bold text-violet-300">
            8. Сравнимость и Поля В
        </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q8-1" class="bloc
        <a href="#q8-2" class="bloc
        <a href="#q8-3" class="bloc
        <a href="#q8-4" class="bloc
        <a href="#q8-5" class="bloc
        <a href="#q8-6" class="bloc
        <a href="#q8-7" class="bloc
```



```
der-cyan-500 pl-3 font-bold text-cyan-400">
    11. НОД и НОК многочлен
    </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q11-1" class="blo
        <a href="#q11-2" class="blo
нственности НОД</a>
        <a href="#q11-3" class="blo
ПИРАЛЬ)</a>
        <a href="#q11-4" class="blo
вращается!)</a>
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-12" clas
der-blue-500 pl-3 font-bold text-blue-400">
        12. Взаимно простые мно
        </a>
    </summary>
    <div class="pl-6 space-y-2 text
        <a href="#q12-1" class="blo
        <a href="#q12-2" class="blo
        <a href="#q12-3" class="blo
еление)</a>
    </div>
</details>

<details class="group">
    <summary class="list-none curso
        <a href="#question-13" clas
der-emerald-500 pl-3 font-bold text-emerald
        13. Корни многочленов.
        </a>
    </summary>
    <div class="pl-6 space-y-2 text
```

```

        <details class="group">
            <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
                16. Неприводимые многоч.
            </a>
            </summary>
            <div class="pl-6 mt-2 space-y-2">
                <a href="#q16-1" class="block">
                <a href="#q16-2" class="block">
                <a href="#q16-3" class="block">
            </div>
        </details>

        <div class="mt-4 mb-2 text-xs font-l">
            <a href="#question-17" class="block text-slate-500 pl-3 text-slate-400">
                <span class="font-bold text-teal">
            </a>
            <a href="#question-18" class="block text-slate-500 pl-3 text-slate-400">
                <span class="font-bold text-blue">
            </a>
            <a href="#question-19" class="block text-slate-500 pl-3 text-slate-400">
                <span class="font-bold text-emerald">
            </a>
            <a href="#question-20" class="block text-slate-500 pl-3 text-slate-400">
                <span class="font-bold text-rose">
            </a>
            <a href="#question-21" class="block text-slate-500 pl-3 text-slate-400">
                <span class="font-bold text-indigo">
            </a>
            <a href="#question-22" class="block text-slate-500 pl-3 text-slate-400">
                <span class="font-bold text-fuchsia">

```

```

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting) {
                visibleId = entry.target.id;

                // Highlight active
                document.querySelector(
                    `a[href="#${visibleId}"]`
                ).style.fontWeight = 'bold';
            }
        });

        // Open the parent
        const correspondingLi = document.querySelector(
            `li[aria-labelledby="#${visibleId}"]`
        );
        if (correspondingLi) {
            const details = correspondingLi.querySelector(
                'div[role="details"]'
            );
            if (details && !details.open) {
                details.open = true;
                document.querySelector(
                    `a[href="#${visibleId}"]`
                ).style.outline = '2px solid #000';
            }
        }
    });
}

});
}, {
    root: null,
    rootMargin: '-10% 0px -70% 0px',
    threshold: [0, 0.5]
});

sections.forEach(section => {
    observer.observe(section);
});
});

```

```

textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

```

```

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

```

```

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#333';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
  if (navigator.clipboard) {
    navigator.clipboard.writeText(textarea.value);
    copyBtn.innerText = 'Скопировано';
  } else {
    textarea.select();
    document.execCommand('copy');
    copyBtn.innerText = 'Скопировано';
  }
  setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';

```

```

        fallbackModal(text, 'Markdown')
        return;
    }
    const blob = new Blob([text], {
    const url = URL.createObjectURL
    const link = document.createElement('a');
    link.href = url;
    link.download = 'vyisshaya_algebra.pdf';
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
}

function setTheme(theme) {

    const body = document.body;
    body.classList.remove('bg-slate-500');

    document.getElementById('theme-dark').checked = theme === 'dark';
    document.getElementById('theme-light').checked = theme === 'light';
    document.getElementById('theme-auto').checked = theme === 'auto';

    let oldStyle = document.getElementById('theme-style');
    if (oldStyle) oldStyle.remove();
    const style = document.createElement('style');
    style.id = 'theme-style';

    if (theme === 'dark') {
        body.classList.add('bg-slate-500');
        document.getElementById('theme-dark').checked = true;
    } else if (theme === 'light') {
        body.classList.add('bg-white');
        document.getElementById('theme-light').checked = true;
        style.innerHTML = `
            body.bg-white section header {
            body.bg-white main { color: #000; }
        `;
    }
}

```

```

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`;
} else if (theme === 'terminal')
  body.classList.add('bg-black')
  document.getElementById('theme')
  style.innerHTML = `
    body.bg-black .bg-slate-950 {
      background-color: #2e3436;
    }
    body.bg-black section header,
    body.bg-black button {
      color: #4ade80 !important;
      border-color: #22c55e !important;
      font-family: monospace;
    }
    body.bg-black svg path,
    stroke: #22c55e !important;
    fill: transparent !important;
  `;
}

document.head.appendChild(style)
</script>

```

```

        терминала"> </button>
      </div>
    </div>
  </div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16
-base min-w-0">
  <header class="text-center mb-12 border-b border-slate-700">
    <h1 class="text-4xl md:text-5xl font-bl
4">
      ВЫСШАЯ АЛГЕБРА
    </h1>
    <p class="text-slate-400 italic text-lg">
      Высшая алгебра: теория и практика

  <!-- Print Table of Contents -->
  <div class="hidden print:block mt-8 text-center">
    <h2 class="font-bold text-xl mb-4">Содержание
    <ul class="text-base space-y-2">
      <li><a href="#question-1">1. Алгебра
      <li><a href="#question-2">2. Коммутативная алгебра
      <li><a href="#question-3">3. Топология
      <li><a href="#question-4">4. Статистика
      <li><a href="#question-5">5. Группы
      <li><a href="#question-6">6. Дифференциальная геометрия
      <li><a href="#question-7">7. Алгебраические структуры
      <li><a href="#question-8">8. Квантовая механика
      <li><a href="#question-9">9. Комбинаторика
      <li><a href="#question-10">10. Теория чисел
      <li><a href="#question-11">11. Теория вероятностей
      <li><a href="#question-12">12. Теория игр
      <li><a href="#question-13">13. Теория информации
      <li><a href="#question-14">14. Теория оптимизации
      <li><a href="#question-15">15. Теория управления

```

```

        <div class="font-bold text-slate-500">
    </a>

    <!-- Row 2: Базовые операции / Евклид / НОД и НОК
    <a href="#question-6" onclick="setMainQuestion(6)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-yellow-500">
            <div class="font-bold text-slate-500">
    </a>
        <a href="#question-10" onclick="setMainQuestion(10)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200
            <div class="text-xs text-orange-500">
                <div class="font-bold text-slate-500">
    </a>
        <a href="#question-3" onclick="setMainQuestion(3)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors duration-200
            <div class="text-xs text-cyan-500">
                <div class="font-bold text-slate-500">
    </a>

    <!-- Row 3: НОД и НОК / Алгоритм Евклида
    <a href="#question-7" onclick="setMainQuestion(7)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-yellow-500">
            <div class="font-bold text-slate-500">
    </a>
        <a href="#question-12" onclick="setMainQuestion(12)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200
            <div class="text-xs text-orange-500">
                <div class="font-bold text-slate-500">
    </a>
    <!-- Empty 3rd row cell -->
    <div class="hidden md:block"></div>

    <!-- Row 4: Факторизация / Структурная теория
    <a href="#question-8" onclick="setMainQuestion(8)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-yellow-500">

```



```
-l-4 border-orange-500 hover:bg-slate-700 t
    <div class="text-xs text-orange
    <div class="font-bold text-slate
</a>
<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

    <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
    <div class="text-xs text-teal-5
    <div class="font-bold text-slate
</a>
    <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
    <div class="text-xs text-blue-5
    <div class="font-bold text-slate
</a>
    <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
    <div class="text-xs text-emeral
    <div class="font-bold text-slate
</a>

    <a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
    <div class="text-xs text-rose-5
    <div class="font-bold text-slate
</a>
    <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
    <div class="text-xs text-indigo
    <div class="font-bold text-slate
</a>
    <a href="#question-22" onclick="setl
-l-4 border-fuchsia-500 hover:bg-slate-700
```

```

<!-- Прямое -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
  <h3 class="text-xl text-blue-400">Прямое
  <p class="text-sm text-slate-400">Прямое
  <ul class="list-disc pl-5 sp-2">
    <li><span class="text-white">
      k$).</li>
    <li><span class="text-white">
      ово.</li>
  </ul>
</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
  <h3 class="text-xl text-fuchsia-400">От противного
  <p class="text-sm text-slate-400">От противного
  <ul class="list-disc pl-5 sp-2">
    <li><span class="text-white">
      i>
    <li><span class="text-white">
      т.д.).</li>
    <li><span class="text-white">
      что так и задумано: "Ой, противоречие, теорема не верна".
  </ul>
</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
  <h3 class="text-xl text-emerald-400">Единственность
  <p class="text-sm text-slate-400">Единственность
  <ul class="list-disc pl-5 sp-2">
    <li><span class="text-white">
      , r_2$).</li>
    <li><span class="text-white">
    <li><span class="text-white">
      счет ограничений (размер остатка) докажи, что
  </ul>

```

```
<p class="text-xs font-  
со следующим. Если в конце 0 – пациент мерт  
</div>
```

```
<div class="bg-slate-800 p-  
<h4 class="text-red-400  
<p class="text-sm text-  
<p class="text-xs font-  
$c$. Пока получается 0 – корень жив. Как то  
корня минус один.</p>  
</div>
```

```
<div class="bg-slate-800 p-  
<h4 class="text-red-400  
<p class="text-sm text-  
<p class="text-xs font-  
ициентов (они делятся на $p$), кроме старше  
зложить.</p>  
</div>
```

```
<div class="bg-slate-800 p-  
<h4 class="text-red-400  
<p class="text-sm text-  
<p class="text-xs font-  
. Чаще всего после этого Фейс-контроль $p$  
</div>  
</div>
```

```
<h2 class="text-2xl font-bold b  
(Правила экстрасенса)</h2>  
<p class="text-sm text-slate-40  
тношения), а ты их не помнишь. Юзай базовые  
нейность) для делимости.</p>
```

```
<ul class="space-y-4 pb-10">  
<li class="bg-slate-800/50 p-  
<span class="text-2xl">
```

```
        </section>
    </div>
```

```
    <div id="questions-container">
```

```
    <!-- ===== ВОПРОС 1 =====
```

РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 78/87: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

ФАЙЛ 79/87: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
```

```
/**
```

```
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
```

```
 * Файл получается автономным: стили защиты внутрь, интер
```

```
-----
.export-gloss { margin-top: 3rem; border-top: 1px solid #0f172a; }
.export-gloss h2 { color: #fff; font-size: 1.25rem; font-weight: bold; }
.export-term { background: #0f172a; border: 1px solid #0f172a; padding: 10px; }
.export-term h3 { color: #c7d2fe; font-size: .95rem; font-weight: bold; }
.export-term p { margin: 0 0 .3rem; font-size: .85rem; }
.export-term .formal { color: #94a3b8; font-size: .8rem; font-weight: bold; }
.export-term .cx { color: #6ee7b7; font-size: .78rem; font-weight: bold; }
.export-foot { margin-top: 3rem; border-top: 1px solid #0f172a; }
@media print {
  body { background: #fff; color: #0f172a; }
  .export-note, details { break-inside: avoid; }
  details { display: block; }
  details > div, details > p { display: block !important; }
  pre { white-space: pre-wrap; }
};
```

```
const escapeHtml = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
```

```
/**
```

```
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
```

```
function prepareBody(html: string): { body: string; terms: string[] } {
  const host = document.createElement("div");
  host.innerHTML = html;
```

```
  try {
    annotateTerms(host);
  } catch {
    /* если браузер не осилил регулярку – выгружаем текст как есть */
  }

  return { body: host.innerHTML, terms: terms };
}
```

```
const ids: string[] = [];
host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
  const id = el.dataset.termId;
  if (!id) return;
  ids.push(id);
});
```

```
-----
function wrap(opts: { title: string; subtitle: string;
  const stamp = new Date().toLocaleString("ru-RU");
  return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>
<style>${EXTRA_CSS}</style>
</head>
<body>
<div class="export-shell">
  <header class="export-head">
    <div style="font-size:.7rem;text-transform:uppercase
    <h1 style="color:#fff;font-size:1.6rem;font-weight:
    <div style="font-size:.72rem;color:#475569;margin-t
  </header>

  <div class="export-note">
    Это автономная копия: стили внутри файла, интернет
    Интерактивные тренажёры и анимации в статичный HTML
    они живут в онлайн-версии: <a href="${escapeHtml(op
  </div>

  <article class="prose prose-invert max-w-none">
${opts.body}
  </article>

${opts.gloss}

  <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}
```

```
function triggerDownload(html: string, filename: string
```

```

const toc = chapters
  .map((c) => `- 

```

```
const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]),
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  };
});
```

```
const has2D = (r) => r.interactive || r.inlineSvg || r.image;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);
```

```
const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));
```

```
const flag = (b) => (b ? " " : "-");
```

```
if (process.argv.includes("--md")) {
  console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
  console.log("| --- | --- | --- | --- | --- |");
  for (const r of rows) {
    console.log(
      `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} |`
    );
  }
} else {
  console.log(`Глав в пособии: ${rows.length}\\n`);
}
```


Универсальное пособие • полный исходный код

```
-----
const esc = (s) => s.replace(/[\.\*+?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
    "giu"
  );
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi
    .replace(/<[^>]+>/g, " ");

  const perTerm = new Map();
  const perSurface = new Map();
  let highlights = 0;
  const ids = new Set();

  for (const m of text.matchAll(re)) {
    const surface = m[1].toLowerCase();
    const id = map.get(surface);
    if (!id) continue;
    const ut = perTerm.get(id) ?? 0;
    const us = perSurface.get(surface) ?? 0;
    if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) con
    perTerm.set(id, ut + 1);
    perSurface.set(surface, us + 1);
    highlights += 1;
    ids.add(id);
    used.add(id);
  }
}
```

```
        .sort();

    if (pages.length === 0) throw new Error("Не найдено ни одной страницы");

    ensureDir(OUT_DIR);

    const doc = new PDFDocument({
      size: [A4.width, A4.height],
      margin: 0,
      autoFirstPage: false,
      info: {
        Title: "Универсальное пособие – полный исходный код",
        Author: "CI: rebuild-pdf",
        Subject: "Экспорт всего кода проекта (код -> txt)",
        CreationDate: new Date(),
      },
    });

    const stream = fs.createWriteStream(PDF_PATH);
    doc.pipe(stream);

    for (const file of pages) {
      doc.addPage({ size: [A4.width, A4.height], margin: 0 });
      doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
    }

    doc.end();
    await new Promise((resolve, reject) => {
      stream.on("finish", resolve);
      stream.on("error", reject);
    });

    console.log("[3/3] png -> pdf");
    console.log(`    страниц: ${pages.length}`);
    console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
  }

  main().catch((err) => {
```

```
[/^src\/components\/visualizers\/, "Визуализаторы (в.  
[/^src\/components\/, "Интерактивные компоненты и сим  
[/^src\/layout\/, "HTML-разметка (layout)"],  
[/^src\/utils\/, "Утилиты"],  
[/^src\/, "Ядро приложения"],  
[/^scripts\/, "Скрипты сборки и экспорта"],  
[/^\.github\/, "CI / автоматизация"],  
[/^[^\/]+$/, "Конфигурация проекта"],  
];
```

```
const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel))
```

```
/** Порядок разделов в документе. */
```

```
const ORDER = [  
  "Конфигурация проекта",  
  "Ядро приложения",  
  "Контент и данные пособия",  
  "Билеты (учебный контент)",  
  "Интерактивные компоненты и симуляторы",  
  "Визуализаторы (вложенные)",  
  "HTML-разметка (layout)",  
  "Утилиты",  
  "Скрипты сборки и экспорта",  
  "CI / автоматизация",  
  "Прочее",  
];
```

```
function listFiles() {  
  let files;  
  try {  
    files = execFileSync("git", ["ls-files", "--cached"], {  
      cwd: ROOT,  
      encoding: "utf8",  
    })  
      .split("\n")  
      .filter(Boolean);  
  } catch {  
    files = [];  
  }  
}
```

```

    }
    return chapters.sort((a, b) => {
        const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
        const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
        return na - nb;
    }));
}

function main() {
    const files = listFiles();
    const chapters = extractChapters(files);
    ensureDir(OUT_DIR);

    const out = [];
    const push = (s = "") => out.push(s);

    const stamp = new Date().toLocaleString("ru-RU", { ti
    const totalBytes = files.reduce((s, f) => s + fs.stat

    push(HR);
    push("          УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТИ
    push("          ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
    push(HR);
    push();
    push(`Дата генерации:                ${stamp}`);
    push(`Файлов исходного кода:         ${files.length}`);
    push(`Суммарный объём кода:           ${humanSize(totalByt
    push(`Глав/билетов в пособии:        ${chapters.length}`)
    push();

    push(HR);
    push("          ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
    push(HR);
    chapters.forEach((c, i) => {
        push(`  ${String(i + 1).padStart(3, " ")}  ${c.titl
    });
    push();

```

```

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`      файлов:  ${files.length}`);
console.log(`      глав:    ${chapters.length}`);
console.log(`      строк:   ${txt.split("\n").length}`);
console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

ФАЙЛ 84/87: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

```

/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt      (collect-code.mjs)
 * txt  -> page-XXXX.png              (render-pages.mjs)
 * png  -> full_code_all_pages.pdf    (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

```

```
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore" });
      return cmd;
    } catch {
      /* пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
  );
}
```

ФАЙЛ 85/87: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 1.5 КБ

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT,
  PAGES_DIR,
  TXT_PATH,
  PAGE,
  ensureDir,
  humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки, чтобы
```

```

    const num = String(idx + 1).padStart(4, "0");
    const header = `Универсальное пособие • полный исходный код
    Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
    const body = [header, "-".repeat(PAGE.cols), ...lines];
    const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
    const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
    fs.writeFileSync(txtFile, body + "\n", "utf8");
    return { txtFile, pngFile, num };
  });

```

```

const args = (job) => [
  "-density", String(PAGE.density),
  "-page", PAGE.size,
  "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "Helvetica",
  "-pointsize", String(PAGE.pointsize),
  `text:${job.txtFile}`,
  "-background", "white",
  "-flatten",
  "-colorspace", "Gray",
  ...(PAGE.monochrome ? ["-monochrome"] : []),
  "-strip",
  "-define", "png:compression-level=9",
  job.pngFile,
];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`отрендерено ${done} страниц\n`);
    }
  }
}

```

```
  - "arena/**"
workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
```



```
-   "**"
paths-ignore:
  # чтобы коммит самого workflow не запускал бесконечный цикл
  - "public/export/**"
  - "**/*.md"
workflow_dispatch:
  inputs:
    density:
      description: "DPI растеризации страниц (по умолчанию 150)"
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-`${{ github.ref }}`
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm
```

```
echo "| --- | --- |"
echo "| \`public/export/full_code_all_pages"
echo "| \`public/export/full_code_all_pages"
echo "| PNG-страниц | ${ls tmp/export-pages.
} >> "$GITHUB_STEP_SUMMARY"
```

- name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7
with:
 name: full-code-export
 path: |
 public/export/full_code_all_pages.pdf
 public/export/full_code_all_pages.txt
 if-no-files-found: error
 retention-days: 30
- name: Upload artifacts (PNG-страницы)
uses: actions/upload-artifact@v7
with:
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7
- name: Commit rebuilt export back to the branch
run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action"
 git add -- public/export/full_code_all_pages.
 if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full_
 git push origin "HEAD:\${GITHUB_REF_NAME}"